Meal Ji app integration guide

Module by module, in the order the app developer integrates them. Each step lists what the app screens need, the call flow, every API with a request and a real response, the errors to handle, and the backend test results for that step.

API v1 · released steps 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12 · 12 Oct 2026. New steps are added to this guide as they are released.

Release steps

Step	Module	Screens	APIs	Tests
01	App start	Splash, Onboarding	2	11/11 passed
02	Sign in	Login, OTP	6	14/14 passed
03	Profile setup	Profile setup, Edit profile	6	10/10 passed
04	Location and addresses	Location, Addresses, Add new address, Update address	12	17/17 passed
05	Home	Home, About chef	8	24/24 passed
06	Menu and dish	Menu, Dish detail, Combos	11	22/22 passed
07	Search and favourites	Search, Favourites	9	22/22 passed
08	Cart and checkout	Cart, Checkout (Bill summary), Offers	12	28/28 passed
09	Place order and pay	Payment (Pay ₹X), Order confirmed, Payment methods	11	16/16 passed
10	Order tracking and history	Order confirmed, Order status, Tracking, Order delivered, Orders list, Order details	11	14/14 passed
11	Notifications	Notifications (and the bell on Home)	7	9/9 passed
12	Rewards and referrals	Rewards (Offers tab), Meal Ji points (View History), Refer a friend	11	17/17 passed

Backend tests for these steps: **204/204 passed** (run 2026-10-14 06:04 UTC against the API with demo data).

Basics for every call

Item	Value
Base URL	Local: `http://<server>:4000/api/v1`. Staging/production URLs are shared separately.
Headers	`Content-Type: application/json`, `x-device-id: <one id per install>`, and after sign-in `Authorization: Bearer <accessToken>`.
Success	`{ "success": true, "message": "…", "data": … }` — read `data`.
Error	`{ "success": false, "message": "…", "errors": [{ "field", "message" }] }` — show `message`; with `errors`, show each under its field.
Status codes	`400` bad request · `401` signed out (refresh, then Login) · `404` not found / not yours · `409` conflict · `422` form errors · `429` too many tries (wait) · `5xx` try again later.
Tokens	Access token ~15 min; refresh token keeps the user signed in for 60 days. Store both in secure storage.
Images	All image fields are full CDN URLs (CloudFront). Use them as they are; keep bundled images as a fallback.
Demo data	Customers `9000000031`–`9000000040`; OTP `123456` on the local/dev server. They never receive a real SMS.
Veg / All	The header toggle is remembered: flipping it calls `PATCH /users/me/preferences {"vegOnly": true|false}`, then reload. Every list (Home, Menu, Dishes, Combos, Search, Popular, suggestions) follows the saved choice by itself; `?veg=true|false` overrides it for one call. Responses return `vegOnly`.

Postman

Import `mealJiNode/postman/mealJiNode.postman_collection.json` and the local environment. Folders “Step 01 · App start” … are in this guide's order; each request has a description and saved example responses (success and errors). Login + Verify OTP store the tokens automatically.

Step 01 · App start

Screens: **Splash, Onboarding**

What the app screens need

- Splash: decide where to go on launch — force update / maintenance first, then a saved session (go to Home) or Onboarding / Login.
- Onboarding has 3 custom-drawn steps, not plain image slides: a chef mascot step, a 6-photo menu grid (Signature Bowls, Steamed Bao, Global Wraps, Street Snacks, Comfort Curries, Desserts) and a fresh step with a benefit strip (Hot Meals, Real Ingredients, Happier You).
- Each title has words drawn in the accent colour (“Meal Ji.”, “hero.”, “Delivered warm.”) and its own button text (“Show me the menu”, “Continue”, “Let's eat”).
- The copy uses the delivery promise (“25–35 minutes”, “pick it up in 15”), which now comes from App config.

Call flow

- On launch: `GET /app/config`. If `forceUpdate` (or the app version is below `minSupportedVersion`) show the update screen; if `maintenanceMode` show `maintenanceMessage`.
- If tokens are stored: `GET /auth/me` (step 2). 200 → Home; 401 → refresh, then Login.
- First open only: `GET /onboarding/slides` and draw each slide by its `layout`. Cache the response; it changes rarely.
- Keep the bundled images as a fallback so onboarding still works offline.

APIs in this step

#	API	Auth	Used for
S01.1	`GET /app/config`	Public	App config
S01.2	`GET /onboarding/slides`	Public	Onboarding slides

S01.1 · App config

`GET /api/v1/app/config`

Public. Call on every launch (Splash).

- `minSupportedVersion` / `forceUpdate`: block old app builds.
- `maintenanceMode` + `maintenanceMessage`: show a maintenance screen.
- `otpLength` (6) and `otpResendSeconds` (30): drive the OTP screen boxes and the resend timer.
- `countryCodes`: only `+91` today.
- `deliveryEtaLabel` (e.g. “25–35 min”), `pickupReadyMinutes` (15), `servedCitiesCount`: the copy on onboarding and home.
- `mealSlotsShown`: which meal slots the app shows.

Admin edits these in Settings → App.

Response · App config (200)


```
{
  "success": true,
  "message": "App config fetched.",
  "data": {
    "minSupportedVersion": "1.0.0",
    "latestVersion": "1.0.0",
    "forceUpdate": false,
    "maintenanceMode": false,
    "maintenanceMessage": "",
    "homeHeadline": "What are you craving today?",
    "searchPriceChipPaise": 20000,
    "searchPlaceholder": "Search for dishes, biryani, meals...",
    "deliveryPromiseLabel": "25–35 min",
    "mealSlotsShown": [
      "lunch"
    ],
    "featurePickup": false,
    "featureScheduledDelivery": true,
    "featureReferrals": false,
```

```
    "featureWallet": false,
    "supportPhone": "",
    "supportEmail": "",
    "deliveryEtaLabel": "25–35 min",
    "pickupReadyMinutes": 15,
    "otpLength": 6,
    "otpResendSeconds": 30,
    "countryCodes": [
      "+91"
    ],
    "servedCitiesCount": 1,
    "apiVersion": "v1",
    "businessTimezone": "Asia/Kolkata",
    "serverTime": "2026-10-14T05:59:13.988Z",
    "experiments": {}
  }
}
```


S01.2 · Onboarding slides

`GET /api/v1/onboarding/slides`

Public, cached 5 minutes. The onboarding screens in order (`sortOrder`).

Each slide: `slideId`, `layout`, `title`, `highlight`, `subtitle`, `ctaLabel`, `imageUrl`, `items`.

- `layout` tells the app which design to draw: `chef` (mascot), `menu` (6-photo category grid), `fresh` (benefit strip), or `basic` (image + text).
- `highlight` is the part of `title` drawn in the accent colour (it is always inside the title).
- `ctaLabel` is the button text (e.g. “Show me the menu”).
- `items`: for `menu` each has `title` + `imageUrl`; for `fresh` only `title` (Hot Meals, Real Ingredients, Happier You).

Images are CDN URLs; the admin manages slides in Content → Onboarding.

Response · Three onboarding steps (200)


```
{
  "success": true,
  "message": "Slides fetched.",
  "data": [
    {
      "slideId": "6acc8b…",
      "layout": "chef",
      "title": "Hey. I'm the chef at Meal Ji.",
      "highlight": "Meal Ji.",
      "subtitle": "One kitchen. One menu. Cooked and packed by the same people every time.",
      "ctaLabel": "Show me the menu",
      "imageUrl": "https://cdn…/mealji/seed/mascot-1024.png",
      "items": [],
      "sortOrder": 0,
      "isActive": true
    },
    {
      "slideId": "6acc8b…",
      "layout": "menu",
      "title": "A short menu. Every dish, a hero.",
      "highlight": "hero.",
      "subtitle": "Nine signature bowls, bao, wraps and desserts. No filler. No maybes.",
      "ctaLabel": "Continue",
      "imageUrl": "https://cdn…/mealji/seed/onboard2.png",
      "items": [
        {
          "title": "Signature Bowls",
          "imageUrl": "https://cdn…/mealji/seed/butter-chicken-bowl.jpg"
        },
        {
          "title": "Steamed Bao",
          "imageUrl": "https://cdn…/mealji/seed/paneer-bao.jpg"
        },
        {
          "title": "Global Wraps",
          "imageUrl": "https://cdn…/mealji/seed/tikka-wrap.jpg"
        },
        {
          "title": "Street Snacks",
          "imageUrl": "https://cdn…/mealji/seed/biryani-arancini.jpg"
        },
```

```
        {
          "title": "Comfort Curries",
          "imageUrl": "https://cdn…/mealji/seed/dal-ramen.jpg"
        },
        {
  …
```

Backend tests · 11/11 passed

Test	What the app needs	Result
01-01	**App config loads before sign-in** Splash decides update/maintenance before anything else	**Pass**
01-02	**Config gives the OTP rules the screens use** OTP screen: 6 boxes, 30-second resend timer, +91 prefix	**Pass**
01-03	**Config gives the delivery promise** Onboarding: “We deliver in 25–35 minutes, or you pick it up in 15”; Addresses: “We deliver to N cities”	**Pass**
01-04	**Config shows only the meal slots the app uses** App shows lunch only	**Pass**
01-05	**Onboarding has the app's 3 steps in order** Onboarding: chef → menu → fresh, each with title, subtitle, image and button	**Pass**
01-06	**Each step tells the app which layout to draw** Steps are custom layouts (chef, menu grid, fresh benefits)	**Pass**
01-07	**Menu step carries its 6-photo category grid** Onboarding menu step: Signature Bowls, Steamed Bao, Global Wraps, Street Snacks, Comfort Curries, Desserts	**Pass**
01-08	**Fresh step carries its benefit strip** Onboarding fresh step: Hot Meals / Real Ingredients / Happier You	**Pass**
01-09	**Accent words are marked** Onboarding titles colour some words (e.g. “Meal Ji.”, “hero.”)	**Pass**
01-10	**Every onboarding image loads** Images are remote CDN URLs	**Pass**
01-11	**Slides are cacheable** Shown on every fresh install; should be cheap	**Pass**

Step 02 · Sign in

Screens: **Login, OTP**

What the app screens need

- Login: one phone box with the +91 prefix. The placeholder says “Phone number or email” but sign-in is by mobile OTP only — an email now gets a clear message.
- OTP: 6 boxes, a 30-second resend countdown, a “wrong OTP” message.
- After OTP a new customer goes to Profile setup; a returning one skips it (and skips Location when it is already saved).
- The app's user model shows name, phone, points, tier and avatar.

Call flow

- Generate one `x-device-id` per install (e.g. a UUID kept in secure storage) and send it on every call.
- `POST /auth/login` → keep `userId`, start the timer from `resendAfterSec`, size the boxes from `otpLength`.
- `POST /auth/verify-otp` → store `accessToken` + `refreshToken` securely. Route on `profileComplete` and `user.currentLocation`.
- Send `Authorization: Bearer <accessToken>`. On 401: `POST /auth/refresh` once, retry the call; if refresh is 401, clear tokens and show Login.
- Profile → Log out: `POST /auth/logout`, then clear tokens.

APIs in this step

#	API	Auth	Used for
S02.1	`POST /auth/login`	Public	Login
S02.2	`POST /auth/resend-otp`	Public	Resend OTP
S02.3	`POST /auth/verify-otp`	Public	Verify OTP
S02.4	`POST /auth/refresh`	Public	Refresh
S02.5	`POST /auth/logout`	Token	Logout
S02.6	`GET /auth/me`	Token	Me

S02.1 · Login

`POST /api/v1/auth/login`

Public. Sends a 6-digit OTP to the mobile number; creates the account when the number is new.

`phoneNumber` may be typed with spaces, a leading 0 or +91 (e.g. “+91 90000 00031”); the server keeps the 10 digits. Only Indian mobile numbers (starting 6–9).

Returns `userId` (pass it to Verify OTP / Resend OTP), `otpLength`, `expiresInSec`, `resendAfterSec` (start the resend timer from this).

Errors: 422 with `errors[].field = phoneNumber` (show under the box). An email gets “Sign in with your 10-digit mobile number; email sign-in is not available”. 429 when too many tries.

Request body

```
{
  "countryCode": "+91",
  "phoneNumber": "{{phoneNumber}}"
}
```

Response · OTP sent (201)

```
{
  "success": true,
  "message": "OTP sent successfully.",
  "data": {
    "userId": "6ac4ba…",
    "message": "OTP sent successfully",
    "otpSent": true,
    "otpLength": 6,
```

```
    "expiresInSec": 300,
    "resendAfterSec": 30
  }
}
```

Errors to handle

Case	Status	Message shown to the user
Invalid number (422)	422	Validation failed (field: phoneNumber)
Email typed (422)	422	Validation failed (field: phoneNumber)

S02.2 · Resend OTP

`POST /api/v1/auth/resend-otp`

Public. Sends a new code for the same `userId`. Allowed once the timer from `resendAfterSec` (30 s) has run out; earlier calls get 429 “Please wait before requesting another OTP.” Returns the same fields as Login.

Request body

```
{
  "userId": "{{userId}}"
}
```

Errors to handle

Case	Status	Message shown to the user
Too soon (429)	429	Please wait before requesting another OTP.

S02.3 · Verify OTP

`POST /api/v1/auth/verify-otp`

Public. Header `x-device-id` is required (one stable id per install).

Returns `accessToken`, `refreshToken`, `expiresIn`, `isNewUser`, `profileComplete` and `user` (`name`, `phoneNumber`, `email`, `avatarUrl`, `points`, `tier`, `currentLocation`, …).

Where to go next:

- `profileComplete: false` → Profile setup (step 3).
- `user.currentLocation` is null → Location (step 4).
- otherwise → Home.

Errors: 400 “Invalid OTP.” (wrong code), 400 when `x-device-id` is missing, 429 after too many wrong codes.

Request body

```
{
  "userId": "{{userId}}",
  "otp": "{{otp}}"
}
```

Response · Returning customer (201)

```
{
  "success": true,
  "message": "Phone number verified successfully.",
  "data": {
    "userId": "6acc8b…",
    "accessToken": "eyJhbGciOiJIUzI1…",
    "refreshToken": "eyJhbGciOiJIUzI1…",
    "expiresIn": 3600,
    "isNewUser": false,
    "profileComplete": true,
    "user": {
      "userId": "6acc8b…",
      "name": "Rahul Sharma",
      "countryCode": "+91",
      "phoneNumber": "9000000031",
      "role": "user",
      "kitchenId": null,
      "permissions": [],
      "isActive": true,
      "isNumberVerified": true,
      "isSuspended": false,
```

```
      "suspendedAt": null,
      "subscription": {
        "planCode": "STANDARD_MONTHLY",
        "status": "active",
        "subscribedAt": "2026-10-12T18:30:00.000Z",
        "expiresAt": "2026-11-11T18:29:00.000Z"
      },
      "currentLocation": {
        "locationText": "WJVF+2RR, KHB Games Village, Koramangala, Bengaluru, Karnataka 560095, …",
        "area": "KHB Games Village",
        "city": "Bengaluru",
        "state": "Karnataka",
        "postalCode": "560095",
        "country": "India",
        "latitude": 12.942795,
        "longitude": 77.624478,
        "updatedAt": "2026-10-12T10:49:24.790Z"
      },
      "lastLoginAt": "2026-10-12T10:52:47.461Z",
      "createdAt": "2026-08-15T07:27:15.875Z",
      "email": null,
      "avatarUrl": "https://cdn…/mealji/seed/profile-avatar.jpg",
      "points": 1300,
      "tier": "Silver Tandoor",
      "deletedAt": null
  …
```


Response · New customer (goes to Profile setup) (201)


```
{
  "success": true,
  "message": "Phone number verified successfully.",
  "data": {
    "userId": "6ac8b1…",
    "accessToken": "eyJhbGciOiJIUzI1…",
    "refreshToken": "eyJhbGciOiJIUzI1…",
    "expiresIn": 3600,
    "isNewUser": true,
    "profileComplete": false,
    "user": {
      "userId": "6ac8b1…",
      "name": "User",
      "countryCode": "+91",
      "phoneNumber": "9000000099",
      "role": "user",
      "kitchenId": null,
      "permissions": [],
      "isActive": true,
      "isNumberVerified": true,
      "isSuspended": false,
      "suspendedAt": null,
      "subscription": {
        "planCode": null,
        "status": "none",
        "subscribedAt": null,
        "expiresAt": null
      },
      "currentLocation": null,
      "lastLoginAt": "2026-10-09T09:18:45.655Z",
      "createdAt": "2026-10-09T09:18:45.630Z",
      "email": null,
      "avatarUrl": null,
      "points": 0,
      "tier": null,
      "deletedAt": null
    },
    "message": "Phone number verified successfully"
  }
}
```


Errors to handle

Case	Status	Message shown to the user
Wrong code (400)	400	Invalid OTP.

S02.4 · Refresh

`POST /api/v1/auth/refresh`

Public. Call when any API returns 401 with an expired access token. Send the saved `refreshToken` and the same `x-device-id`. Returns a new pair; store both (the old refresh token stops working). 401 means the session ended: go to Login.

Request body

```
{
  "refreshToken": "{{refreshToken}}",
  "deviceId": "{{deviceId}}"
}
```

Response · New token pair (200)

```
{
  "success": true,
  "message": "Token refreshed successfully.",
  "data": {
    "userId": "6ac4ba…",
    "accessToken": "eyJhbGciOiJIUzI1…",
    "refreshToken": "eyJhbGciOiJIUzI1…",
    "expiresIn": 3600
  }
}
```

Errors to handle

Case	Status	Message shown to the user
Session ended (401)	401	Invalid or expired token

S02.5 · Logout

`POST /api/v1/auth/logout`

Authenticated. Profile → Log out. Ends this device's session (access and refresh token). Clear the stored tokens afterwards.

Response · Signed out (200)

```
{
  "success": true,
  "message": "Logged out successfully.",
  "data": {}
}
```

S02.6 · Me

`GET /api/v1/auth/me`

Authenticated. Use on launch to check the saved session: 200 → signed in; 401 → refresh, then Login. Returns the user, role and permission keys.

Response · Signed in (200)

```
{
  "success": true,
  "message": "Profile fetched successfully.",
  "data": {
    "userId": "6acc8b…",
    "name": "Rahul Sharma",
    "countryCode": "+91",
    "phoneNumber": "9000000031",
    "role": "user",
    "kitchenId": null,
    "permissions": [],
    "isActive": true,
    "isNumberVerified": true,
    "isSuspended": false,
    "suspendedAt": null,
    "subscription": {
      "planCode": "STANDARD_MONTHLY",
      "status": "active",
      "subscribedAt": "2026-10-12T18:30:00.000Z",
      "expiresAt": "2026-11-11T18:29:00.000Z"
    },
```

```
    "currentLocation": {
      "locationText": "WJVF+2RR, KHB Games Village, Koramangala, Bengaluru, Karnataka 560095, …",
      "area": "KHB Games Village",
      "city": "Bengaluru",
      "state": "Karnataka",
      "postalCode": "560095",
      "country": "India",
      "latitude": 12.942795,
      "longitude": 77.624478,
      "updatedAt": "2026-10-12T10:55:30.070Z"
    },
    "lastLoginAt": "2026-10-14T05:59:13.647Z",
    "createdAt": "2026-08-15T07:27:15.875Z",
    "email": null,
    "avatarUrl": "https://cdn…/mealji/seed/profile-avatar.jpg",
    "points": 1300,
    "tier": "Silver Tandoor",
    "deletedAt": null
  }
}
```


Backend tests · 14/14 passed

Test	What the app needs	Result
02-01	**Short or wrong phone is refused with a field message** Login: phone input	Pass
02-02	**An email in the phone box gets a clear message** Login placeholder says “Phone number or email”; sign-in is by mobile OTP	Pass
02-03	**Phone with spaces or a +91 prefix still works** Users type “90000 00031” or “+91 9000000031”	Pass
02-04	**Send OTP returns what the OTP screen needs** OTP screen: 6 boxes, 30-second resend timer	Pass
02-05	**Resend before the timer ends is refused** OTP screen: resend only after the countdown	Pass
02-06	**Wrong code is refused** OTP screen: wrong OTP message	Pass
02-07	**Verify needs the device id** Sessions are per device	Pass
02-08	**New customer: tokens + “new user, profile incomplete”** After OTP a new customer goes to Profile setup	Pass
02-09	**Returning customer: not new, profile complete, location known** Returning users skip Profile setup and Location	Pass
02-10	**Signed-in user carries name, phone, points, tier and avatar** App user model: name, phone, points, tier, avatar	Pass
02-11	**Who-am-I works with the token, not without** App restores the session on launch	Pass
02-12	**Refresh gives a new pair; a junk token is refused** Keep the user signed in for 60 days	Pass
02-13	**Logout ends the session** Profile: Log out	Pass
02-14	**Changing letter case never skips sign-in** Security: protected screens need a token	Pass

Step 03 · Profile setup

Screens: **Profile setup, Edit profile**

What the app screens need

- Profile setup: “Complete Profile” with full name (min 2 characters) and optional email, then Continue.
- Edit profile: name, phone (read-only), email, date of birth, gender (“Male”, “Female”, “Other”, “Prefer not to say”), change photo.
- Profile header: name, photo, points, tier and counts (orders, favourites, addresses, payment methods).

Call flow

- Profile setup → `PATCH /users/me` with `name` (and `email`). The response has `profileComplete: true`; continue to Location.
- Edit profile loads `GET /users/me`, saves with `PATCH /users/me` (only changed fields). Show `errors[].message` under the matching field.
- Photo: `POST /uploads/presign` → PUT the file bytes to `uploadUrl` with the returned `headers` → `POST /users/me/avatar` with `fileUrl`.

APIs in this step

#	API	Auth	Used for
S03.1	`GET /users/me`	Token	My profile
S03.2	`PATCH /users/me`	Token	Save profile
S03.3	`POST /uploads/presign`	Token	Photo upload URL
S03.4	`POST /users/me/avatar`	Token	Save photo
S03.5	`GET /users/me/preferences`	Token	Preferences
S03.6	`PATCH /users/me/preferences`	Token	Save preferences

S03.1 · My profile

`GET /api/v1/users/me`

Authenticated. Everything the profile screens show: `name`, `phoneNumber`, `email`, `dob`, `gender`, `avatarUrl`, `points`, `tier`, `profileComplete`, `currentLocation`, and `stats` (`orders`, `favorites`, `addresses`, `paymentMethods`) for the profile header.

Response · Profile (200)


```
{
  "success": true,
  "message": "Profile fetched.",
  "data": {
    "userId": "6acc8b…",
    "name": "Rahul Sharma",
    "countryCode": "+91",
    "phoneNumber": "9000000031",
    "email": null,
    "emailVerified": false,
    "dob": null,
    "gender": "male",
    "avatarUrl": "https://cdn…/mealji/seed/profile-avatar.jpg",
    "profileComplete": true,
    "referralCode": "RAHUB035",
    "points": 1300,
    "tier": "Silver Tandoor",
    "preferences": {
      "language": "en",
      "vegOnly": false,
      "channels": {
        "push": true,
        "whatsapp": false,
        "email": true,
        "sms": true
      },
      "topics": {
        "offers": true,
```

```
        "rewards": true,
        "account": true
      }
    },
    "subscription": {
      "planCode": "STANDARD_MONTHLY",
      "status": "active",
      "expiresAt": "2026-11-11T18:29:00.000Z"
    },
    "currentLocation": {
      "locationText": "WJVF+2RR, KHB Games Village, Koramangala, Bengaluru, Karnataka 560095, …",
      "area": "KHB Games Village",
      "city": "Bengaluru",
      "state": "Karnataka",
      "postalCode": "560095",
      "country": "India",
      "latitude": 12.942795,
      "longitude": 77.624478,
  …
```


S03.2 · Save profile

`PATCH /api/v1/users/me`

Authenticated. Send only the fields that changed. Used by Profile setup (name, optional email) and Edit profile.

- `name`: 2–80 characters ("Please enter your full name (min 2 characters)").
- `email`: valid email or null.
- `dob`: `YYYY-MM-DD`, not in the future.
- `gender`: `male`, `female`, `other`, `prefer_not` (the app's "Male" / "Prefer not to say" are accepted too).

Returns the full profile; `profileComplete` turns true once a real name is set. 422 errors carry `errors[].field`.

Request body


```
{
  "name": "Rahul Sharma",
  "email": "rahul.sharma@example.com",
  "dob": "1995-10-12",
  "gender": "Male"
}
```


Response · Saved (200)


```
{
  "success": true,
  "message": "Profile updated.",
  "data": {
    "userId": "6acc8b…",
    "name": "Rahul Sharma",
    "countryCode": "+91",
    "phoneNumber": "9000000031",
    "email": "rahul@example.com",
    "emailVerified": false,
    "dob": "1994-05-21",
    "gender": "male",
    "avatarUrl": "https://cdn…/mealji/seed/profile-avatar.jpg",
    "profileComplete": true,
    "referralCode": "RAHUB035",
    "points": 1300,
    "tier": "Silver Tandoor",
    "preferences": {
      "language": "en",
      "vegOnly": false,
      "channels": {
        "push": true,
        "whatsapp": false,
        "email": true,
        "sms": true
      },
      "topics": {
        "offers": true,
        "rewards": true,
        "account": true
      }
    },
    "subscription": {
      "planCode": "STANDARD_MONTHLY",
```

```
      "status": "active",
      "expiresAt": "2026-11-11T18:29:00.000Z"
    },
    "currentLocation": {
      "locationText": "WJVF+2RR, KHB Games Village, Koramangala, Bengaluru, Karnataka 560095, …",
      "area": "KHB Games Village",
      "city": "Bengaluru",
      "state": "Karnataka",
      "postalCode": "560095",
      "country": "India",
      "latitude": 12.942795,
      "longitude": 77.624478,
  …
```


Errors to handle

Case	Status	Message shown to the user
Name too short (422)	422	Please enter your full name (min 2 characters) (field: name)
Bad email (422)	422	Enter a valid email address (field: email)
Wrong date of birth (422)	422	Enter a valid date of birth (field: dob)

S03.3 · Photo upload URL

`POST /api/v1/uploads/presign`

Authenticated. Step 1 of changing the photo. `purpose: avatar`, `contentType` image/jpeg | image/png | image/webp, `size` in bytes (max 2 MB). Returns `uploadUrl`, `headers` and `fileUrl`. Then PUT the raw file bytes to `uploadUrl` with exactly those headers (no Authorization header), and send `fileUrl` to Save photo. The upload URL expires in a few minutes.

Request body


```
{
  "purpose": "avatar",
  "contentType": "image/jpeg",
  "size": 120000
}
```


Response · Upload URL (201)


```
{
  "success": true,
  "message": "Upload URL created.",
  "data": {
    "key": "mealji/avatar/6acc8bd3189320566a1fa95b/mv54v79b-389d1d37573e.jpg",
    "method": "PUT",
    "uploadUrl": "https://yoraaecommerce.s3.ap-south-1.amazonaws.com/mealji/avatar/6acc8b…",
    "headers": {
      "Content-Type": "image/jpeg"
    },
    "fileUrl": "https://cdn…/mealji/avatar/6acc8bd3189320566a1fa95b/mv54v79b-389d1d3757…",
    "maxBytes": 2097152,
    "expiresAt": "2026-10-12T11:02:47.615Z"
  }
}
```


Errors to handle

Case	Status	Message shown to the user
Not an image (422)	422	Allowed file types: image/jpeg, image/png, image/webp

S03.4 · Save photo

`POST /api/v1/users/me/avatar`

Authenticated. Step 3 of changing the photo: send the `fileUrl` from the upload step after the PUT succeeded. Only URLs from our own uploads are accepted. Returns the profile with the new `avatarUrl`.

Request body

```
{
  "avatarUrl": "{{fileUrl}}"
}
```

Errors to handle

Case	Status	Message shown to the user
Not our upload (422)	422	Avatar must be uploaded through MealJi

S03.5 · Preferences

`GET /api/v1/users/me/preferences`

Authenticated. `vegOnly`, `language` (en | hi), notification `channels` and `topics`.

Response · Preferences (200)

```
{
  "success": true,
  "message": "Preferences fetched.",
  "data": {
    "language": "en",
    "vegOnly": false,
    "channels": {
      "push": true,
      "whatsapp": false,
      "email": true,
      "sms": true
    },
    "topics": {
      "offers": true,
      "rewards": true,
      "account": true
    },
    "consents": []
  }
}
```

S03.6 · Save preferences

`PATCH /api/v1/users/me/preferences`

Authenticated. Send only what changed. Unknown language → 422.

Veg / All toggle: when the user flips the switch on Home, send `{ "vegOnly": true|false }` here, then reload the screen. Every list (Home, Menu, Dishes, Combos, Search, Popular, suggestions) follows this saved choice.

Request body

```
{
  "vegOnly": false,
  "language": "en"
}
```

Errors to handle

Case	Status	Message shown to the user
Unknown language (422)	422	Validation failed (field: language)

Backend tests · 10/10 passed

Test	What the app needs	Result
03-01	**Profile has everything the profile screens show** Edit profile: name, phone, email, date of birth, gender, photo; header: points and tier	Pass
03-02	**Profile header counts come with the profile** Profile header: orders, favourites, addresses, payment methods	Pass
03-03	**Name shorter than 2 characters is refused** Profile setup: “Please enter your full name (min 2 characters)”	Pass
03-04	**Bad email and future birthday are refused** Edit profile validation	Pass
03-05	**Gender as the app writes it (“Male”) is accepted** Edit profile shows gender as “Male“	Pass
03-06	**Saving name, email and birthday returns the updated profile** Edit profile: Save changes	Pass
03-07	**Photo upload: presign, upload, save, shows on the profile** Edit profile: change photo	Pass
03-08	**Wrong photo type or size is refused** Edit profile: only images, max 2 MB	Pass
03-09	**Preferences save and come back** Profile: veg-only and language	Pass
03-10	**New customer: setting a name completes the profile** Profile setup: “Complete Profile” → Continue	Pass

Step 04 · Location and addresses

Screens: **Location, Addresses, Add new address, Update address**

What the app screens need

- Location: “Use current location”, a search box, and a “we don't deliver here yet” state.
- Addresses: cards labelled Home / Office / Other or a custom name (e.g. “Parents' Home”), one default, Edit and Delete.
- Add new address form: Full name, Phone, Pincode, House/Flat, Landmark, City, State and the Home / Office / Other chips — no map pin.
- Update address: the same form, pre-filled.

Call flow

- “Use current location”: get the GPS point, `PUT /users/me/location`; read `data.serviceability.serviceable`. Yes → Home. No → show `serviceability.message` and let the user search or pick an address.
- Search: `GET /geo/autocomplete?input=…&sessionToken=…` while typing (debounce 300 ms) → `GET /geo/place/{placeId}` on tap → `PUT /users/me/location` with its point.
- Addresses screen: `GET /users/me/addresses`. Print `displayLabel` and `fullAddress`; grey out cards with `serviceable: false`.
- Add address: `POST /users/me/addresses` with the form fields and the chip text as `label`. Send `latitude/longitude` only if the user pinned a map.
- Edit: `PATCH /users/me/addresses/{addressId}`. Choose for delivery: `PATCH …/{addressId}/default`. Delete: `DELETE …/{addressId}`.

APIs in this step

#	API	Auth	Used for
S04.1	`GET /geo/autocomplete?input=Koramangala 6th block&latitude=12.93&longitude=77.62&sessionToken={{sessionToken}}`	Token	Place search
S04.2	`GET /geo/place/{{placeId}}?sessionToken={{sessionToken}}`	Token	Place details
S04.3	`GET /serviceability?latitude=12.9568&longitude=77.6349`	Token	Delivery check by point
S04.4	`GET /serviceability/pincode/{{pincode}}`	Token	Delivery check by pincode
S04.5	`PUT /users/me/location`	Token	Update current location
S04.6	`GET /users/me/location`	Token	Get current location
S04.7	`GET /users/me/serviceability?latitude=12.9352&longitude=77.6245`	Token	Check serviceability
S04.8	`GET /users/me/addresses`	Token	My addresses
S04.9	`POST /users/me/addresses`	Token	Add address
S04.10	`PATCH /users/me/addresses/{{addressId}}`	Token	Update address
S04.11	`PATCH /users/me/addresses/{{addressId}}/default`	Token	Make default
S04.12	`DELETE /users/me/addresses/{{addressId}}`	Token	Delete address

S04.1 · Place search

`GET /api/v1/geo/autocomplete?input=Koramangala 6th block&latitude=12.93&longitude=77.62&sessionToken={{sessionToken}}`

Authenticated. The Location search box. Call while the user types (2+ characters, debounce ~300 ms).

- `input`: the typed text.
- `latitude`/`longitude` (optional): rank places near the user first.
- `sessionToken`: a new UUID when the user starts typing; reuse it for every keystroke and for Place details, then start a new one (Google bills one session).

Returns up to 8 suggestions: `placeId`, `primaryText` (bold line), `secondaryText` (grey line), `description`. No match → empty list.

Response · Suggestions (200)


```
{
  "success": true,
  "message": "Suggestions fetched.",
  "data": [
    {
      "placeId": "ChIJE5dLT0QUrjsRZcDy37bywDQ",
      "primaryText": "6th Block Koramangala, Koramangala",
      "secondaryText": "Bengaluru, Karnataka, India",
      "description": "6th Block Koramangala, Koramangala, Bengaluru, Karnataka, India"
    },
    {
      "placeId": "ChIJ9VrZJAAVrjsR66B5sAyA5xw",
      "primaryText": "Koramangala 6th block",
      "secondaryText": "1st Block Koramangala, Koramangala, Bengaluru, Karnataka, India",
      "description": "Koramangala 6th block, 1st Block Koramangala, Koramangala, Bengaluru, K…"
    },
    {
      "placeId": "ChIJUcKeIl4UrjsR1lvnxF7I5-8",
      "primaryText": "Apollo Pharmacy Koramangala 6th Block",
      "secondaryText": "6th Cross Road, 6th Block, Koramangala, Bengaluru, Karnataka, India",
      "description": "Apollo Pharmacy Koramangala 6th Block, 6th Cross Road, 6th Block, Koram…"
    },
    {
      "placeId": "ChIJlZbvPwAVrjsRM7RbDp6YrRs",
      "primaryText": "Koramangala",
      "secondaryText": "7th Cross Road, KHB Colony, 6th Block, Koramangala, Bengaluru, Karnatak…",
      "description": "Koramangala, 7th Cross Road, KHB Colony, 6th Block, Koramangala, Bengal…"
    },
    {
      "placeId": "EkE4MCBGZWV0IFJvYWQsIDZ0aCBCbG9jaywgS29yYW1hbmdhbGEsIEJlbmdhbHVydSwgS2F…",
      "primaryText": "80 Feet Road",
      "secondaryText": "6th Block, Koramangala, Bengaluru, Karnataka, India",
      "description": "80 Feet Road, 6th Block, Koramangala, Bengaluru, Karnataka, India"
    }
  ]
}
```


S04.2 · Place details

`GET /api/v1/geo/place/{{placeId}}?sessionToken={{sessionToken}}`

Authenticated. When the user taps a suggestion. Pass its `placeId` and the same `sessionToken`.

Returns `name`, `fullAddress`, `locality`, `city`, `state`, `pincode` (may be null for an area), `latitude`, `longitude` and `serviceability` (same as the location call).

Then save it as the current location with `PUT /users/me/location`, or pre-fill the Add new address form with these fields and coordinates. Unknown id → 404.

Response · Picked place (200)


```
{
  "success": true,
  "message": "Place fetched.",
  "data": {
    "placeId": "ChIJE5dLT0QUrjsRZcDy37bywDQ",
    "name": "6th Block",
    "fullAddress": "6th Block, Koramangala, Bengaluru, Karnataka, India",
    "locality": "6th Block",
    "city": "Bengaluru",
```

```
    "state": "Karnataka",
    "pincode": null,
    "latitude": 12.939670200000002,
    "longitude": 77.62223,
    "serviceability": {
      "serviceable": true,
      "distanceKm": 0.75,
      "etaMinutes": 28,
      "etaLabel": "23–33 min",
      "deliveryFeePaise": 4000,
      "deliveryFeeMode": "flat",
      "freeDeliveryAbovePaise": 29900,
      "minOrderPaise": 0,
      "codEnabled": true,
      "pickupEnabled": false,
      "isOpenNow": true,
      "reason": null,
      "opensAt": null,
      "message": null,
      "kitchen": {
        "kitchenId": "6ac4ba…",
        "name": "Koramangala Kitchen",
        "serviceRadiusKm": 3,
        "acceptingOrders": true,
        "opensAt": "08:00",
        "closesAt": "23:00",
        "area": "Koramangala",
        "city": "Bengaluru",
        "ratingAvg": 4.7,
        "ratingCount": 117
      },
      "alternatives": []
    }
  }
}
```


Errors to handle

Case	Status	Message shown to the user
Unknown place (404)	404	Place not found

S04.3 · Delivery check by point

`GET /api/v1/serviceability?latitude=12.9568&longitude=77.6349`

Authenticated. Checks a point without saving it (e.g. while dragging the map pin). `latitude` and `longitude` are required.

Response · Delivered here (200)


```
{
  "success": true,
  "message": "Serviceability checked.",
  "data": {
    "serviceable": true,
    "distanceKm": 1.81,
    "etaMinutes": 32,
    "etaLabel": "27–37 min",
    "deliveryFeePaise": 4000,
    "deliveryFeeMode": "flat",
    "freeDeliveryAbovePaise": 29900,
    "minOrderPaise": 0,
    "codEnabled": true,
    "pickupEnabled": false,
    "isOpenNow": true,
    "reason": null,
    "opensAt": null,
    "message": null,
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Indiranagar Kitchen",
      "serviceRadiusKm": 3,
      "acceptingOrders": true,
      "opensAt": "08:00",
      "closesAt": "23:00",
      "area": "Indiranagar",
      "city": "Bengaluru",
      "ratingAvg": 5,
      "ratingCount": 33
```

```
    },
    "alternatives": [
      {
        "kitchenId": "6acc8b…",
        "name": "Koramangala Kitchen",
        "area": "Koramangala",
        "city": "Bengaluru",
        "distanceKm": 2.65,
        "isOpenNow": true,
        "ratingAvg": 4.6,
        "ratingCount": 25
      }
    ]
  }
}
```

S04.4 · Delivery check by pincode

`GET /api/v1/serviceability/pincode/{{pincode}}`

Authenticated. For the manual pincode box. Returns the same `serviceable`, `kitchen`, `etaLabel`, `message` as the location call. 422 for anything but 6 digits.

Response · Pincode served (200)

```
{
  "success": true,
  "message": "Serviceability checked.",
  "data": {
    "serviceable": true,
    "distanceKm": 0.57,
    "etaMinutes": 27,
    "etaLabel": "22–32 min",
    "deliveryFeePaise": 4000,
    "deliveryFeeMode": "flat",
    "freeDeliveryAbovePaise": 29900,
    "minOrderPaise": 0,
    "codEnabled": true,
    "pickupEnabled": false,
    "isOpenNow": true,
    "reason": null,
    "opensAt": null,
    "message": null,
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen",
      "serviceRadiusKm": 3,
      "acceptingOrders": true,
      "opensAt": "08:00",
      "closesAt": "23:00",
      "area": "Koramangala",
      "city": "Bengaluru",
      "ratingAvg": 4.6,
      "ratingCount": 25
    },
    "alternatives": []
  }
}
```

S04.5 · Update current location

`PUT /api/v1/users/me/location`

Authenticated. The one call the app makes for location: send the GPS point; the backend saves it (and looks up the address text with Google Maps when GOOGLE_MAPS_API_KEY is set) and returns data.serviceability in the same response.

Kitchen choice (all on the backend): every active kitchen whose service radius covers the point is found, nearest first. The nearest one that can take orders now (open and accepting orders) is chosen; if none can, the nearest one is chosen and isOpenNow is false. The other kitchens in range are listed in data.serviceability.alternatives (for a Switch kitchen option: send ?kitchenId= on menu calls).

The status is 200 either way; read data.serviceability.serviceable. Outside every radius: serviceable false, kitchen null, and the point is logged as demand.

Sample point 12.9568, 77.6349 lies between Indiranagar and Koramangala (both 3 km radius), so both cover it. The test script stores the chosen kitchen in {{servingKitchenId}}.

Request body

```
{
  "latitude": 12.9568,
  "longitude": 77.6349
}
```

Response · Serviceable (200)


```
{
  "success": true,
  "message": "Location updated. We deliver here.",
  "data": {
    "locationText": "WJVF+2RR, KHB Games Village, Koramangala, Bengaluru, Karnataka 560095, …",
    "area": "KHB Games Village",
    "city": "Bengaluru",
    "state": "Karnataka",
    "postalCode": "560095",
    "country": "India",
    "latitude": 12.942795,
    "longitude": 77.624478,
    "updatedAt": "2026-10-12T10:52:47.641Z",
    "serviceability": {
      "serviceable": true,
      "distanceKm": 0.84,
      "etaMinutes": 28,
      "etaLabel": "23–33 min",
      "deliveryFeePaise": 4000,
      "deliveryFeeMode": "flat",
      "freeDeliveryAbovePaise": 29900,
      "minOrderPaise": 0,
      "codEnabled": true,
      "pickupEnabled": false,
      "isOpenNow": true,
      "reason": null,
      "opensAt": null,
      "message": null,
      "kitchen": {
        "kitchenId": "6acc8b…",
        "name": "Koramangala Kitchen",
        "serviceRadiusKm": 3,
        "acceptingOrders": true,
        "opensAt": "08:00",
        "closesAt": "23:00",
        "area": "Koramangala",
        "city": "Bengaluru",
        "ratingAvg": 4.6,
        "ratingCount": 25
      },
      "alternatives": []
    }
  }
}
```


Response · Not delivered here (200)


```
{
  "success": true,
  "message": "Location updated. We don't deliver here yet.",
  "data": {
    "locationText": "Kempegowda International Airport Bengaluru (BLR), Devanahalli, Bengalur…",
    "area": "Devanahalli",
    "city": "Bengaluru",
    "state": "Karnataka",
    "postalCode": "560300",
    "country": "India",
    "latitude": 13.1986,
    "longitude": 77.7066,
    "updatedAt": "2026-10-12T10:52:47.666Z",
    "serviceability": {
      "serviceable": false,
      "reason": "not_serviceable",
      "kitchen": null,
      "distanceKm": null,
      "message": "We don't deliver here yet. We'll let you know when we do."
    }
```

```
  }
}
```

S04.6 · Get current location

`GET /api/v1/users/me/location`

Authenticated. Returns the saved textual location.

Response · Saved location (200)

```
{
  "success": true,
  "message": "Current location fetched.",
  "data": {
    "locationText": "WJVF+2RR, KHB Games Village, Koramangala, Bengaluru, Karnataka 560095, …",
    "area": "KHB Games Village",
    "city": "Bengaluru",
    "state": "Karnataka",
    "postalCode": "560095",
    "country": "India",
    "latitude": 12.942795,
    "longitude": 77.624478,
    "updatedAt": "2026-10-12T10:55:30.070Z"
  }
}
```

S04.7 · Check serviceability

`GET /api/v1/users/me/serviceability?latitude=12.9352&longitude=77.6245`

Authenticated. Optional: Update current location already returns the same result. Use this to re-check without saving: it uses these coordinates, or the saved location when latitude and longitude are omitted. Same kitchen choice as Update current location (nearest kitchen that can order now, else nearest), with data.alternatives listing the other kitchens in range. Inside a radius the message is Serviceable and data.kitchen is set; outside every radius the status is still 200, the message is Not serviceable, and data.kitchen is null.

Response · Serviceable (200)

```
{
  "success": true,
  "message": "Serviceable",
  "data": {
    "serviceable": true,
    "distanceKm": 0,
    "etaMinutes": 25,
    "etaLabel": "20–30 min",
    "deliveryFeePaise": 4000,
    "deliveryFeeMode": "flat",
    "freeDeliveryAbovePaise": 29900,
    "minOrderPaise": 0,
    "codEnabled": true,
    "pickupEnabled": false,
    "isOpenNow": true,
    "reason": null,
    "opensAt": null,
    "message": null,
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen",
      "serviceRadiusKm": 3,
      "acceptingOrders": true,
      "opensAt": "08:00",
      "closesAt": "23:00",
      "area": "Koramangala",
      "city": "Bengaluru",
      "ratingAvg": 4.6,
      "ratingCount": 25
    },
    "alternatives": []
  }
}
```

S04.8 · My addresses

`GET /api/v1/users/me/addresses`

Authenticated. Default address first, then the most recently changed. Each has `addressId`, `label`, `customLabel`, `displayLabel`, `recipientName`, `phone`, the address parts, `fullAddress`, `latitude`, `longitude`, `isDefault`, `serviceable`, `kitchenId`, `distanceKm`. Show a "not deliverable" note when `serviceable` is false.

Response · Addresses (200)


```
{
  "success": true,
  "message": "Addresses fetched.",
  "data": [
    {
      "addressId": "6acc8b…",
      "label": "home",
      "customLabel": null,
      "displayLabel": "Home",
      "recipientName": "Rahul Sharma",
      "phone": "9000000031",
      "houseFlat": "Flat 302, Lakeview Residency",
      "street": "5th Cross, 6th Block",
      "locality": "Koramangala",
      "landmark": null,
      "city": "Bengaluru",
      "state": "Karnataka",
      "pincode": "560095",
      "latitude": 12.9412,
      "longitude": 77.6205,
      "fullAddress": "Flat 302, Lakeview Residency, 5th Cross, 6th Block, Koramangala, Bengal…",
      "isDefault": true,
      "updatedAt": "2026-10-12T10:52:47.806Z",
      "serviceable": true,
      "kitchenId": "6acc8b…",
      "distanceKm": 0.8
    }
  ]
}
```


S04.9 · Add address

`POST /api/v1/users/me/addresses`

Authenticated. The Add new address form. Required: `houseFlat`, `city`, `pincode` (6 digits). Optional: `recipientName`, `phone`, `street`, `locality`, `landmark`, `state`, `isDefault`.

- `label`: the chip — `Home`, `Office` (stored as `work`) or `Other`; any other text (e.g. "Parents' Home") is kept as `customLabel`. The card shows `displayLabel`.
- `latitude`/`longitude`: optional. Send them when the user pinned the map; without them the server finds the address from its text, then from the pincode. If it cannot be found: 422 on `pincode` "We couldn't find this address. Check the pincode or pin it on the map."
- The first address becomes the default.

Returns the address with `addressId`, `displayLabel`, `fullAddress`, `serviceable`, `kitchenId`, `distanceKm`. Up to 20 addresses (409 after that).

`phone` may be typed as "98765 43210", "+91 98765 43210" or "098765-43210" (the 10 digits are kept, like sign-in). The landmark appears in `fullAddress` exactly as typed ("Near Sony Signal", "Opposite Forum Mall").

Request body


```
{
  "label": "Office",
  "recipientName": "Rahul Sharma",
  "phone": "9000000031",
  "houseFlat": "3rd Floor, 80 Feet Road",
  "landmark": "Near Sony Signal",
  "city": "Bengaluru",
  "state": "Karnataka",
  "pincode": "560034"
}
```


Response · From the form (no map pin) (201)

```
{
  "success": true,
  "message": "Address saved.",
  "data": {
    "addressId": "6acf1a…",
    "label": "work",
    "customLabel": null,
    "displayLabel": "Office",
    "recipientName": "Rahul Sharma",
    "phone": "9000000031",
    "houseFlat": "3rd Floor, 80 Feet Road",
    "street": null,
    "locality": null,
    "landmark": "Near Sony Signal",
    "city": "Bengaluru",
    "state": "Karnataka",
    "pincode": "560034",
    "latitude": 12.9261382,
    "longitude": 77.62210910000002,
    "fullAddress": "3rd Floor, 80 Feet Road, Near Sony Signal, Bengaluru, Karnataka, 560034",
    "isDefault": false,
    "updatedAt": "2026-10-14T05:59:15.110Z",
    "serviceable": true,
    "kitchenId": "6acc8b…",
    "distanceKm": 1.04
  }
}
```


Response · Custom label with map pin (201)


```
{
  "success": true,
  "message": "Address saved.",
  "data": {
    "addressId": "6accbb…",
    "label": "other",
    "customLabel": "Parents' Home",
    "displayLabel": "Parents' Home",
    "recipientName": null,
    "phone": "9876543210",
    "houseFlat": "No. 14, 2nd Floor",
    "street": "17th Main, 3rd Block",
    "locality": "Koramangala",
    "landmark": "Opposite Forum Mall",
    "city": "Bengaluru",
    "state": "Karnataka",
    "pincode": "560034",
    "latitude": 12.9346,
    "longitude": 77.6311,
    "fullAddress": "No. 14, 2nd Floor, 17th Main, 3rd Block, Koramangala, Opposite Forum Ma…",
    "isDefault": false,
    "updatedAt": "2026-10-12T10:52:47.701Z",
    "serviceable": true,
    "kitchenId": "6acc8b…",
    "distanceKm": 0.72
  }
}
```


Errors to handle

Case	Status	Message shown to the user
Missing fields (422)	422	Please enter your house / flat (field: houseFlat, pincode)
Address not found (422)	422	We couldn't find this address. Check the pincode or pin it on the map. (field: pincode)

S04.10 · Update address

`PATCH /api/v1/users/me/addresses/{{addressId}}`

Authenticated. Update address screen: send only the changed fields (same rules as Add address). Changing the address text without a new map pin finds the location again. 404 for an address that is not yours.

Request body

```
{
  "recipientName": "Mom",
  "landmark": "Opposite Forum Mall"
}
```

Response · Updated (200)

```
{
  "success": true,
  "message": "Address updated.",
  "data": {
    "addressId": "6accbb…",
    "label": "other",
    "customLabel": "Parents' Home",
    "displayLabel": "Parents' Home",
    "recipientName": "Rahul Sharma",
    "phone": "9876543210",
    "houseFlat": "No. 14, 2nd Floor",
    "street": "17th Main, 3rd Block",
    "locality": "Koramangala",
    "landmark": "Behind the temple",
    "city": "Bengaluru",
    "state": "Karnataka",
    "pincode": "560034",
    "latitude": 12.9346,
    "longitude": 77.6311,
    "fullAddress": "No. 14, 2nd Floor, 17th Main, 3rd Block, Koramangala, Behind the temple…",
    "isDefault": false,
    "updatedAt": "2026-10-12T10:52:47.731Z",
    "serviceable": true,
    "kitchenId": "6acc8b…",
    "distanceKm": 0.72
  }
}
```

S04.11 · Make default

`PATCH /api/v1/users/me/addresses/{{addressId}}/default`

Authenticated. Choose the delivery address. Exactly one address is default; the previous one is cleared.

Response · Default set (200)

```
{
  "success": true,
  "message": "Default address set.",
  "data": {
    "addressId": "6accbb…",
    "label": "other",
    "customLabel": "Parents' Home",
    "displayLabel": "Parents' Home",
    "recipientName": "Rahul Sharma",
    "phone": "9876543210",
    "houseFlat": "No. 14, 2nd Floor",
    "street": "17th Main, 3rd Block",
    "locality": "Koramangala",
    "landmark": "Behind the temple",
    "city": "Bengaluru",
    "state": "Karnataka",
    "pincode": "560034",
    "latitude": 12.9346,
    "longitude": 77.6311,
    "fullAddress": "No. 14, 2nd Floor, 17th Main, 3rd Block, Koramangala, Behind the temple…",
    "isDefault": true,
    "updatedAt": "2026-10-12T10:52:47.731Z",
    "serviceable": true,
    "kitchenId": "6acc8b…",
    "distanceKm": 0.72
  }
}
```

S04.12 · Delete address

`DELETE /api/v1/users/me/addresses/{{addressId}}`

Authenticated. Removes the address (orders keep their own copy). When it was the default, the most recent remaining address becomes default. 404 if already deleted.

Response · Deleted (200)


```
{
  "success": true,
  "message": "Address deleted.",
  "data": {
    "addressId": "6accbb…",
    "deleted": true
  }
}
```


Backend tests · 17/17 passed

Test	What the app needs	Result
04-01	**“Use current location” in a served area** Location: “Do we deliver to you?” → yes	**Pass**
04-02	**“Use current location” outside the area says so** Location: not delivered here	**Pass**
04-03	**Saved location comes back** Home “Deliver to” after relaunch	**Pass**
04-04	**Delivery check by point and by pincode; bad input refused** Location search and pincode entry	**Pass**
04-05	**Place search, then the picked place with its delivery check** Location: search box → tap a suggestion	**Pass**
04-06	**Add address from the form (no map pin), label “Office”** Add new address: Full name, Phone, Pincode, House/Flat, Landmark, City, State; chips Home/Office/Other	**Pass**
04-07	**First address becomes the default** Addresses: the first saved address is selected	**Pass**
04-08	**Custom label like “Parents' Home” is kept** Addresses list shows “Parents' Home”	**Pass**
04-09	**Missing house/flat or a 5-digit pincode is refused** Add new address validation	**Pass**
04-10	**An address nobody can place is refused clearly** Add new address: wrong pincode/city	**Pass**
04-11	**Address list: default first, with label, full address and delivery check** Addresses screen	**Pass**
04-12	**Edit an address** Update address screen	**Pass**
04-13	**Make another address the default (only one default)** Addresses: choose delivery address	**Pass**
04-14	**Another customer cannot touch my address** Security	**Pass**
04-15	**Delete the default: another address takes over** Addresses: Delete	**Pass**
04-16	**Phone typed as “+91 98765 43210” is kept as 10 digits; a “Near …” landmark is not doubled** Add address: phone (+91 shown) and landmark	**Pass**
04-17	**Two “make default” taps at once leave exactly one default** Addresses: Make default (double tap)	**Pass**

Step 05 · Home

Screens: **Home, About chef**

What the app screens need

- Header: seasonal gradient with two promo images (e.g. Halloween), “Deliver to” pill, bell with unread count, greeting with the first name, “What are you craving today?”, Veg / All switch, search box hint, filter button.
- Sections in this order: signature hero carousel (eyebrow, title with an accent word, subtitle, Order Now), Today's Menu tiles + See all, MEAL COMBOS card, three feature icons, Your usual? (Reorder), Meal Ji Plus plans with monthly price and what you get, How we cook, Popular today.
- Floating bar: “Unlock Free delivery · Shop for ₹99 more” and the cart count.
- Filters sheet: Pure Veg / Fast Delivery / Top Rated chips, five sort options, cuisines (several can be picked), Clear All / Apply Filters.
- About chef: hero photo with a “FOUNDED 2024 • GREATER NOIDA” tag, “THE MAN BEHIND THE FLAME”, the chef's quote and name, two story paragraphs, “The Meal Ji Standard” with three icon points, and a “Taste the Menu” button.

Call flow

- Open Home: `GET /home` (add `veg=true` when the Veg switch is on). Draw `header` (or the default header when null), then `sections` in the order received, by `type`. Pull-to-refresh calls it again.
- When `serviceability.serviceable` is false, show `serviceability.message` and a Change location button (step 4).
- Filters: `GET /dishes/filters` once to build the sheet; Apply → `GET /dishes?quick=…&sort=…&cuisine=…` and show the result list.
- Banners: report `impression` when shown and `click` when tapped (`POST /banners/{id}/{event}`); open `deepLink`.
- Also `GET /in-app-messages?screen=home` and show any message once (report shown / clicked / dismissed).
- How we cook (`mealji://about-chef`) → About chef: `GET /kitchen-about`. “Taste the Menu” opens `ctaDeepLink`.
- Taps on a dish, combo, See all, Cart or Plus open screens from later steps (6, 8, 14).

APIs in this step

#	API	Auth	Used for
S05.1	`GET /home`	Token	Home
S05.2	`GET /dishes/filters`	Token	Filters sheet options
S05.3	`GET /dishes?quick=veg,fast&sort=prep_time&cuisine=North Indian,Fusion&limit=20&page=1`	Token	Apply filters
S05.4	`GET /kitchen-about`	Token	About chef
S05.5	`GET /banners?placement=home_hero`	Token	Banners by placement
S05.6	`POST /banners/{{id}}/{{event}}`	Token	Banner seen / tapped
S05.7	`GET /in-app-messages?screen=home`	Token	In-app messages
S05.8	`POST /in-app-messages/{{id}}/{{event}}`	Token	In-app message seen / tapped / closed

S05.1 · Home

`GET /api/v1/home`

Authenticated. The whole Home screen in one call. Optional: `veg=true` (header Veg switch), `latitude+longitude` (a point other than the saved location), `kitchenId`.

Top of the screen:

- `header`: seasonal theme — `header.backgroundColors` (gradient), `statusBarStyle`, text colours, and `promo` (title, subtitle, `ctaLabel`, `leftImageUrl/rightImageUrl`, `couponCode`, `deepLink`). Null = draw the default header.
- `greeting` ("Good afternoon, Rahul"), `headline` ("What are you craving today?"), `searchPlaceholder`.
- `deliverTo` (`label`, `line`) for the "Deliver to" pill; `serviceability` (`etaLabel`, `serviceable`, `message`).

`sections`: draw in this order, by `type`:

- `banners` (hero carousel: `eyebrow`, `title`, `highlight` = words in accent colour, `subtitle`, `imageUrl`, `ctaLabel`, `deepLink`).
- `categories` (Today's Menu tiles: `name`, `imageUrl`, `dishCount`).
- `combos` (`banner` = the MEAL COMBOS card, `items` = combos with `pricePaise`).
- `features` (3 icons: `icon`, `title`, `subtitle`).
- `usual` (Your usual?: past dishes; missing for new customers).
- `subscription_promo` (Meal Ji Plus plans: `pricePaise`, `cycleLabel`, `benefits`).
- `how_we_cook` (image card → `mealji://about-chef`).
- `popular` (dishes).

Bottom bar: `cart.itemCount` and `cart.amountToFreeDeliveryPaise` ("Shop for ₹99 more"); `unreadNotifications` for the bell; `subscription` card when the user has Meal Ji Plus.

Outside the delivery area `serviceability.serviceable` is false with a `message`, and menu sections are left out. All prices are in paise (₹1 = 100).

Veg / All: the saved preference (`vegOnly`, set by the header toggle via PATCH /users/me/preferences) applies here automatically. Send `veg=true|false` only to override it for this one call; the response's `vegOnly` says which mode was used.

Response · Home (200)


```
{
  "success": true,
  "message": "Home fetched.",
  "data": {
    "header": {
      "themeId": "6acc8b…",
      "name": "Halloween: Spooky Bites",
      "endsAt": "2026-10-31T18:30:00.000Z",
      "header": {
        "backgroundColors": [
          "#FF7A00",
          "#3B0A57",
          "#1A0B2E"
        ],
        "gradientAngle": 160,
        "backgroundImageUrl": null,
        "statusBarStyle": "light",
        "statusBarColor": "#3B0A57",
        "textColor": "#FFFFFF",
        "subTextColor": "#FFE0B2"
      },
      "promo": {
        "title": "Spooky Bites",
        "badge": "BOO",
        "subtitle": "Flat ₹131 OFF, trick or treat!",
        "ctaLabel": "GRAB A TREAT",
        "deepLink": "mealji://offers",
        "couponCode": "BOO131",
        "leftImageUrl": "https://cdn…/mealji/seed/halloween-pumpkin.png",
        "rightImageUrl": "https://cdn…/mealji/seed/halloween-ghost.png",
        "backgroundColors": [
          "#2A0F45",
          "#14081F"
        ],
        "titleColor": "#FFB74D",
        "subtitleColor": "#FFFFFFCC",
```

```
        "ctaColor": "#FF7A00",
        "ctaTextColor": "#FFFFFF"
      }
    },
    "greeting": "Good morning, Rahul",
    "headline": "What are you craving today?",
    "searchPlaceholder": "Search for dishes, biryani, meals...",
    "deliverTo": {
      "source": "location",
      "addressId": null,
  …
```


Response · Veg only (200)


```
{
  "success": true,
  "message": "Home fetched.",
  "data": {
    "header": {
      "themeId": "6acc8b…",
      "name": "Halloween: Spooky Bites",
      "endsAt": "2026-10-31T18:30:00.000Z",
      "header": {
        "backgroundColors": [
          "#FF7A00",
          "#3B0A57",
          "#1A0B2E"
        ],
        "gradientAngle": 160,
        "backgroundImageUrl": null,
        "statusBarStyle": "light",
        "statusBarColor": "#3B0A57",
        "textColor": "#FFFFFF",
        "subTextColor": "#FFE0B2"
      },
      "promo": {
        "title": "Spooky Bites",
        "badge": "BOO",
        "subtitle": "Flat ₹131 OFF, trick or treat!",
        "ctaLabel": "GRAB A TREAT",
        "deepLink": "mealji://offers",
        "couponCode": "BOO131",
        "leftImageUrl": "https://cdn…/mealji/seed/halloween-pumpkin.png",
        "rightImageUrl": "https://cdn…/mealji/seed/halloween-ghost.png",
        "backgroundColors": [
          "#2A0F45",
          "#14081F"
        ],
        "titleColor": "#FFB74D",
        "subtitleColor": "#FFFFFFCC",
        "ctaColor": "#FF7A00",
        "ctaTextColor": "#FFFFFF"
      }
    },
    "greeting": "Good morning, Rahul",
    "headline": "What are you craving today?",
    "searchPlaceholder": "Search for dishes, biryani, meals...",
    "deliverTo": {
      "source": "location",
      "addressId": null,
  …
```

S05.2 · Filters sheet options

`GET /api/v1/dishes/filters`

Authenticated. What to show in the Filters sheet (Home → filter icon):

- `quick`: chips — Pure Veg (`veg`), Fast Delivery (`fast`, ready in 15 min or less), Top Rated (`top_rated`, 4.5□+).
- `sort`: options with `value` + `label` (Relevance, Rating: High to Low, Delivery Time, Cost: Low to High, Cost: High to Low, Popular).
- `cuisines`: cuisines on this kitchen's menu (several can be picked).
- also `categories`, `tags`, `priceRangePaise`.

Send the chosen values to Apply filters.

Response · Filters (200)


```
{
  "success": true,
  "message": "Filters fetched.",
  "data": {
    "quick": [
      {
        "value": "veg",
        "label": "Pure Veg"
      },
      {
        "value": "fast",
        "label": "Fast Delivery"
      },
      {
        "value": "top_rated",
        "label": "Top Rated"
      }
    ],
    "sort": [
      {
        "value": "relevance",
        "label": "Relevance"
      },
      {
        "value": "rating",
        "label": "Rating: High to Low"
      },
      {
        "value": "prep_time",
        "label": "Delivery Time"
      },
      {
        "value": "price_asc",
        "label": "Cost: Low to High"
      },
      {
        "value": "price_desc",
        "label": "Cost: High to Low"
      },
      {
        "value": "popular",
        "label": "Popular"
      }
    ],
    "cuisines": [
      "Bakery",
  …
```

S05.3 · Apply filters

`GET /api/v1/dishes?quick=veg,fast&sort=prep_time&cuisine=North Indian,Fusion&limit=20&page=1`

Authenticated. Filters sheet → Apply filters. All parameters optional:

- `quick`: comma-separated chip values (`veg,fast,top_rated`).
- `sort`: a `value` from Filters sheet options.
- `cuisine`: one or more cuisines, comma-separated (a dish matches any of them).
- `limit` + `page` for paging (`hasMore` says if there is a next page).

Returns `items` (dishes), `total`, `page`, `limit`, `hasMore`. Unknown chip or sort → 422.

Response · Veg + fast, quickest first (200)


```
{
  "success": true,
  "message": "Dishes fetched.",
  "data": {
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "closedReason": null,
      "closedMessage": null,
      "nextOpenAt": null,
      "acceptingOrders": true,
      "opensAt": "08:00",
      "closesAt": "23:00"
    },
    "vegOnly": false,
    "items": [
      {
        "dishId": "6acc8b…",
        "kitchenId": "6acc8b…",
        "categoryId": "6acc8b…",
        "name": "Chilled Soft Drink",
        "description": "Ice-cold fizzy drink (300 ml).",
        "story": "",
        "images": [
          "https://cdn…/mealji/seed/cold-drink.jpg"
        ],
        "imageUrl": "https://cdn…/mealji/seed/cold-drink.jpg",
        "pricePaise": 5900,
        "originalPricePaise": null,
        "isVeg": true,
        "isAvailable": true,
        "unavailableReason": null,
        "unavailableMessage": null,
        "stockLeft": null,
        "isBestseller": false,
        "badge": null,
        "spicyLevel": 0,
        "calories": 140,
        "preparationMinutes": 1,
        "servesCount": 1,
        "highlights": [],
        "highlightBadges": [],
        "tags": [
          "drink",
          "cold"
  …
```


Errors to handle

Case	Status	Message shown to the user
Unknown chip (422)	422	Choose a filter from the list (field: quick)

S05.4 · About chef

`GET /api/v1/kitchen-about`

Authenticated. The About chef screen for the kitchen that serves the user (or `?kitchenId=`).

- `imageUrl`: hero photo; `tagline`: tag on the photo ("FOUNDED 2024 • GREATER NOIDA").
- `title`: small eyebrow line ("THE MAN BEHIND THE FLAME" — upper-case it in the app); `quote`; `chefName` ("— Chef Mujahid Khan").
- `paragraphs`: the story, already split into paragraphs (`story` is the raw text).
- `standardTitle` ("The Meal Ji Standard") and `pillars` [{ `icon`, `title`, `description` }] — `icon` is the app's icon name (fire, utensils, leaf).
- `ctaLabel` ("Taste the Menu") and `ctaDeepLink` (`mealji://menu`).
- `gallery`: more kitchen photos; `ratingAvg`, `ratingCount`, `opensAt`, `closesAt`.

Edited by the admin in Content → About the chef.

Response · About chef (200)


```
{
  "success": true,
  "message": "About fetched.",
  "data": {
    "kitchenId": "6acc8b…",
    "name": "Koramangala Kitchen",
    "area": "Koramangala",
    "city": "Bengaluru",
    "chefName": "Chef Mujahid Khan",
    "title": "The man behind the flame",
    "tagline": "FOUNDED 2024 • GREATER NOIDA",
    "quote": "We cook Indian food the way it was meant to be: unapologetic, slow-simm…",
    "story": "Meal Ji started from a simple frustration: why does delivery Indian foo…",
    "paragraphs": [
      "Meal Ji started from a simple frustration: why does delivery Indian foo…",
      "We built Meal Ji not as a massive marketplace with 50 mediocre cuisines…"
    ],
    "standardTitle": "The Meal Ji Standard",
    "pillars": [
      {
        "icon": "fire",
        "title": "24-Hour Makhani Simmer",
        "description": "No cornstarch, no fillers. Real butter, ripe tomatoes & whole green car…"
      },
      {
        "icon": "utensils",
        "title": "Handmade Baos & Breads",
        "description": "Puffed to order off our iron tawa and steam baskets. Never reheated."
      },
      {
        "icon": "leaf",
        "title": "Zero Preservatives",
        "description": "Clean eating you can enjoy 4 times a week without feeling sluggish."
      }
    ],
    "ctaLabel": "Taste the Menu",
    "ctaDeepLink": "mealji://menu",
    "imageUrl": "https://cdn…/mealji/seed/meals-hero-kitchen.jpg",
    "gallery": [
      "https://cdn…/mealji/seed/hero-kitchen-chef.jpg",
      "https://cdn…/mealji/seed/kitchen-flame.jpg",
      "https://cdn…/mealji/seed/kitchen-prep.jpg",
      "https://cdn…/mealji/seed/kitchen-fresh.jpg"
    ],
    "ratingAvg": 4.6,
    "ratingCount": 25,
  …
```

S05.5 · Banners by placement

`GET /api/v1/banners?placement=home_hero`

Authenticated. Banners for one placement (`placement` is required, e.g. ?placement=home_hero). Home (GET /home) already returns its banners; use this only for screens that load banners on their own.

Places: `home_hero`, `home_promo`, `home_combos`, `home_how_we_cook`, `menu_hero` (Menu tab's "MEAL COMBOS · Explore Combos" card), `combos_intro` (Combos screen: "Curated by the chef · Meal Ji Meals · Complete combos, priced fairly…"), `combos_hero` (Combos screen card: "MEAL JI MEALS · Bigger Meals Happier Moments · Signature Meal"), `offers`. All are edited in the admin console (Content → Banners). `title` may contain a line break (\n).

Response · Hero banners (200)


```
{
  "success": true,
  "message": "Banners fetched.",
  "data": [
    {
      "bannerId": "6acc8b…",
      "placement": "home_hero",
      "eyebrow": "OUR SIGNATURE CHICKEN BIRYANI",
      "title": "A BOWL OF HAPPINESS",
      "highlight": "HAPPINESS",
      "subtitle": "Rich flavours. Freshly cooked. Always for you.",
      "imageUrl": "https://cdn…/mealji/seed/hero-chicken-biryani.jpg",
      "ctaLabel": "Order Now",
      "deepLink": "mealji://menu?category=biryani",
      "couponCode": null,
      "startsAt": null,
      "endsAt": null
    },
    {
      "bannerId": "6acc8b…",
      "placement": "home_hero",
      "eyebrow": "24-HOUR MAKHANI",
      "title": "BUTTER CHICKEN, DONE RIGHT",
      "highlight": "DONE RIGHT",
      "subtitle": "Real butter. No cornstarch. No shortcuts.",
      "imageUrl": "https://cdn…/mealji/seed/hero-butter-chicken.jpg",
      "ctaLabel": "Order Now",
      "deepLink": "mealji://menu?category=signature-bowls",
      "couponCode": null,
      "startsAt": null,
      "endsAt": null
    },
    {
      "bannerId": "6acc8b…",
      "placement": "home_hero",
      "eyebrow": "NEW HERE?",
      "title": "₹100 OFF YOUR FIRST ORDER",
      "highlight": "₹100 OFF",
      "subtitle": "Use code WELCOME100 on orders above ₹299.",
      "imageUrl": "https://cdn…/mealji/seed/hero-butter-chicken-rice.jpg",
      "ctaLabel": "Order Now",
      "deepLink": "mealji://menu",
      "couponCode": "WELCOME100",
      "startsAt": null,
      "endsAt": null
    },
  …
```


Response · menu_hero (200)


```
{
  "success": true,
  "message": "Banners fetched.",
  "data": [
    {
      "bannerId": "6acc9b…",
      "placement": "menu_hero",
      "eyebrow": null,
      "title": "MEAL COMBOS",
      "highlight": "COMBOS",
      "subtitle": "Great food. Better together.",
      "imageUrl": "https://cdn…/mealji/seed/meal-combos.png",
      "ctaLabel": "Explore Combos",
      "deepLink": "mealji://combos",
```

```
      "couponCode": null,
      "startsAt": null,
      "endsAt": null
    }
  ]
}
```

S05.6 · Banner seen / tapped

`POST /api/v1/banners/{{id}}/{{event}}`

Authenticated. Call `impression` when a banner is shown and `click` when tapped (hero, combos card, how we cook). Used for banner reports; no response data to use. `event` must be impression or click.

Response · Tap counted (200)

```
{
  "success": true,
  "message": "Tracked.",
  "data": {
    "tracked": false
  }
}
```

S05.7 · In-app messages

`GET /api/v1/in-app-messages?screen=home`

Authenticated. Pop-ups / announcements to show on a screen. Call on Home with `screen=home`. Returns a list (often empty). `screen` is required.

Response · Messages for Home (200)

```
{
  "success": true,
  "message": "In-app messages.",
  "data": []
}
```

Errors to handle

Case	Status	Message shown to the user
Missing screen (422)	422	Please check what you entered (field: screen)

S05.8 · In-app message seen / tapped / closed

`POST /api/v1/in-app-messages/{{id}}/{{event}}`

Authenticated. Report `shown`, `clicked` or `dismissed` for a message so it is not shown again when it should not be.

Backend tests · 24/24 passed

Test	What the app needs	Result
05-01	**Home needs sign-in and loads in one call** Home: everything comes from GET /home	**Pass**
05-02	**Header: seasonal theme, greeting with first name, headline and search hint** Header gradient + 2 promo images, “Good Morning, Alex!”, “What are you craving today?”, “Search for dishes, biryani, meals…”	**Pass**
05-03	**“Deliver to” pill and delivery promise** Header: Deliver to <label>, <city>	**Pass**
05-04	**Sections come in the app's order** Hero → Today's Menu → Meal Combos → features → Your usual? → Meal Ji Plus → How we cook → Popular	**Pass**
05-05	**Hero cards carry eyebrow, title with highlight, subtitle, image and button** Signature card: “OUR SIGNATURE CHICKEN BIRYANI / A BOWL OF HAPPINESS / Order Now”	**Pass**
05-06	**Today's Menu tiles: name, photo, only categories with dishes** Today's Menu row + See all	**Pass**
05-07	**Meal Combos card and combos** “MEAL COMBOS · Great food. Better together. · Explore Combos”	**Pass**

Test	What the app needs	Result
05-08	**Feature strip: 3 icons with title and subtitle** Fast Delivery / Fresh Ingredients / Hygienic Kitchen	Pass
05-09	**Your usual: past dishes with name, price, photo** “Your usual? · Order again in one tap” + Reorder	Pass
05-10	**Meal Ji Plus: plans with monthly price and what you get** Plus cards: ₹3,499 / ₹4,499 / ₹5,499 monthly + feature list	Pass
05-11	**How we cook opens About chef** How we cook banner → About chef	Pass
05-12	**Cart bar: item count and “Shop for ₹X more” to free delivery** Floating bar: Unlock Free delivery / Shop For ₹99 more / Cart badge	Pass
05-13	**Veg toggle: only veg dishes and combos** Header Veg / All switch	Pass
05-14	**Outside the delivery area: says so, no menu** Home when the location is not served	Pass
05-15	**New customer: no “Your usual”, still a full home** First visit after sign-up	Pass
05-16	**Filters sheet options match the app** Filters: Pure Veg / Fast Delivery / Top Rated; Relevance, Rating, Delivery Time, Cost ↑, Cost ↓; Cuisines	Pass
05-17	**Applying filters returns matching dishes in order** Filters: Apply	Pass
05-18	**Banner taps are counted** Hero/combos card tap	Pass
05-19	**In-app messages for the home screen** Popups/announcements on Home	Pass
05-20	**Every image on Home loads** No broken images	Pass
05-21	**About chef has everything the screen shows** Hero photo + “FOUNDED 2024 • GREATER NOIDA”, “THE MAN BEHIND THE FLAME”, quote, “— Chef Mujahid Khan”, 2 paragraphs	Pass
05-22	**“The Meal Ji Standard” points and the menu button** 3 points with icon, title, description; “Taste the Menu” → Menu	Pass
05-23	**About photos load** Hero image and gallery	Pass
05-24	**About editor refuses bad content** Admin: Content → About the chef	Pass

Step 06 · Menu and dish

Screens: **Menu, Dish detail, Combos**

What the app screens need

- Menu: category tabs (name, icon/photo, tagline such as “Aromatic. Authentic. Always a good idea.”), a grid of dish cards (photo, Bestseller badge, heart, name, 2-line description, price, Add +), the MEAL COMBOS banner and a search box.
- Dish detail: feature badges (Fresh Ingredients / Slow Cooked / Halal Certified), MOST POPULAR label, name, price, □ rating with review count, minutes, Serves, description, “Select a portion” cards (Regular · Serves 1 / Large · Serves 2 / Family Pack · Serves 3-4), option groups, “Make it a meal” (components, ~~₹417~~ ₹349, Save ₹68), “From the chef” card, “You may also like”, quantity and Add to cart.
- Combos: “Curated by the chef”, chips All / For one / For two / Family / Under ₹500, cards with title, items, price, struck original price, “Save ₹X”, badge and Add.
- States the app has images for: sold out and kitchen closed.

Call flow

- Menu tab: `GET /menu` (or the app's own path `GET /kitchens/{kitchenId}/menu`). Draw `categories` as tabs and each category's `dishes` as the grid. `kitchen.isOpenNow` false → show `kitchen.closedMessage` and disable Add.
- A dish with `isAvailable: false` shows `unavailableMessage` (e.g. “Sold out for today”, “Available for Breakfast (7–11 am)”) instead of Add.
- Dish card → `GET /menu/items/{dishId}`; then `GET /menu/items/{dishId}/recommendations` for “You may also like”. Use `portions`, `customizationGroups` (`required`, `multiple`, `minSelect`, `maxSelect`, option `pricePaise`) and `mealUpgrade` to build the choices; price = portion (or dish) price + chosen options + meal upgrade.
- Heart: `POST /users/me/favorites` / `DELETE /users/me/favorites/{dishId}` (step 7); every dish in these lists carries `isFavorite`.
- Combos: `GET /combos/filters` for the chips, `GET /combos?chip=…` for the list, `GET /combos/{comboId}` for one combo.
- Add to cart and the cart badge belong to step 8.

APIs in this step

#	API	Auth	Used for
S06.1	`GET /menu`	Token	Menu
S06.2	`GET /kitchens/{{kitchenId}}/menu`	Token	Menu (app path)
S06.3	`GET /menu/items/{{dishId}}`	Token	Dish detail
S06.4	`GET /menu/items/{{dishId}}/recommendations`	Token	You may also like
S06.5	`GET /combos/filters`	Token	Combo chips
S06.6	`GET /combos?chip=all`	Token	Combos
S06.7	`GET /combos/{{comboId}}`	Token	One combo
S06.8	`GET /combos/signature`	Token	Signature combos
S06.9	`GET /dishes?limit=20&page=1&sort=popular`	Token	Dish list (paged)
S06.10	`GET /menu/categories`	Token	Categories only
S06.11	`GET /menu/popular`	Token	Popular dishes

S06.1 · Menu

`GET /api/v1/menu`

Authenticated. The Menu tab for the kitchen that delivers to the user (or `?kitchenId=`).

- `kitchen`: `name`, `isOpenNow`, `closedReason` / `closedMessage` (show and disable Add when closed or paused), `opensAt`, `closesAt`, `nextOpenAt`.
- `categories`: the tabs, in order — `name`, `subtitle` (the tagline), `imageUrl`, `dishes`. Each dish appears once, inside its category.
- dish card fields: `dishId`, `name`, `description`, `imageUrl`, `pricePaise`, `originalPricePaise`, `isVeg`, `isBestseller`, `badge`, `ratingAvg`, `isFavorite` (heart), `isAvailable` + `unavailableMessage`.

Optional filters (same as Apply filters): `veg`, `quick`, `sort`, `cuisine`, `categoryId`, `minPricePaise`, `maxPricePaise`. A wrong value is a 422.

Veg / All: the saved preference (`vegOnly`, set by the header toggle via PATCH /users/me/preferences) applies here automatically. Send `veg=true|false` only to override it for this one call; the response's `vegOnly` says which mode was used.

Each category has `slug` (e.g. biryani, signature-bowls — what deep links like mealji://menu?category=biryani use) and `icon` (the app's icon name for the Menu tab: utensils, gift, fire, sparkles, clock, heart, leaf, food, star).

Response · Menu (200)


```
{
  "success": true,
  "message": "Menu fetched.",
  "data": {
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "closedReason": null,
      "closedMessage": null,
      "nextOpenAt": null,
      "acceptingOrders": true,
      "opensAt": "08:00",
      "closesAt": "23:00"
    },
    "categories": [
      {
        "categoryId": "6acc8b…",
        "name": "Paneer",
        "slug": "paneer",
        "icon": "leaf",
        "subtitle": "Paneer, every way you like it",
        "imageUrl": "https://cdn…/mealji/seed/paneer-bao.jpg",
        "sortOrder": 0,
        "isActive": true,
        "dishes": [
          {
            "dishId": "6acc8b…",
            "kitchenId": "6acc8b…",
            "categoryId": "6acc8b…",
            "name": "Paneer Bao (3 pcs)",
            "description": "Puffy steamed bao buns stuffed with smoky clay-oven paneer, pickled oni…",
            "story": "East Asian technique meets Old Delhi tandoor mastery. Fluffy, spicy, ta…",
            "images": [
              "https://cdn…/mealji/seed/paneer-bao.jpg"
            ],
            "imageUrl": "https://cdn…/mealji/seed/paneer-bao.jpg",
            "pricePaise": 28900,
            "originalPricePaise": 32900,
            "isVeg": true,
            "isAvailable": true,
            "unavailableReason": null,
            "unavailableMessage": null,
            "stockLeft": null,
            "isBestseller": true,
            "badge": "Most Loved",
  …
```


Errors to handle

Case	Status	Message shown to the user
Wrong filter (422)	422	Choose breakfast, lunch, snacks or dinner (field: slot)

S06.2 · Menu (app path)

`GET /api/v1/kitchens/{{kitchenId}}/menu`

Authenticated. Same response as Menu, for the path the app's menuApi.ts already uses: `GET /kitchens/{kitchenId}/menu`. A kitchen id that is not valid → 422; a kitchen that is not active → 404.

Veg / All: the saved preference (`vegOnly`, set by the header toggle via PATCH /users/me/preferences) applies here automatically. Send `veg=true|false` only to override it for this one call; the response's `vegOnly` says which mode was used.

Response · Menu by kitchen (200)


```
{
  "success": true,
  "message": "Menu fetched.",
  "data": {
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "closedReason": null,
      "closedMessage": null,
      "nextOpenAt": null,
      "acceptingOrders": true,
      "opensAt": "08:00",
      "closesAt": "23:00"
    },
    "categories": [
      {
        "categoryId": "6acc8b…",
        "name": "Paneer",
        "slug": "paneer",
        "icon": "leaf",
        "subtitle": "Paneer, every way you like it",
        "imageUrl": "https://cdn…/mealji/seed/paneer-bao.jpg",
        "sortOrder": 0,
        "isActive": true,
        "dishes": [
          {
            "dishId": "6acc8b…",
            "kitchenId": "6acc8b…",
            "categoryId": "6acc8b…",
            "name": "Paneer Bao (3 pcs)",
            "description": "Puffy steamed bao buns stuffed with smoky clay-oven paneer, pickled oni…",
            "story": "East Asian technique meets Old Delhi tandoor mastery. Fluffy, spicy, ta…",
            "images": [
              "https://cdn…/mealji/seed/paneer-bao.jpg"
            ],
            "imageUrl": "https://cdn…/mealji/seed/paneer-bao.jpg",
            "pricePaise": 28900,
            "originalPricePaise": 32900,
            "isVeg": true,
            "isAvailable": true,
            "unavailableReason": null,
            "unavailableMessage": null,
            "stockLeft": null,
            "isBestseller": true,
            "badge": "Most Loved",
  …
```

S06.3 · Dish detail

`GET /api/v1/menu/items/{{dishId}}`

Authenticated. The Dish detail screen.

- Top: `badge` (MOST POPULAR), `name`, `pricePaise`, `ratingAvg` + `ratingCount`, `preparationMinutes`, `servesCount`, `description`, `highlights` (feature badges), `isFavorite`, `isVeg`, `spicyLevel`, `calories`.
- `portions`: "Select a portion" cards — `label`, `serves` ("Serves 2"), `pricePaise` (the full price for that portion), one `isDefault`.
- `customizationGroups`: `name`, `required`, `multiple`, `minSelect`, `maxSelect`, `options` [{ `optionId`, `name`, `pricePaise` (added), `isAvailable` }].
- `mealUpgrade`: "Make it a meal" — `label`, `description` (components), `pricePaise` (added to the dish), `originalPricePaise`, `savingsPaise`, `imageUrl`. Null when the dish has none.
- `chef`: "From the chef" — `name`, `imageUrl`, `note`.
- `kitchen`: open state (as on Menu). `isAvailable` false → show `unavailableMessage` instead of Add to cart.

Price shown on Add to cart = (portion price or `pricePaise`) + chosen option prices + meal upgrade, × quantity. Unknown dish → 404.

Dish detail extras: `highlightBadges` [{ label, icon }] for the badges row (Fresh Ingredients → leaf, Slow Cooked → fire, Halal Certified → checkc); `deepLink` (mealji://dish/{id}) and `shareUrl` for Share; `mealUpgrade` adds the whole-meal card prices `totalPricePaise`, `totalOriginalPricePaise`, `totalSavingsPaise` ("~~₹417~~ ₹349 · Save ₹68") and `isRecommended`.

Response · Butter Chicken Bowl (200)


```
{
  "success": true,
  "message": "Dish fetched.",
  "data": {
    "dishId": "6acc8b…",
    "kitchenId": "6acc8b…",
    "categoryId": "6acc8b…",
    "name": "Butter Chicken Bowl",
    "description": "24-hr simmered makhani sauce, tender boneless chicken, fragrant cumin j…",
    "story": "Chef Mujahid’s signature bowl, cooked over low flame with house-ground …",
    "images": [
      "https://cdn…/mealji/seed/butter-chicken-bowl.jpg"
    ],
    "imageUrl": "https://cdn…/mealji/seed/butter-chicken-bowl.jpg",
    "pricePaise": 34900,
    "originalPricePaise": 39900,
    "isVeg": false,
    "isAvailable": true,
    "unavailableReason": null,
    "unavailableMessage": null,
    "stockLeft": null,
    "isBestseller": true,
    "badge": "Chef’s Kiss",
    "spicyLevel": 1,
    "calories": 620,
    "preparationMinutes": 20,
    "servesCount": 1,
    "highlights": [
      "Fresh Ingredients",
      "Slow Cooked",
      "Halal Certified"
    ],
    "highlightBadges": [
      {
        "label": "Fresh Ingredients",
        "icon": "leaf"
      },
      {
        "label": "Slow Cooked",
        "icon": "fire"
      },
      {
        "label": "Halal Certified",
        "icon": "checkc"
      }
    ],
  …
```

Errors to handle

Case	Status	Message shown to the user
Unknown dish (404)	404	Dish not found

S06.4 · You may also like

`GET /api/v1/menu/items/{{dishId}}/recommendations`

Authenticated. “You may also like” on Dish detail: up to 6 dishes that go well with this one (other categories first), each with `isFavorite`. Unknown dish → 404.

Response · Suggestions (200)


```
{
  "success": true,
  "message": "Recommendations fetched.",
  "data": [
    {
      "dishId": "6acc8b…",
      "kitchenId": "6acc8b…",
      "categoryId": "6acc8b…",
      "name": "Chicken Biryani",
      "description": "Our signature dum biryani: aged basmati, bone-in chicken, saffron and f…",
      "story": "",
      "images": [
        "https://cdn…/mealji/seed/chicken-biryani.jpg"
      ],
      "imageUrl": "https://cdn…/mealji/seed/chicken-biryani.jpg",
      "pricePaise": 32900,
      "originalPricePaise": null,
      "isVeg": false,
      "isAvailable": true,
      "unavailableReason": null,
      "unavailableMessage": null,
      "stockLeft": null,
      "isBestseller": true,
      "badge": "Signature",
      "spicyLevel": 2,
      "calories": 780,
      "preparationMinutes": 22,
      "servesCount": 1,
      "highlights": [
        "Fresh Ingredients",
        "Slow Cooked",
        "Halal Certified"
      ],
      "highlightBadges": [
        {
          "label": "Fresh Ingredients",
          "icon": "leaf"
        },
        {
          "label": "Slow Cooked",
          "icon": "fire"
        },
        {
          "label": "Halal Certified",
          "icon": "checkc"
        }
  …
```

S06.5 · Combo chips

`GET /api/v1/combos/filters`

Authenticated. The Combos screen chips, in order: All, For one, For two, Family, Under ₹500 (`value` + `label`). Send the chosen `value` as `chip` to Combos.

Response · Chips (200)


```
{
  "success": true,
  "message": "Combo filters.",
  "data": {
    "chips": [
      {
        "value": "all",
        "label": "All"
      },
      {
        "value": "one",
        "label": "For one"
      },
      {
        "value": "two",
        "label": "For two"
      },
      {
        "value": "family",
        "label": "Family"
      },
      {
        "value": "under_500",
        "label": "Under ₹500"
      }
    ]
  }
}
```


S06.6 · Combos

`GET /api/v1/combos?chip=all`

Authenticated. Combo cards: `comboId`, `name`, `subtitle` (what's inside), `imageUrl`, `pricePaise`, `originalPricePaise`, `savingsPaise` ("Save ₹X"), `badge`, `servesCount`, `audience` (one | two | family), `isVeg`, `isAvailable`, `items` [{ `name`, `qty`, `imageUrl`, `pricePaise` }].

Optional: `chip` (all | one | two | family | under_500), `veg=true`.

Veg / All: the saved preference (`vegOnly`, set by the header toggle via PATCH /users/me/preferences) applies here automatically. Send `veg=true|false` only to override it for this one call; the response's `vegOnly` says which mode was used.

Response · All combos (200)


```
{
  "success": true,
  "message": "Combos fetched.",
  "data": [
    {
      "comboId": "6acc8b…",
      "kitchenId": "6acc8b…",
      "name": "Biryani Feast for 4",
      "title": "Biryani Feast for 4",
      "subtitle": "2 chicken, 1 mutton and 1 paneer biryani",
      "imageUrl": "https://cdn…/mealji/seed/combo-spread.jpg",
      "pricePaise": 119900,
      "originalPricePaise": 135600,
      "itemsTotalPaise": 135600,
      "savingsPaise": 15700,
      "badge": "Party pack",
      "servesCount": 4,
      "serves": 4,
      "items": [
        {
          "dishId": "6acc8b…",
          "qty": 2,
          "name": "Chicken Biryani",
          "imageUrl": "https://cdn…/mealji/seed/chicken-biryani.jpg",
```

```
          "pricePaise": 32900,
          "isVeg": false
        },
        {
          "dishId": "6acc8b…",
          "qty": 1,
          "name": "Mutton Biryani",
          "imageUrl": "https://cdn…/mealji/seed/mutton-biryani.jpg",
          "pricePaise": 39900,
          "isVeg": false
        },
        {
          "dishId": "6acc8b…",
          "qty": 1,
          "name": "Paneer Biryani",
          "imageUrl": "https://cdn…/mealji/seed/paneer-biryani.jpg",
          "pricePaise": 29900,
          "isVeg": true
        }
      ],
      "isVeg": false,
      "filterTags": [],
  …
```


Response · For two (200)


```
{
  "success": true,
  "message": "Combos fetched.",
  "data": [
    {
      "comboId": "6acc8b…",
      "kitchenId": "6acc8b…",
      "name": "Sharing Platter for 2",
      "title": "Sharing Platter for 2",
      "subtitle": "2 butter chicken bowls, arancini and cheesecake",
      "imageUrl": "https://cdn…/mealji/seed/meal-sharing-plate.jpg",
      "pricePaise": 99900,
      "originalPricePaise": 116600,
      "itemsTotalPaise": 116600,
      "savingsPaise": 16700,
      "badge": "Chef's pick",
      "servesCount": 2,
      "serves": 2,
      "items": [
        {
          "dishId": "6acc8b…",
          "qty": 2,
          "name": "Butter Chicken Bowl",
          "imageUrl": "https://cdn…/mealji/seed/butter-chicken-bowl.jpg",
          "pricePaise": 34900,
          "isVeg": false
        },
        {
          "dishId": "6acc8b…",
          "qty": 1,
          "name": "Biryani Arancini (4 pcs)",
          "imageUrl": "https://cdn…/mealji/seed/biryani-arancini.jpg",
          "pricePaise": 24900,
          "isVeg": true
        },
        {
          "dishId": "6acc8b…",
          "qty": 1,
          "name": "Gulab Cheesecake",
          "imageUrl": "https://cdn…/mealji/seed/gulab-cheesecake.jpg",
          "pricePaise": 21900,
          "isVeg": true
        }
      ],
      "isVeg": false,
      "filterTags": [],
  …
```

S06.7 · One combo

`GET /api/v1/combos/{{comboId}}`

Authenticated. One combo by id (deep links, combo sheet). Same fields as a card. Unknown → 404.

Response · Combo (200)


```
{
  "success": true,
  "message": "Combo fetched.",
  "data": {
    "comboId": "6acc8b…",
    "kitchenId": "6acc8b…",
    "name": "Biryani Feast for 4",
    "title": "Biryani Feast for 4",
    "subtitle": "2 chicken, 1 mutton and 1 paneer biryani",
    "imageUrl": "https://cdn…/mealji/seed/combo-spread.jpg",
    "pricePaise": 119900,
    "originalPricePaise": 135600,
    "itemsTotalPaise": 135600,
    "savingsPaise": 15700,
    "badge": "Party pack",
    "servesCount": 4,
    "serves": 4,
    "items": [
      {
        "dishId": "6acc8b…",
        "qty": 2,
        "name": "Chicken Biryani",
        "imageUrl": "https://cdn…/mealji/seed/chicken-biryani.jpg",
        "pricePaise": 32900,
        "isVeg": false
      },
      {
        "dishId": "6acc8b…",
        "qty": 1,
        "name": "Mutton Biryani",
        "imageUrl": "https://cdn…/mealji/seed/mutton-biryani.jpg",
        "pricePaise": 39900,
        "isVeg": false
      },
      {
        "dishId": "6acc8b…",
        "qty": 1,
        "name": "Paneer Biryani",
        "imageUrl": "https://cdn…/mealji/seed/paneer-biryani.jpg",
        "pricePaise": 29900,
        "isVeg": true
      }
    ],
    "isVeg": false,
    "filterTags": [],
    "isSignature": true,
  …
```


S06.8 · Signature combos

`GET /api/v1/combos/signature`

Authenticated. Only the chef's signature combos (the “Signature Meal” button). Same fields as Combos.

Response · Signature combos (200)


```
{
  "success": true,
  "message": "Signature combos fetched.",
  "data": [
    {
      "comboId": "6acc8b…",
      "kitchenId": "6acc8b…",
      "name": "Biryani Feast for 4",
      "title": "Biryani Feast for 4",
      "subtitle": "2 chicken, 1 mutton and 1 paneer biryani",
      "imageUrl": "https://cdn…/mealji/seed/combo-spread.jpg",
      "pricePaise": 119900,
      "originalPricePaise": 135600,
      "itemsTotalPaise": 135600,
      "savingsPaise": 15700,
```

```
      "badge": "Party pack",
      "servesCount": 4,
      "serves": 4,
      "items": [
        {
          "dishId": "6acc8b…",
          "qty": 2,
          "name": "Chicken Biryani",
          "imageUrl": "https://cdn…/mealji/seed/chicken-biryani.jpg",
          "pricePaise": 32900,
          "isVeg": false
        },
        {
          "dishId": "6acc8b…",
          "qty": 1,
          "name": "Mutton Biryani",
          "imageUrl": "https://cdn…/mealji/seed/mutton-biryani.jpg",
          "pricePaise": 39900,
          "isVeg": false
        },
        {
          "dishId": "6acc8b…",
          "qty": 1,
          "name": "Paneer Biryani",
          "imageUrl": "https://cdn…/mealji/seed/paneer-biryani.jpg",
          "pricePaise": 29900,
          "isVeg": true
        }
      ],
      "isVeg": false,
      "filterTags": [],
  …
```

S06.9 · Dish list (paged)

`GET /api/v1/dishes?limit=20&page=1&sort=popular`

Authenticated. All dishes as one paged list (See all, Apply filters). Same filters as Menu plus `limit` (1–100) and `page`. Returns `items`, `total`, `page`, `limit`, `hasMore`, `kitchen`.

Veg / All: the saved preference (`vegOnly`, set by the header toggle via PATCH /users/me/preferences) applies here automatically. Send `veg=true|false` only to override it for this one call; the response's `vegOnly` says which mode was used.

Response · First page (200)

```
{
  "success": true,
  "message": "Dishes fetched.",
  "data": {
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "closedReason": null,
      "closedMessage": null,
      "nextOpenAt": null,
      "acceptingOrders": true,
      "opensAt": "08:00",
      "closesAt": "23:00"
    },
    "vegOnly": false,
    "items": [
      {
        "dishId": "6acc8b…",
        "kitchenId": "6acc8b…",
        "categoryId": "6acc8b…",
        "name": "Butter Chicken Bowl",
        "description": "24-hr simmered makhani sauce, tender boneless chicken, fragrant cumin j…",
        "story": "Chef Mujahid’s signature bowl, cooked over low flame with house-ground …",
        "images": [
          "https://cdn…/mealji/seed/butter-chicken-bowl.jpg"
        ],
        "imageUrl": "https://cdn…/mealji/seed/butter-chicken-bowl.jpg",
        "pricePaise": 34900,
        "originalPricePaise": 39900,
        "isVeg": false,
```

```
      "isAvailable": true,
      "unavailableReason": null,
      "unavailableMessage": null,
      "stockLeft": null,
      "isBestseller": true,
      "badge": "Chef's Kiss",
      "spicyLevel": 1,
      "calories": 620,
      "preparationMinutes": 20,
      "servesCount": 1,
      "highlights": [
        "Fresh Ingredients",
        "Slow Cooked",
        "Halal Certified"
      ],
  …
```


S06.10 · Categories only

`GET /api/v1/menu/categories`

Authenticated. Just the category tiles (name, subtitle, imageUrl, dishCount) without dishes.

Response · Categories (200)


```
{
  "success": true,
  "message": "Categories fetched.",
  "data": [
    {
      "categoryId": "6acc8b…",
      "name": "Paneer",
      "slug": "paneer",
      "icon": "leaf",
      "subtitle": "Paneer, every way you like it",
      "imageUrl": "https://cdn…/mealji/seed/paneer-bao.jpg",
      "sortOrder": 0,
      "isActive": true,
      "dishCount": 2
    },
    {
      "categoryId": "6acc8b…",
      "name": "Daal & Curries",
      "slug": "daal-curries",
      "icon": "fire",
      "subtitle": "Slow-simmered comfort",
      "imageUrl": "https://cdn…/mealji/seed/dal-ramen.jpg",
      "sortOrder": 1,
      "isActive": true,
      "dishCount": 2
    },
    {
      "categoryId": "6acc8b…",
      "name": "Roti & Rice",
      "slug": "roti-rice",
      "icon": "food",
      "subtitle": "Fresh off the tawa",
      "imageUrl": "https://cdn…/mealji/seed/butter-naan.jpg",
      "sortOrder": 2,
      "isActive": true,
      "dishCount": 3
    },
    {
      "categoryId": "6acc8b…",
      "name": "Meal Ji Meals",
      "slug": "meal-ji-meals",
      "icon": "gift",
      "subtitle": "The complete feast",
      "imageUrl": "https://cdn…/mealji/seed/meal-ji-meal.jpg",
      "sortOrder": 3,
      "isActive": true,
  …
```

S06.11 · Popular dishes

`GET /api/v1/menu/popular`

Authenticated. Best sellers right now (“Popular right now”, “Popular Picks for You”), each with `isFavorite`.

Veg / All: the saved preference (`vegOnly`, set by the header toggle via PATCH /users/me/preferences) applies here automatically. Send `veg=true|false` only to override it for this one call; the response's `vegOnly` says which mode was used.

Response · Popular (200)


```
{
  "success": true,
  "message": "Popular dishes fetched.",
  "data": [
    {
      "dishId": "6acc8b…",
      "kitchenId": "6acc8b…",
      "categoryId": "6acc8b…",
      "name": "Butter Chicken Bowl",
      "description": "24-hr simmered makhani sauce, tender boneless chicken, fragrant cumin j…",
      "story": "Chef Mujahid’s signature bowl, cooked over low flame with house-ground …",
      "images": [
        "https://cdn…/mealji/seed/butter-chicken-bowl.jpg"
      ],
      "imageUrl": "https://cdn…/mealji/seed/butter-chicken-bowl.jpg",
      "pricePaise": 34900,
      "originalPricePaise": 39900,
      "isVeg": false,
      "isAvailable": true,
      "unavailableReason": null,
      "unavailableMessage": null,
      "stockLeft": null,
      "isBestseller": true,
      "badge": "Chef’s Kiss",
      "spicyLevel": 1,
      "calories": 620,
      "preparationMinutes": 20,
      "servesCount": 1,
      "highlights": [
        "Fresh Ingredients",
        "Slow Cooked",
        "Halal Certified"
      ],
      "highlightBadges": [
        {
          "label": "Fresh Ingredients",
          "icon": "leaf"
        },
        {
          "label": "Slow Cooked",
          "icon": "fire"
        },
        {
          "label": "Halal Certified",
          "icon": "checkc"
        }
  …
```


Backend tests · 22/22 passed

Test	What the app needs	Result
06-01	**Menu: categories with name, tagline and photo, dishes inside each** Menu tabs (Biryani, Meals, Curries…) + “Aromatic. Authentic. Always a good idea.”	**Pass**
06-02	**Each dish appears once** No duplicate cards	**Pass**
06-03	**Dish cards carry what the grid shows** Card: photo, Bestseller badge, heart, name, 2-line description, price, Add +	**Pass**
06-04	**Menu says whether the kitchen can take orders now** Closed / paused state	**Pass**

Test	What the app needs	Result
06-05	**The app's own menu path works** menuApi.ts: GET /kitchens/{id}/menu	Pass
06-06	**Wrong menu filters are refused clearly** Bad parameters	Pass
06-07	**Dish detail: the top of the screen** MOST POPULAR badge, name, price, □ rating (reviews), prep minutes, Serves, description	Pass
06-08	**Feature badges and heart** Fresh Ingredients / Slow Cooked / Halal Certified; heart filled when saved	Pass
06-09	**Portions: name, serves and price, one default** “Select a portion”: Regular · Serves 1 / Large · Serves 2 / Family Pack · Serves 3-4	Pass
06-10	**Option groups say required, single or multi, and price of each option** Choose Rice Portion (required, pick 1) / Add a Side (optional)	Pass
06-11	**“Make it a meal” card** Complete Meal · components · ~~₹417~~ ₹349 · Save ₹68 · photo	Pass
06-12	**“From the chef” card** Chef's note and “— Chef, Meal Ji”	Pass
06-13	**You may also like** Recommendations with price and +	Pass
06-14	**A dish outside its hours says when it is available** Sold out / not now state	Pass
06-15	**Unknown dish is a 404** Deep link to a removed dish	Pass
06-16	**Combo cards: title, items, price, original price, Save ₹X, badge, photo** “Bao & Bowl · Paneer bao + dal ramen · ₹449 ~~₹538~~ Save ₹89 · BESTSELLER”	Pass
06-17	**Combos carry no internal fields** Customer view only	Pass
06-18	**Combo chips: All / For one / For two / Family / Under ₹500** Combos screen filter chips	Pass
06-19	**One combo by id** Combo detail / deep link	Pass
06-20	**Every menu, dish and combo image loads** No broken images	Pass
06-21	**Veg / All: the saved choice applies on every list; the toggle changes it; ?veg overrides one call** Header Veg / All switch (remembered)	Pass
06-22	**Menu tabs, header cards and dish badges as the app draws them** Menu tabs with icons · MEAL COMBOS card · Combos header · Dish badges, Share, Make it a meal	Pass

Step 07 · Search and favourites

Screens: **Search, Favourites**

What the app screens need

- Search, empty: “Trending searches” chips and “Recent” rows.
- Search, results: “N dishes match "query"” and cards with badge or veg mark, name, price, description, photo, Add +.
- Search, nothing found: “No results found”, “Try these instead” chips and “Popular right now” cards with hearts.
- Favourites: the user's saved dishes (or the empty state “No favorites yet”), “Popular Picks for You”, the count on the Profile tile, and hearts on every dish card.

Call flow

- Open Search: `GET /search/trending` and `GET /search/recent`. Tap □ on a recent row → `DELETE /search/recent/{query}`; clear all → `DELETE /search/recent`.
- Typing: debounce ~300 ms, then `GET /search?q=…` (add `veg=true` when the Veg switch is on). Small typos still match. The server keeps one recent entry per typed word, not every letter.
- Search + filters are ONE call: the filter button on Search opens the same sheet as Home (built from `GET /dishes/filters`); Apply calls `GET /search?q=…&quick=…&sort=…&cuisine=…&minPricePaise=…&maxPricePaise=…&limit=20&page=1`. Clear filters = call again with only `q`. Sort without `sort` keeps the best match first.
- Show `dishCount` (“N dishes match”) and `dishes`; load more while `hasMore`. `filtersApplied` lists the active filters (for chips on the results screen). When `total` is 0: if `filteredOut` show `emptyMessage` (“No dishes match these filters”) with Clear filters; otherwise show `suggestions` as “Try these instead” and `popular` as “Popular right now”.
- Tap a result → `POST /search/{searchId}/click` with the `dishId`, then open Dish detail.
- Heart: `POST /users/me/favorites` (201 first time, 200 if already saved) and `DELETE /users/me/favorites/{dishId}`.
- Favourites screen: `GET /users/me/favorites`, newest first. `orderableHere` false → the dish is from a kitchen that does not deliver to the current location (or not available now): show it greyed with `kitchenName`. Empty → show the empty state and `GET /menu/popular` as Popular Picks.
- The Profile tile count is `stats.favorites` from `GET /users/me`.

APIs in this step

#	API	Auth	Used for
S07.1	`GET /search/trending`	Token	Trending searches
S07.2	`GET /search/recent`	Token	Recent searches
S07.3	`GET /search?q=biryani&veg=true&quick=top_rated&sort=price_asc&cuisine=&minPricePaise=&maxPricePaise=&limit=20&page=1`	Token	Search
S07.4	`POST /search/{{searchId}}/click`	Token	Result tapped
S07.5	`DELETE /search/recent/{{query}}`	Token	Remove one recent search
S07.6	`DELETE /search/recent`	Token	Clear recent searches
S07.7	`POST /users/me/favorites`	Token	Heart a dish
S07.8	`GET /users/me/favorites`	Token	My favourites
S07.9	`DELETE /users/me/favorites/{{dishId}}`	Token	Un-heart a dish

S07.1 · Trending searches

`GET /api/v1/search/trending`

Authenticated. “Trending searches” chips on the empty Search screen: up to 10 `{ query, searches }`, most searched by different people in the last 7 days; topped up with best sellers when there is little history.

Each chip has `type`: "query" (tap → GET /search?q={query}) or "filter" — the last chip “Under ₹200” (amount set in the admin, Settings → App) opens the dish list with its `filters`: GET /dishes?maxPricePaise=20000&sort=price_asc.

Response · Trending (200)


```
{
  "success": true,
  "message": "Trending.",
  "data": [
    {
      "type": "query",
      "query": "Kulfi",
      "label": "Kulfi",
      "filters": null,
      "searches": 17
    },
    {
      "type": "query",
      "query": "Bao",
      "label": "Bao",
      "filters": null,
      "searches": 17
    },
    {
      "type": "query",
      "query": "Ramen",
      "label": "Ramen",
      "filters": null,
      "searches": 10
    },
    {
      "type": "query",
      "query": "Meal Ji Meal",
      "label": "Meal Ji Meal",
      "filters": null,
      "searches": 10
    },
    {
      "type": "query",
      "query": "Butter chicken",
      "label": "Butter chicken",
      "filters": null,
      "searches": 10
    },
    {
      "type": "query",
      "query": "Biryani",
      "label": "Biryani",
      "filters": null,
      "searches": 51
    },
  …
```


S07.2 · Recent searches

`GET /api/v1/search/recent`

Authenticated. The user's “Recent” rows: up to 10 `{ query, at }`, newest first. Typing letter by letter is saved as one search.

Response · Recent (200)


```
{
  "success": true,
  "message": "Recent searches.",
  "data": [
    {
      "query": "paneer",
      "at": "2026-10-12T10:55:36.086Z"
    }
  ]
```

```
}
```

S07.3 · Search

`GET /api/v1/search?q=biryani&veg=true&quick=top_rated&sort=price_asc&cuisine=&minPricePaise=&maxPricePaise=&limit=20&page=1`

Authenticated. Search and the Filters sheet in ONE call. `q` (1–60 characters) is required; everything else is optional and works exactly like the dish list (`GET /dishes`):

- `veg=true` (Veg toggle), `quick` = veg, fast, top_rated (comma-separated chips), `cuisine` (one or more, comma-separated), `minPricePaise` / `maxPricePaise`, `categoryId`, `tag`, `slot`, `availableOnly`.
- `sort` = relevance (default: best match first, orderable first), rating, prep_time, price_asc, price_desc, popular.
- `limit` (1–100, default 40) + `page`; `hasMore` says if there is a next page.
- `latitude` + `longitude` or `kitchenId`: which kitchen to search (default: the one that delivers to the user).

Draw the filter sheet from `GET /dishes/filters` (same chips, sort labels and cuisines on Home and Search). Small typos still match (“briyani”).

Returns `searchId` (send with taps), `dishCount` (all matches after filters — “N dishes match”), `dishes` (this page), `combos` (follow veg and price; left out when a dish-only filter is on), `categories`, `filtersApplied`, `page`, `limit`, `hasMore`, `total`.

Nothing found: `emptyMessage`; `filteredOut: true` + `matchesWithoutFilters` means the filters removed the results — show “Clear filters” (call again with only `q`); otherwise show `suggestions` (“Try these instead”) and `popular` (“Popular right now”).

A wrong filter value is 422 with the same message as the dish list.

Veg / All: the saved preference (`vegOnly`, set by the header toggle via PATCH /users/me/preferences) applies here automatically. Send `veg=true|false` only to override it for this one call; the response's `vegOnly` says which mode was used.

Response · Search only (200)


```
{
  "success": true,
  "message": "Search results.",
  "data": {
    "query": "biryani",
    "vegOnly": false,
    "searchId": "6acf1a…",
    "total": 7,
    "dishes": [
      {
        "dishId": "6acc8b…",
        "kitchenId": "6acc8b…",
        "categoryId": "6acc8b…",
        "name": "Biryani Arancini (4 pcs)",
        "description": "Golden spiced biryani croquettes with melted aged mozzarella centers & …",
        "story": "Crisp outside, molten cheesy center bursting with saffron spices.",
        "images": [
          "https://cdn…/mealji/seed/biryani-arancini.jpg"
        ],
        "imageUrl": "https://cdn…/mealji/seed/biryani-arancini.jpg",
        "pricePaise": 24900,
        "originalPricePaise": null,
        "isVeg": true,
        "isAvailable": true,
        "unavailableReason": null,
        "unavailableMessage": null,
        "stockLeft": null,
        "isBestseller": false,
        "badge": null,
        "spicyLevel": 1,
        "calories": 440,
        "preparationMinutes": 14,
        "servesCount": 1,
        "highlights": [],
        "highlightBadges": [],
        "tags": [
          "snack",
          "biryani"
        ],
        "cuisine": "Fusion",
```

```
        "portions": [],
        "mealUpgrade": null,
        "deepLink": "mealji://dish/6acc8bdd189320566a1fab42",
        "shareUrl": null,
        "shareText": "Biryani Arancini (4 pcs) on Meal Ji — order it in the Meal Ji app",
        "customizationGroups": [],
  …
```


Response · Search + filters + sort (200)


```
{
  "success": true,
  "message": "Search results.",
  "data": {
    "query": "biryani",
    "vegOnly": true,
    "searchId": "6acf1a…",
    "total": 1,
    "dishes": [
      {
        "dishId": "6acc8b…",
        "kitchenId": "6acc8b…",
        "categoryId": "6acc8b…",
        "name": "Biryani Arancini (4 pcs)",
        "description": "Golden spiced biryani croquettes with melted aged mozzarella centers & …",
        "story": "Crisp outside, molten cheesy center bursting with saffron spices.",
        "images": [
          "https://cdn…/mealji/seed/biryani-arancini.jpg"
        ],
        "imageUrl": "https://cdn…/mealji/seed/biryani-arancini.jpg",
        "pricePaise": 24900,
        "originalPricePaise": null,
        "isVeg": true,
        "isAvailable": true,
        "unavailableReason": null,
        "unavailableMessage": null,
        "stockLeft": null,
        "isBestseller": false,
        "badge": null,
        "spicyLevel": 1,
        "calories": 440,
        "preparationMinutes": 14,
        "servesCount": 1,
        "highlights": [],
        "highlightBadges": [],
        "tags": [
          "snack",
          "biryani"
        ],
        "cuisine": "Fusion",
        "portions": [],
        "mealUpgrade": null,
        "deepLink": "mealji://dish/6acc8bdd189320566a1fab42",
        "shareUrl": null,
        "shareText": "Biryani Arancini (4 pcs) on Meal Ji — order it in the Meal Ji app",
        "customizationGroups": [],
  …
```


Errors to handle

Case	Status	Message shown to the user
Unknown filter value (422)	422	Choose a filter from the list (field: quick)

S07.4 · Result tapped

`POST /api/v1/search/{{searchId}}/click`

Authenticated. Call when the user taps a result (search analytics), with the `searchId` from Search and the tapped `dishId`. Unknown search → 404.

Request body

```
{
  "dishId": "{{dishId}}"
}
```

S07.5 · Remove one recent search

`DELETE /api/v1/search/recent/{{query}}`

Authenticated. The on one Recent row: removes that search (any letter case).

S07.6 · Clear recent searches

`DELETE /api/v1/search/recent`

Authenticated. Clears the whole Recent list.

S07.7 · Heart a dish

`POST /api/v1/users/me/favorites`

Authenticated. Heart on any dish card or Dish detail. 201 the first time, 200 when it was already saved (safe to repeat). Only dishes on a live menu can be saved (404 otherwise).

Request body

```
{
  "dishId": "{{dishId}}"
}
```

Response · Saved (201)

```
{
  "success": true,
  "message": "Added to favourites.",
  "data": {
    "dishId": "6ac76e…",
    "isFavorite": true
  }
}
```

Response · Already saved (200)

```
{
  "success": true,
  "message": "Already in favourites.",
  "data": {
    "dishId": "6ac76e…",
    "isFavorite": true
  }
}
```

Errors to handle

Case	Status	Message shown to the user
Bad id (422)	422	Validation failed (field: dishId)

S07.8 · My favourites

`GET /api/v1/users/me/favorites`

Authenticated. The Favourites screen, newest first. Each dish has the card fields plus `isFavorite: true`, `kitchenName`, `kitchenArea`, `fromServingKitchen` and `orderableHere`.

`orderableHere` false → the dish is from a kitchen that does not deliver to the current location, or not available right now: show it greyed with `kitchenName`. Empty list → empty state + Popular dishes (step 6). The Profile tile count is `stats.favorites` from GET /users/me.

Response · Favourites (200)


```
{
  "success": true,
  "message": "Favourites fetched.",
  "data": [
    {
      "dishId": "6acc8b…",
      "kitchenId": "6acc8b…",
      "categoryId": "6acc8b…",
      "name": "Mutton Biryani",
      "description": "Tender mutton slow-cooked on dum with whole spices and aged basmati.",
      "story": "",
      "images": [
        "https://cdn…/mealji/seed/mutton-biryani.jpg"
      ],
      "imageUrl": "https://cdn…/mealji/seed/mutton-biryani.jpg",
      "pricePaise": 39900,
      "originalPricePaise": null,
      "isVeg": false,
      "isAvailable": true,
      "unavailableReason": null,
      "unavailableMessage": null,
      "stockLeft": null,
      "isBestseller": false,
      "badge": null,
      "spicyLevel": 2,
      "calories": 850,
      "preparationMinutes": 25,
      "servesCount": 1,
      "highlights": [],
      "highlightBadges": [],
      "tags": [
        "biryani",
        "mutton"
      ],
      "cuisine": "Hyderabadi",
      "portions": [],
      "mealUpgrade": null,
      "deepLink": "mealji://dish/6acc8bdd189320566a1fab15",
      "shareUrl": null,
      "shareText": "Mutton Biryani on Meal Ji — order it in the Meal Ji app",
      "customizationGroups": [],
      "availableSlots": [
        "lunch",
        "dinner",
        "snacks"
      ],
  …
```

S07.9 · Un-heart a dish

`DELETE /api/v1/users/me/favorites/{{dishId}}`

Authenticated. Tap the heart again. Always 200 (also when it was not saved).

Response · Removed (200)

```
{
  "success": true,
  "message": "Removed from favourites.",
  "data": {
    "dishId": "6ac76e…",
    "isFavorite": false
  }
}
```

Backend tests · 22/22 passed

Test	What the app needs	Result
07-01	**Trending searches for the empty search screen** “Trending searches” chips (Butter chicken, Bao, Ramen… Under ₹200)	**Pass**
07-02	**Results: count, dishes with badge, veg mark, price, photo, heart** “2 dishes match "butter"” + cards with Add +	**Pass**
07-03	**A small typo still finds the dish** Users type “briyani”, “panner”	**Pass**
07-04	**No results: suggestions and popular dishes** “No results found” + “Try these instead” + “Popular right now”	**Pass**
07-05	**Veg search and the user's location** Veg switch on; search near the selected address	**Pass**
07-06	**Typing does not fill Recent with every letter** Recent shows “paneer”, not “p, pa, pan…”	**Pass**
07-07	**Recent: remove one, clear all** Recent rows and clear	**Pass**
07-08	**Tapping a result is counted; unknown search is 404** Search analytics	**Pass**
07-09	**Bad search input is refused** Empty / too long / not text	**Pass**
07-17	**Search and filters together: every result matches the word and the filters** Search → filter button → Pure Veg / Top Rated / cuisine / price	**Pass**
07-18	**Sort the search results; without sort the best match stays first** Sort by: Relevance / Rating / Delivery Time / Cost ↑ / Cost ↓	**Pass**
07-19	**Search results page with “load more”** Long result lists	**Pass**
07-20	**Filters that remove everything say so (Clear filters)** Empty result because of filters vs no match at all	**Pass**
07-21	**Same filter values and errors as the dish list** One Filters sheet for Home and Search	**Pass**
07-22	**Combos follow veg and price filters; dish-only filters leave them out** Combos in search results	**Pass**
07-10	**Heart a dish: first time 201, again 200** Heart on cards and dish detail	**Pass**
07-11	**Favourites list: newest first, can it be ordered here, from which kitchen** Favourites screen cards + Add +	**Pass**
07-12	**Profile shows the favourites count** Profile tile “Favorites 4”	**Pass**

Test	What the app needs	Result
07-13	**Un-heart removes it everywhere** Tap the heart again	**Pass**
07-14	**Hidden or unpublished dishes cannot be saved** Only live dishes	**Pass**
07-15	**A favourite from another kitchen is marked “not here”** Saved dish from a kitchen that does not deliver to you	**Pass**
07-16	**Popular picks for the empty favourites screen** “Popular Picks for You“	**Pass**

Step 08 · Cart and checkout

Screens: **Cart, Checkout (Bill summary), Offers**

What the app screens need

- Cart: kitchen name and ETA ("Meal Ji • 26 min delivery"), lines with photo, veg mark, name, choices ("Large • + Butter Naan", or "Regular"), price and a −/+ stepper (below 1 removes), Customize, clear cart, "Add a few more favorites?" suggestions, "Apply promo code", and the bottom bar with the total and Checkout.
- Checkout (the app's Checkout route is the Bill summary screen): Delivery Address card (Deliver to / + Add new address), Delivery Details ("Standard Delivery · 26 min (Today)" or "Schedule for later"), Payment Method list ("UPI (Recommended) · Pay with any UPI app", "Credit / Debit Card", "Cash on Delivery"), Order Summary, Bill Details (Item total (n items), Delivery fee, Packaging charge, Taxes (GST), Total amount) with "You're saving ₹X on this order!", Special Instructions, and Pay ₹X.
- Offers: today the app has only the "Apply promo code" row (no offers sheet yet); the API returns a ready list of usable offers for that sheet. The Offers tab in the app is the Rewards (points) screen — step 12.
- Home floating bar: "Unlock Free delivery · Shop for ₹X more" and the cart count.

Call flow

- Add: `POST /cart/items` from Dish detail (with `portionId`, `optionIds`, `mealUpgrade`), a card's Add + or Combos (`comboId`). On 409 `CART_KITCHEN_MISMATCH` ask "Replace your cart?" and resend with `replaceCart: true`.
- Cart screen: `GET /cart`; stepper → `PATCH /cart/items/{lineId}` with `qty` (0 removes); trash → `DELETE /cart`; suggestions → `GET /cart/recommendations`.
- Offers: `GET /promos/available` → `POST /cart/promo` with the code (show `message` on 422) → `DELETE /cart/promo`.
- Checkout: `GET /delivery/slots` for Schedule for later; save choices with `PATCH /cart` (`addressId`, `scheduledFor`, `tipPaise`, `usePoints`, `chefNote`). Use `POST /checkout/summary` to try a payment method without saving it.
- Print `billLines` top to bottom and the `savingsMessage`; never add up money in the app. Disable Pay when `canCheckout` is false and show the first blocker's `message`.
- Pay ₹X → place the order (step 9).

APIs in this step

#	API	Auth	Used for
S08.1	`GET /cart`	Token	My cart
S08.2	`POST /cart/items`	Token	Add to cart
S08.3	`PATCH /cart/items/{{lineId}}`	Token	Change a cart line
S08.4	`DELETE /cart/items/{{lineId}}`	Token	Remove a cart line
S08.5	`DELETE /cart`	Token	Clear the cart
S08.6	`GET /cart/recommendations`	Token	Add a few more favorites?
S08.7	`GET /promos/available`	Token	Offers for my cart
S08.8	`POST /cart/promo`	Token	Apply an offer
S08.9	`DELETE /cart/promo`	Token	Remove the offer
S08.10	`GET /delivery/slots`	Token	Delivery slots
S08.11	`PATCH /cart`	Token	Checkout choices
S08.12	`POST /checkout/summary`	Token	Checkout summary

S08.1 · My cart

`GET /api/v1/cart`

Authenticated. The Cart screen (also after app restart: the cart is kept on the server).

Every cart call returns the whole priced cart (the server does all the money):

- `kitchen` (`name`, `isOpenNow`, `message`), `etaLabel` ("28 min"), `delivery` (`type` asap | scheduled, `label`).
- `items`: `lineId`, `name`, `imageUrl`, `isVeg`, `qty`, `optionsText` ("Large • Extra Butter Jeera Rice • + Complete Meal", or "Regular"), `unitPricePaise`, `originalUnitPricePaise`, `totalPaise`, `isAvailable` + `issue`, `priceChanged`.
- `billLines`: rows to print in order (`label`, `amountPaise`, `text` such as FREE, `strikePaise`, `isTotal`). Rows add up to the total.
- `bill` (all raw amounts), `savingsMessage` ("You're saving ₹56 on this order!"), `freeDeliveryMessage` ("Add ₹99 more for free delivery", null when delivery is free already).
- `coupon`, `tip` (`enabled`, `presetsPaise`), `points` (`available`, `usable`, `usedPoints`), `address` (chosen or default), `paymentMethods` (`label`, `description`, `enabled`, `note`), `chefNote`.
- `canCheckout` and `blockers` [{ `code`, `message` }]: EMPTY, ITEMS_UNAVAILABLE, KITCHEN_CLOSED, MIN_ORDER, ADDRESS_REQUIRED, ADDRESS_NOT_SERVICEABLE, PICKUP_DISABLED, COD_NOT_ALLOWED, SLOT_UNAVAILABLE, COUPON_INVALID (soft: checkout still allowed).

`kitchen` also has `address` { line, area, city, pincode, fullAddress, latitude, longitude } and `pickupReadyLabel` ("Pickup ready in 15–20 mins") for the pickup card on Checkout.

Response · Cart (200)


```
{
  "success": true,
  "message": "Cart fetched.",
  "data": {
    "cartId": "6acc8b…",
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "message": null,
      "address": {
        "line": "12 4th Block",
        "area": "Koramangala",
        "city": "Bengaluru",
        "pincode": "560034",
        "fullAddress": "12 4th Block, Koramangala, Bengaluru, 560034",
        "latitude": 12.9352,
        "longitude": 77.6245
      },
      "pickupReadyLabel": "Pickup ready in 15–20 mins"
    },
    "items": [
      {
        "lineId": "ce2d8aadd9be",
        "kind": "dish",
        "dishId": "6acc8b…",
        "name": "Masala Papad",
        "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
        "isVeg": true,
        "qty": 2,
        "portion": null,
        "mealUpgrade": null,
        "options": [],
        "specialInstructions": null,
        "unitPricePaise": 7900,
        "basePricePaise": 7900,
        "originalUnitPricePaise": 7900,
        "packagingPaise": 0,
        "totalPaise": 15800,
        "isAvailable": true,
        "issue": null,
        "priceChanged": false,
        "dishIds": [
          {
            "dishId": "6acc8b…",
            "qty": 2
  …
```

S08.2 · Add to cart

`POST /api/v1/cart/items`

Authenticated. Dish detail → Add to cart, a card's Add +, Combos → Add. Send `dishId` or `comboId`, `qty` (1–50), and for dishes the chosen `portionId`, `optionIds` (from `customizationGroups`), `mealUpgrade`, `specialInstructions` (≤200).

The same dish with the same choices adds to the existing line. Quick repeated taps are safe (no item is lost).

Errors: 422 when a required option is missing or a portion/option is not on the dish (`errors[].field`), 404 unknown dish/combo, 409 when the dish or combo cannot be ordered now (message says why), 409 `CART_KITCHEN_MISMATCH` when the cart holds another kitchen's food — ask the user, then send `replaceCart: true`.

Every cart call returns the whole priced cart (the server does all the money):

- `kitchen` (`name`, `isOpenNow`, `message`), `etaLabel` ("28 min"), `delivery` (`type` asap | scheduled, `label`).
- `items`: `lineId`, `name`, `imageUrl`, `isVeg`, `qty`, `optionsText` ("Large • Extra Butter Jeera Rice • + Complete Meal", or "Regular"), `unitPricePaise`, `originalUnitPricePaise`, `totalPaise`, `isAvailable` + `issue`, `priceChanged`.
- `billLines`: rows to print in order (`label`, `amountPaise`, `text` such as FREE, `strikePaise`, `isTotal`). Rows add up to the total.
- `bill` (all raw amounts), `savingsMessage` ("You're saving ₹56 on this order!"), `freeDeliveryMessage` ("Add ₹99 more for free delivery", null when delivery is free already).
- `coupon`, `tip` (`enabled`, `presetsPaise`), `points` (`available`, `usable`, `usedPoints`), `address` (chosen or default), `paymentMethods` (`label`, `description`, `enabled`, `note`), `chefNote`.
- `canCheckout` and `blockers` [{ `code`, `message` }]: EMPTY, ITEMS_UNAVAILABLE, KITCHEN_CLOSED, MIN_ORDER, ADDRESS_REQUIRED, ADDRESS_NOT_SERVICEABLE, PICKUP_DISABLED, COD_NOT_ALLOWED, SLOT_UNAVAILABLE, COUPON_INVALID (soft: checkout still allowed).

Request body

```
{
  "dishId": "{{dishId}}",
  "qty": 1,
  "portionId": "large",
  "optionIds": [
    "ca08cfa3b4"
  ],
  "mealUpgrade": false,
  "specialInstructions": null
}
```

Response · Added (portion, options, meal) (201)

```
{
  "success": true,
  "message": "Added to cart.",
  "data": {
    "cartId": "6acc8b…",
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "message": null,
      "address": {
        "line": "12 4th Block",
        "area": "Koramangala",
        "city": "Bengaluru",
        "pincode": "560034",
        "fullAddress": "12 4th Block, Koramangala, Bengaluru, 560034",
        "latitude": 12.9352,
        "longitude": 77.6245
      },
      "pickupReadyLabel": "Pickup ready in 15–20 mins"
    },
    "items": [
      {
        "lineId": "ce2d8aadd9be",
        "kind": "dish",
        "dishId": "6acc8b…",
        "name": "Masala Papad",
        "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
        "isVeg": true,
        "qty": 2,
        "portion": null,
```

```
        "mealUpgrade": null,
        "options": [],
        "specialInstructions": null,
        "unitPricePaise": 7900,
        "basePricePaise": 7900,
        "originalUnitPricePaise": 7900,
        "packagingPaise": 0,
        "totalPaise": 15800,
        "isAvailable": true,
        "issue": null,
        "priceChanged": false,
        "dishIds": [
          {
            "dishId": "6acc8b…",
            "qty": 2
  …
```


Errors to handle

Case	Status	Message shown to the user
Required option missing (422)	422	Validation failed (field: optionIds)

S08.3 · Change a cart line

`PATCH /api/v1/cart/items/{{lineId}}`

Authenticated. Stepper −/+ (`qty`; 0 removes the line), "Customize" (`portionId`, `optionIds`, `mealUpgrade`) and the line note (`specialInstructions`). If the new choices equal another line, the two lines merge.

422 for a portion/option the dish does not have or qty over 50; 404 for a line that is not in your cart.

Request body


```
{
  "qty": 3
}
```


Response · Quantity 3 (200)


```
{
  "success": true,
  "message": "Cart updated.",
  "data": {
    "cartId": "6ac76e…",
    "kitchen": {
      "kitchenId": "6ac4ba…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "message": null
    },
    "items": [
      {
        "lineId": "c7883b85156d",
        "kind": "dish",
        "dishId": "6ac76e…",
        "name": "Butter Chicken Bowl",
        "imageUrl": "https://cdn…/mealji/seed/butter-chicken-bowl.jpg",
        "isVeg": false,
        "qty": 1,
        "portion": {
          "portionId": "large",
          "label": "Large"
        },
        "mealUpgrade": {
          "label": "Complete Meal",
          "pricePaise": 9900
        },
        "options": [
          {
            "optionId": "ca08cfa3b4",
            "name": "Extra Butter Jeera Rice",
            "groupName": "Choose Rice Portion",
            "pricePaise": 4000
          },
          {
            "optionId": "3736c09a82",
            "name": "Crispy Masala Papad",
            "groupName": "Add a Side",
```

```
          "pricePaise": 3900
        }
      ],
      "specialInstructions": "Less spicy",
      "unitPricePaise": 77700,
      "originalUnitPricePaise": 79600,
      "packagingPaise": 0,
  …
```

Errors to handle

Case	Status	Message shown to the user
Not a portion of this dish (422)	422	Choose a valid portion (field: portionId)

S08.4 · Remove a cart line

`DELETE /api/v1/cart/items/{{lineId}}`

Authenticated. The same as setting qty to 0. Returns the cart.

S08.5 · Clear the cart

`DELETE /api/v1/cart`

Authenticated. The trash icon on Cart. Removes all items, the offer, tip, points and note.

Response · Cleared (200)

```
{
  "success": true,
  "message": "Cart cleared.",
  "data": {
    "cartId": "6ac76e…",
    "kitchen": null,
    "items": [],
    "itemCount": 0,
    "coupon": null,
    "bill": null,
    "deliveryMode": "delivery",
    "tipPaise": 0,
    "usePoints": false,
    "chefNote": null,
    "address": null,
    "canCheckout": false,
    "blockers": [
      {
        "code": "EMPTY",
        "message": "Your cart is empty"
      }
    ]
  }
}
```

S08.6 · Add a few more favorites?

`GET /api/v1/cart/recommendations`

Authenticated. Up to 6 popular dishes not in the cart, each with `isFavorite`. Empty when the cart is empty.

Response · Suggestions (200)

```
{
  "success": true,
  "message": "Recommendations fetched.",
  "data": [
    {
      "dishId": "6acc8b…",
      "kitchenId": "6acc8b…",
      "categoryId": "6acc8b…",
      "name": "Gulab Cheesecake",
      "description": "Velvety New York style cheesecake studded with miniature warm gulab jam…",
      "story": "The dessert that broke the internet. Sweet, decadent and unforgettable.",
      "images": [
        "https://cdn…/mealji/seed/gulab-cheesecake.jpg"
      ],
      "imageUrl": "https://cdn…/mealji/seed/gulab-cheesecake.jpg",
      "pricePaise": 21900,
      "originalPricePaise": null,
```

```
      "isVeg": true,
      "isAvailable": true,
      "unavailableReason": null,
      "unavailableMessage": null,
      "stockLeft": null,
      "isBestseller": true,
      "badge": "Must Try",
      "spicyLevel": 0,
      "calories": 380,
      "preparationMinutes": 5,
      "servesCount": 1,
      "highlights": [],
      "highlightBadges": [],
      "tags": [
        "dessert",
        "bestseller"
      ],
      "cuisine": "Fusion",
      "portions": [],
      "mealUpgrade": null,
      "deepLink": "mealji://dish/6acc8bdd189320566a1fab67",
      "shareUrl": null,
      "shareText": "Gulab Cheesecake on Meal Ji — order it in the Meal Ji app",
      "customizationGroups": [],
      "availableSlots": [
        "lunch",
        "dinner",
        "snacks"
      ],
  …
```


S08.7 · Offers for my cart

`GET /api/v1/promos/available`

Authenticated. The offers sheet (“Apply promo code”). Each offer: `code`, `title`, `label` (“₹150 off”, “20% off up to ₹100”), `description`, `terms`, `minOrderPaise`, `maxDiscountPaise`, `validTo`, `firstOrderOnly`, `paymentMethods`, and for this cart: `applicable`, `discountPaise` (what it would save), `message`, `shortByPaise` (“Add ₹X more”).

Offers the user can never use (other city/kitchen, already used, first-order only) are left out. Usable ones come first, biggest saving first.

Response · Offers (200)


```
{
  "success": true,
  "message": "Offers fetched.",
  "data": [
    {
      "couponId": "6acc8b…",
      "code": "FOODIE150",
      "title": "Foodie Weekend: flat ₹150 off",
      "label": "₹150 off",
      "description": "On orders above ₹599",
      "terms": [
        "Minimum order ₹599",
        "Up to 2 times per customer"
      ],
      "type": "flat",
      "value": 15000,
      "maxDiscountPaise": 0,
      "minOrderPaise": 59900,
      "validTo": null,
      "firstOrderOnly": false,
      "paymentMethods": [],
      "isReward": false,
      "dishId": null,
      "applicable": true,
      "message": "You save ₹150",
      "discountPaise": 15000,
      "shortByPaise": 0
    },
    {
      "couponId": "6acc8b…",
      "code": "BOO131",
      "title": "Spooky Bites: flat ₹131 off",
      "label": "₹131 off",
```

```
      "description": "Halloween week, orders above ₹499",
      "terms": [],
      "type": "flat",
      "value": 13100,
      "maxDiscountPaise": 0,
      "minOrderPaise": 49900,
      "validTo": "2026-10-31T18:30:00.000Z",
      "firstOrderOnly": false,
      "paymentMethods": [],
      "isReward": false,
      "dishId": null,
      "applicable": true,
      "message": "You save ₹131",
  …
```


S08.8 · Apply an offer

`POST /api/v1/cart/promo`

Authenticated. Send `code` (any letter case). Returns the cart with `coupon` and a "Coupon discount (CODE)" row in `billLines`.

422 with the reason as `message` (not valid, expired, add more to use this offer, already used, not at this kitchen…). The offer is re-checked on every cart view and at order time.

Request body


```
{
  "code": "FOODIE150"
}
```


Response · Applied (200)


```
{
  "success": true,
  "message": "Offer applied.",
  "data": {
    "cartId": "6ac76e…",
    "kitchen": {
      "kitchenId": "6ac4ba…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "message": null
    },
    "items": [
      {
        "lineId": "c7883b85156d",
        "kind": "dish",
        "dishId": "6ac76e…",
        "name": "Butter Chicken Bowl",
        "imageUrl": "https://cdn…/mealji/seed/butter-chicken-bowl.jpg",
        "isVeg": false,
        "qty": 1,
        "portion": {
          "portionId": "large",
          "label": "Large"
        },
        "mealUpgrade": {
          "label": "Complete Meal",
          "pricePaise": 9900
        },
        "options": [
          {
            "optionId": "ca08cfa3b4",
            "name": "Extra Butter Jeera Rice",
            "groupName": "Choose Rice Portion",
            "pricePaise": 4000
          },
          {
            "optionId": "3736c09a82",
            "name": "Crispy Masala Papad",
            "groupName": "Add a Side",
            "pricePaise": 3900
          }
        ],
        "specialInstructions": "Less spicy",
        "unitPricePaise": 77700,
        "originalUnitPricePaise": 79600,
```

```
      "packagingPaise": 0,
  …
```

Errors to handle

Case	Status	Message shown to the user
Unknown code (422)	422	This code is not valid (field: code)

S08.9 · Remove the offer

`DELETE /api/v1/cart/promo`

Authenticated. Returns the cart without the offer.

Response · Removed (200)


```
{
  "success": true,
  "message": "Offer removed.",
  "data": {
    "cartId": "6ac76e…",
    "kitchen": {
      "kitchenId": "6ac4ba…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "message": null
    },
    "items": [
      {
        "lineId": "c7883b85156d",
        "kind": "dish",
        "dishId": "6ac76e…",
        "name": "Butter Chicken Bowl",
        "imageUrl": "https://cdn…/mealji/seed/butter-chicken-bowl.jpg",
        "isVeg": false,
        "qty": 1,
        "portion": {
          "portionId": "large",
          "label": "Large"
        },
        "mealUpgrade": {
          "label": "Complete Meal",
          "pricePaise": 9900
        },
        "options": [
          {
            "optionId": "ca08cfa3b4",
            "name": "Extra Butter Jeera Rice",
            "groupName": "Choose Rice Portion",
            "pricePaise": 4000
          },
          {
            "optionId": "3736c09a82",
            "name": "Crispy Masala Papad",
            "groupName": "Add a Side",
            "pricePaise": 3900
          }
        ],
        "specialInstructions": "Less spicy",
        "unitPricePaise": 77700,
        "originalUnitPricePaise": 79600,
        "packagingPaise": 0,
  …
```

S08.10 · Delivery slots

`GET /api/v1/delivery/slots`

Authenticated. Checkout → Delivery Details. `options`: “As soon as possible” (`available` false when the kitchen is closed now). `scheduled`: half-hour windows today and tomorrow (`label` “Today 17:30–18:00”, `startsAt` to send as `scheduledFor`), when `scheduledEnabled` (admin: Settings → App → Schedule for later).

`options` are the two Checkout rows: { type: "asap", label: "Standard Delivery", sub: "31 min (Today)", icon: "bike" } and, when scheduling is on, { type: "scheduled", label: "Schedule for later", sub: "Choose date & time", icon: "clock" } — then pick one of `scheduled` (the half-hour windows).

Response · Slots (200)


```
{
  "success": true,
  "message": "Delivery slots fetched.",
  "data": {
    "options": [
      {
        "type": "asap",
        "label": "Standard Delivery",
        "sub": "29 min (Today)",
        "icon": "bike",
        "available": true
      },
      {
        "type": "scheduled",
        "label": "Schedule for later",
        "sub": "Choose date & time",
        "icon": "clock",
        "available": true
      }
    ],
    "scheduled": [
      {
        "date": "2026-10-14",
        "from": "12:30",
        "to": "13:00",
        "startsAt": "2026-10-14T07:00:00.000Z",
        "label": "Today 12:30–13:00"
      },
      {
        "date": "2026-10-14",
        "from": "13:00",
        "to": "13:30",
        "startsAt": "2026-10-14T07:30:00.000Z",
        "label": "Today 13:00–13:30"
      },
      {
        "date": "2026-10-14",
        "from": "13:30",
        "to": "14:00",
        "startsAt": "2026-10-14T08:00:00.000Z",
        "label": "Today 13:30–14:00"
      },
      {
        "date": "2026-10-14",
        "from": "14:00",
        "to": "14:30",
  …
```

S08.11 · Checkout choices

`PATCH /api/v1/cart`

Authenticated. Saves the checkout choices on the cart (send only what changed):

- `addressId` (Delivery Address card; null = use the default address), `deliveryMode` (delivery | pickup).
- `scheduledFor`: a `startsAt` from Delivery slots (Schedule for later), or null for as soon as possible. Any other time → 422.
- `tipPaise` (rider tip, presets in `tip.presetsPaise`), `usePoints` (Meal Ji points), `chefNote` (Special Instructions, ≤300).

Request body

```
{
  "tipPaise": 2000,
  "usePoints": false,
  "chefNote": "No onion, less spicy",
  "deliveryMode": "delivery",
  "addressId": "{{addressId}}",
  "scheduledFor": null
}
```

Response · Tip and note (200)

```
{
  "success": true,
  "message": "Cart updated.",
  "data": {
    "cartId": "6acc8b…",
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "message": null,
      "address": {
        "line": "12 4th Block",
        "area": "Koramangala",
        "city": "Bengaluru",
        "pincode": "560034",
        "fullAddress": "12 4th Block, Koramangala, Bengaluru, 560034",
        "latitude": 12.9352,
        "longitude": 77.6245
      },
      "pickupReadyLabel": "Pickup ready in 15–20 mins"
    },
    "items": [
      {
        "lineId": "ce2d8aadd9be",
        "kind": "dish",
        "dishId": "6acc8b…",
        "name": "Masala Papad",
        "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
        "isVeg": true,
        "qty": 2,
        "portion": null,
        "mealUpgrade": null,
        "options": [],
        "specialInstructions": null,
        "unitPricePaise": 7900,
        "basePricePaise": 7900,
        "originalUnitPricePaise": 7900,
        "packagingPaise": 0,
        "totalPaise": 15800,
        "isAvailable": true,
        "issue": null,
        "priceChanged": false,
        "dishIds": [
          {
            "dishId": "6acc8b…",
            "qty": 2
  …
```

Response · Scheduled (200)

```
{
  "success": true,
  "message": "Cart updated.",
  "data": {
    "cartId": "6ac76e…",
```

```
    "kitchen": {
      "kitchenId": "6ac4ba…",
      "name": "Koramangala Kitchen",
      "isOpenNow": true,
      "message": null
    },
    "items": [
      {
        "lineId": "c7883b85156d",
        "kind": "dish",
        "dishId": "6ac76e…",
        "name": "Butter Chicken Bowl",
        "imageUrl": "https://cdn…/mealji/seed/butter-chicken-bowl.jpg",
        "isVeg": false,
        "qty": 1,
        "portion": {
          "portionId": "large",
          "label": "Large"
        },
        "mealUpgrade": {
          "label": "Complete Meal",
          "pricePaise": 9900
        },
        "options": [
          {
            "optionId": "ca08cfa3b4",
            "name": "Extra Butter Jeera Rice",
            "groupName": "Choose Rice Portion",
            "pricePaise": 4000
          },
          {
            "optionId": "3736c09a82",
            "name": "Crispy Masala Papad",
            "groupName": "Add a Side",
            "pricePaise": 3900
          }
        ],
        "specialInstructions": "Less spicy",
        "unitPricePaise": 77700,
        "originalUnitPricePaise": 79600,
        "packagingPaise": 0,
  …
```


Errors to handle

Case	Status	Message shown to the user
Time not offered (422)	422	Choose one of the delivery times offered (field: scheduledFor)

S08.12 · Checkout summary

`POST /api/v1/checkout/summary`

Authenticated. The Checkout / Bill summary screen for choices the user is trying out, without saving them (e.g. switching payment method to see if Cash on Delivery is allowed). Same response as My cart.

Pay ₹X (place the order) is step 9.

Request body


```
{
  "addressId": "{{addressId}}",
  "deliveryMode": "delivery",
  "paymentMethod": "upi",
  "tipPaise": 0,
  "usePoints": false,
  "scheduledFor": null
}
```


Response · Summary (cash on delivery) (200)


```
{
  "success": true,
  "message": "Checkout summary.",
  "data": {
    "cartId": "6acc8b…",
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen",
```

```
      "isOpenNow": true,
      "message": null,
      "address": {
        "line": "12 4th Block",
        "area": "Koramangala",
        "city": "Bengaluru",
        "pincode": "560034",
        "fullAddress": "12 4th Block, Koramangala, Bengaluru, 560034",
        "latitude": 12.9352,
        "longitude": 77.6245
      },
      "pickupReadyLabel": "Pickup ready in 15–20 mins"
    },
    "items": [
      {
        "lineId": "ce2d8aadd9be",
        "kind": "dish",
        "dishId": "6acc8b…",
        "name": "Masala Papad",
        "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
        "isVeg": true,
        "qty": 2,
        "portion": null,
        "mealUpgrade": null,
        "options": [],
        "specialInstructions": null,
        "unitPricePaise": 7900,
        "basePricePaise": 7900,
        "originalUnitPricePaise": 7900,
        "packagingPaise": 0,
        "totalPaise": 15800,
        "isAvailable": true,
        "issue": null,
        "priceChanged": false,
        "dishIds": [
          {
            "dishId": "6acc8b…",
            "qty": 2
…
```


Backend tests · 28/28 passed

Test	What the app needs	Result
08-01	**Empty cart says so and cannot check out** Cart: empty state	**Pass**
08-02	**Add a dish with portion, options and meal: price adds up** Dish detail → Add to cart; cart line “Large • … • + Complete Meal”	**Pass**
08-03	**Wrong choices are refused clearly** Required option missing, bad portion, unknown dish	**Pass**
08-04	**Same dish and choices again is one line; different choices a new line** Tapping Add twice	**Pass**
08-05	**Fast taps on + never lose an item** Quantity stepper tapped quickly	**Pass**
08-06	**Change quantity, remove at 0, refuse bad values** Stepper − / + and remove	**Pass**
08-07	**Changing a line's options to match another merges them** Cart: Customize	**Pass**
08-08	**Items from another kitchen ask to replace the cart** Cart from a different kitchen	**Pass**
08-09	**Combos go in the cart at the combo price** Combos → Add	**Pass**
08-10	**Bill rows add up to the total, with GST** Bill details: Item total (n items), Delivery fee, Packaging, Taxes (GST), Total amount	**Pass**
08-11	**Header, savings and line texts are ready to show** “Meal Ji • 26 min delivery”, “You're saving ₹X on this order!”, options line	**Pass**

Test	What the app needs	Result
08-12	**Free delivery from ₹299 up; "₹X more" matches Home; Plus members never see it** Unlock Free delivery / Shop For ₹X more	Pass
08-13	**Offers list: code, title, label, terms, can I use it** Apply promo code → offers sheet	Pass
08-14	**Apply an offer: discount row, lower total; remove it again** Apply / Remove	Pass
08-15	**Wrong or too-small offers explain why** Invalid code / add more to use this offer	Pass
08-16	**"Add a few more favorites?" suggestions** Cart recommendations with +	Pass
08-17	**Delivery address: the default is picked; another can be chosen** Checkout: Delivery Address card	Pass
08-18	**An address the kitchen does not reach blocks checkout** Checkout: address not serviceable	Pass
08-19	**Tip and points change the total** Tip for your rider (₹20/₹30/₹50), use Meal Ji points	Pass
08-20	**Deliver now or schedule for later** Checkout: Standard Delivery "26 min (Today)" / Schedule for later	Pass
08-21	**Payment choices in the app's words; summary never saves** Checkout: Payment Method list	Pass
08-22	**Special instructions are kept (max 300 characters)** "E.g. no onion, less spicy, extra gravy..."	Pass
08-23	**Pickup (off at this kitchen) is explained** Delivery / Pickup	Pass
08-24	**A paused kitchen blocks checkout with a message** Kitchen closed / paused	Pass
08-25	**Another customer cannot touch my cart lines** Security	Pass
08-26	**Delivery options read like the app** Standard Delivery · 26 min (Today) / Schedule for later · Choose date & time	Pass
08-27	**Yes/no sent as text: "false" means false** Checkout: Use points switch	Pass
08-28	**Pickup card: kitchen address and ready time** Checkout (pickup): kitchen name, address, "Pickup ready in 15-20 mins"	Pass

Step 09 · Place order and pay

Screens: **Payment (Pay ₹X), Order confirmed, Payment methods**

What the app screens need

- Pay ₹X on the Checkout (Bill summary) screen with UPI (Recommended), Credit / Debit Card, Wallets (Paytm, PhonePe, GPay) or Cash on Delivery, and "Save this payment method for faster checkout".
- The app has a Razorpay placeholder (services/payment/razorpay.ts) expecting: create order → open checkout with order id + amount in paise → verify razorpay_payment_id, razorpay_order_id, razorpay_signature. No payment SDK is installed yet (react-native-razorpay).
- Order confirmed: "Your order is confirmed!", "Order #MJ… Placed on 9 Oct 2026, 9:41 AM", Estimated delivery "26 minutes", the 4-step bar (Order confirmed → Preparing your food → Out for delivery → Delivered), Track order / View receipt, the item list with options and Total paid.
- Profile → Payment Methods: saved cards ("Visa •••• 4242", Expires 12/26, Default) and UPI.
- Not in the app yet: payment failed / pending / retry screens — the API supports them (Payment failed, Retry payment).

Call flow

- Pay ₹X: `POST /orders` with `paymentMethod` and a new `Idempotency-Key` header per tap. Disable the button until it answers.
- Cash on delivery: the order is placed → open Order confirmed with `order.orderId`.
- Online: open Razorpay checkout with `payment.keyId`, `payment.gatewayOrderId` (order_id), `payment.amountPaise`, `payment.prefill`, `payment.description`.
- Success → `POST /payments/verify` with the three razorpay_* values → Order confirmed. Closed / declined → `POST /payments/failed`, show "Payment failed" with Retry → `POST /orders/{orderId}/retry-payment` and open checkout again.
- `paymentError` on placement = could not open payment: show Retry payment for that order.
- Order confirmed: `GET /orders/{orderId}` (placedAtLabel, etaLabel, progress, items with optionsText, paymentMethodLabel, amountPaidPaise). View receipt: `GET /orders/{orderId}/receipt` (`?format=pdf`). Track order: step 10.
- Save this payment method: after Verify, `POST /payment-methods` with the `paymentId`; list / default / delete on Profile → Payment Methods.
- Locally (no Razorpay keys) use `POST /payments/test/complete` in place of the Razorpay sheet.

APIs in this step

#	API	Auth	Used for
S09.1	`POST /orders`	Token	Place order (Pay ₹X)
S09.2	`POST /payments/test/complete`	Token	Test gateway: pay
S09.3	`POST /payments/verify`	Token	Verify payment
S09.4	`POST /payments/failed`	Token	Payment failed
S09.5	`POST /orders/{{orderId}}/retry-payment`	Token	Retry payment
S09.6	`GET /payment-methods`	Token	My payment methods
S09.7	`POST /payment-methods`	Token	Save the method just used
S09.8	`PATCH /payment-methods/{{id}}/default`	Token	Make a method default
S09.9	`DELETE /payment-methods/{{id}}`	Token	Delete a method
S09.10	`GET /payments/test/links/{{id}}`	Token	Test gateway: payment link page
S09.11	`GET /wallet`	Token	Points wallet card

S09.1 · Place order (Pay ₹X)

`POST /api/v1/orders`

Authenticated. Places the order from the server cart with the checkout choices saved in step 8 (address, delivery time, tip, points, note). Body: `paymentMethod` (upi | card | netbanking | wallet | cod, from the cart's `paymentMethods`), optional `platform`.

Send a new `Idempotency-Key` header (a UUID) per tap on Pay: retrying with the same key returns the same order. A second Pay while one is being placed gets 409 `ORDER_IN_PROGRESS`.

Cash on delivery → `order.status` placed (or accepted), `payment` null, `amountDueOnDeliveryPaise`. Go to Order confirmed.

Online → `order.status` payment_pending and `payment`: open Razorpay checkout with `keyId`, `gatewayOrderId` (order_id), `amountPaise`, `currency`, `prefill`, `description`. Then Verify payment (success) or Payment failed (closed / declined).

`paymentError` (not null) = the payment page could not be opened; the order exists — show Retry payment.

409 with `errors[].message` = the cart blocker code (EMPTY, ITEMS_UNAVAILABLE, KITCHEN_CLOSED, ADDRESS_NOT_SERVICEABLE, SLOT_UNAVAILABLE, COUPON_INVALID…) and a message to show; 422 for an unavailable payment method.

Optional `savePaymentMethod` (true when "Save this payment method" is ticked). `payment` adds `customerId`, `canSaveMethod` and `checkoutOptions` — pass them to Razorpay checkout as they are.

The order has `totalLabel` + `totalPaise` for the summary's last row: "Total paid" (paid online, or cash collected), "To pay on delivery" (cash, not yet paid), "Refunded", or "Total to pay".

Request body

```
{
  "paymentMethod": "upi",
  "platform": "android"
}
```

Response · Online (UPI) (201)


```
{
  "success": true,
  "message": "Order created.",
  "data": {
    "order": {
      "orderId": "6acf1a…",
      "orderNumber": "MJ-10247",
      "status": "payment_pending",
      "statusLabel": "Waiting for payment",
      "kitchen": {
        "kitchenId": "6acc8b…",
        "name": "Indiranagar Kitchen"
      },
      "items": [
        {
          "lineId": "39b132b96f35",
          "kind": "dish",
          "dishId": "6acc8b…",
          "comboId": null,
          "name": "Masala Papad",
          "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
          "isVeg": true,
          "qty": 3,
          "portion": null,
          "mealUpgrade": null,
          "options": [],
          "specialInstructions": null,
          "unitPricePaise": 7900,
          "totalPaise": 23700,
          "optionsText": "Regular"
        }
      ],
      "itemCount": 3,
      "placedAtLabel": "14 Oct 2026, 11:29 AM",
      "etaLabel": "28 minutes",
      "progress": [
        {
          "key": "confirmed",
          "label": "Order confirmed",
          "at": null,
          "time": null,
```

```
          "state": "pending"
        },
        {
          "key": "preparing",
          "label": "Preparing your food",
  …
```


Response · Cash on delivery (201)


```
{
  "success": true,
  "message": "Order created.",
  "data": {
    "order": {
      "orderId": "6acf1a…",
      "orderNumber": "MJ-10246",
      "status": "placed",
      "statusLabel": "Order placed",
      "kitchen": {
        "kitchenId": "6acc8b…",
        "name": "Indiranagar Kitchen"
      },
      "items": [
        {
          "lineId": "43e4927bcab9",
          "kind": "dish",
          "dishId": "6acc8b…",
          "comboId": null,
          "name": "Masala Papad",
          "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
          "isVeg": true,
          "qty": 3,
          "portion": null,
          "mealUpgrade": null,
          "options": [],
          "specialInstructions": null,
          "unitPricePaise": 7900,
          "totalPaise": 23700,
          "optionsText": "Regular"
        }
      ],
      "itemCount": 3,
      "placedAtLabel": "14 Oct 2026, 11:29 AM",
      "etaLabel": "28 minutes",
      "progress": [
        {
          "key": "confirmed",
          "label": "Order confirmed",
          "at": "2026-10-14T05:59:15.744Z",
          "time": "11:29 AM",
          "state": "done"
        },
        {
          "key": "preparing",
          "label": "Preparing your food",
  …
```


Errors to handle

Case	Status	Message shown to the user
Empty cart (409)	409	Your cart is empty (field: cart)

S09.2 · Test gateway: pay

`POST /api/v1/payments/test/complete`

Development only (no Razorpay keys on the server): does what the Razorpay sheet does — returns `gatewayPaymentId` and `signature` for the gateway order (and verifies it). `outcome: "failure"` simulates a declined payment. Not available in production.

Request body

```
{
  "gatewayOrderId": "{{gatewayOrderId}}",
  "outcome": "success"
}
```

Response · Paid (test) (200)

```
{
  "success": true,
  "message": "Test payment completed.",
  "data": {
    "gatewayOrderId": "order_test_4f5b31d9f55e23",
    "gatewayPaymentId": "pay_test_4f07387bad88ab",
    "signature": "dfe8837e171d384740426d4214bbaafdc4b9d1c2a36f2d05bee5724b3dde64cb",
    "verified": {
      "paymentId": "6acf1a…",
      "status": "captured",
      "refType": "order",
      "refId": "6acf1a…"
    }
  }
}
```

S09.3 · Verify payment

`POST /api/v1/payments/verify`

Authenticated. Right after Razorpay checkout succeeds, send `razorpay_order_id` → `gatewayOrderId`, `razorpay_payment_id` → `gatewayPaymentId`, `razorpay_signature` → `signature`. The order becomes placed and the receipt is issued; then open Order confirmed with the order id.

Safe to call twice; the server also learns the result from Razorpay's webhook, so the order is confirmed even if the app closes. 400 when the signature is wrong; 404 for someone else's payment.

If the same order ends up paid twice (an old checkout finished after a retry), the extra payment is refunded automatically.

Request body

```
{
  "gatewayOrderId": "{{gatewayOrderId}}",
  "gatewayPaymentId": "{{gatewayPaymentId}}",
  "signature": "{{signature}}"
}
```

Response · Verified (200)

```
{
  "success": true,
  "message": "Payment verified.",
  "data": {
    "paymentId": "6ac8d2…",
    "status": "captured",
    "refType": "order",
    "refId": "6ac8d2…"
  }
}
```

Errors to handle

Case	Status	Message shown to the user
Wrong signature (400)	400	Payment could not be verified

S09.4 · Payment failed

`POST /api/v1/payments/failed`

Authenticated. Razorpay sheet closed or declined: send the `gatewayOrderId` and the error description as `reason`. The order waits for payment (offer and points are kept for a while); show “Payment failed” with Retry payment.

Request body

```
{
  "gatewayOrderId": "{{gatewayOrderId}}",
  "reason": "Payment cancelled by the user"
}
```

Response · Marked failed (200)

```
{
  "success": true,
  "message": "Payment marked failed.",
  "data": {
    "status": "failed"
  }
}
```

S09.5 · Retry payment

`POST /api/v1/orders/{{orderId}}/retry-payment`

Authenticated. Retry payment on a failed / unfinished payment: returns a new `payment` (open checkout again). 409 when the order no longer needs payment or a dish sold out meanwhile.

Response · New checkout (200)

```
{
  "success": true,
  "message": "Payment restarted.",
  "data": {
    "order": {
      "orderId": "6ac8d2…",
      "orderNumber": "MJ-10097",
      "status": "payment_pending",
      "statusLabel": "Waiting for payment",
      "kitchen": {
        "kitchenId": "6ac4ba…",
        "name": "Indiranagar Kitchen"
      },
      "items": [
        {
          "lineId": "538a9db8f71e",
          "kind": "dish",
          "dishId": "6ac76e…",
          "comboId": null,
          "name": "Masala Papad",
          "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
          "isVeg": true,
          "qty": 4,
          "portion": null,
          "mealUpgrade": null,
          "options": [],
          "specialInstructions": null,
          "unitPricePaise": 7900,
          "totalPaise": 31600,
          "optionsText": "Regular"
        }
      ],
      "itemCount": 4,
      "placedAtLabel": "9 Oct 2026, 5:10 PM",
      "etaLabel": "29 minutes",
      "progress": [
        {
          "key": "confirmed",
          "label": "Order confirmed",
          "at": null,
          "time": null,
          "state": "pending"
        },
        {
          "key": "preparing",
          "label": "Preparing your food",
```

```
  …
```

S09.6 · My payment methods

`GET /api/v1/payment-methods`

Authenticated. Profile → Payment Methods: `paymentMethodId`, `type` (card | upi), `label` (“Visa •••• 4242”), `network`, `last4`, `expMonth`, `expYear`, `vpaMasked`, `isDefault`, `expired`. Default first.

Response · Methods (200)

```
{
  "success": true,
  "message": "Saved methods.",
  "data": [
    {
      "paymentMethodId": "6ac8d2…",
      "type": "card",
      "label": "Visa •••• 4242",
      "network": "Visa",
      "issuer": null,
      "last4": "4242",
      "expMonth": 12,
      "expYear": 2029,
      "vpaMasked": null,
      "isDefault": true,
      "expired": false
    }
  ]
}
```

S09.7 · Save the method just used

`POST /api/v1/payment-methods`

Authenticated. “Save this payment method for faster checkout”: after a successful payment send its `paymentId` (from Verify payment). With Razorpay the card/UPI token is taken from that payment (checkout must be opened with saving enabled for a Razorpay customer); on the test gateway the app may send `type`, `network`, `last4`. Up to 10 methods; the first becomes default.

Usually not needed: send `savePaymentMethod: true` with POST /orders when “Save this payment method” is ticked and open Razorpay checkout with `payment.checkoutOptions` (customer_id + save: 1); the method is saved by itself once the payment is captured. Rows show `title` (“Visa •••• 4242” / “Google Pay”) and `subtitle` (“Expires 12/26” / “UPI linked to pr••••@okhdfcbank”).

Request body

```
{
  "paymentId": "{{paymentId}}",
  "makeDefault": true
}
```

Response · Saved (201)

```
{
  "success": true,
  "message": "Payment method saved.",
  "data": {
    "paymentMethodId": "6ac8d2…",
    "type": "card",
    "label": "Visa •••• 4242",
    "network": "Visa",
    "issuer": null,
    "last4": "4242",
    "expMonth": 12,
    "expYear": 2029,
    "vpaMasked": null,
    "isDefault": true,
    "expired": false
  }
}
```

S09.8 · Make a method default

`PATCH /api/v1/payment-methods/{{id}}/default`

Authenticated. Exactly one default.

S09.9 · Delete a method

`DELETE /api/v1/payment-methods/{{id}}`

Authenticated. Removes the saved card / UPI.

S09.10 · Test gateway: payment link page

`GET /api/v1/payments/test/links/{{id}}`

Development only: the hosted payment-link page for subscription cycles (step 14). Opening it marks the link paid.

S09.11 · Points wallet card

`GET /api/v1/wallet`

Authenticated. The "Meal ji wallet" card on Payment methods: `points` ("2,450"), `pointsValuePaise` (what they are worth), `tier`, `updatedLabel` ("Updated 2m ago"), `rewardsMessage` ("You have enough points for 3 free rewards!"), `affordableCount`. `moneyWallet` is null — there is no money wallet in v1, so the app hides the "Meal Ji Wallet ₹… available" row.

Response · Wallet card (200)


```
{
  "success": true,
  "message": "Wallet.",
  "data": {
    "points": 1300,
    "pointsValuePaise": 32500,
    "tier": "Silver Tandoor",
    "updatedLabel": "Updated 1d ago",
    "rewardsMessage": "You have enough points for 6 free rewards!",
    "affordableCount": 6,
    "moneyWallet": null
  }
}
```


Backend tests · 16/16 passed

Test	What the app needs	Result
09-01	**Cash on delivery: order placed at once, cart emptied** Pay ₹X with Cash on Delivery → Order confirmed	Pass
09-02	**Online payment: order waits for payment and returns checkout details** Pay ₹X with UPI → open the payment sheet	Pass
09-03	**Paying confirms the order, the receipt is issued** Payment success → Order confirmed, View receipt	Pass
09-04	**A wrong signature or someone else's payment is refused** Security: verify	Pass
09-05	**Payment failed → retry works; an old checkout paid later is refunded** Payment failed / Retry payment (no double charge)	Pass
09-06	**Double tap on Pay makes one order; the same Idempotency-Key replays it** Pay pressed twice	Pass
09-07	**Nothing to order or a blocked cart is refused with the reason** Empty cart / not deliverable	Pass
09-08	**Offer and points are carried into the order** Order total with coupon and points	Pass
09-09	**Scheduled order keeps the chosen slot** Schedule for later → Order confirmed shows the time	Pass

Test	What the app needs	Result
09-10	**Order confirmed screen has everything it shows** “Your order is confirmed!”, Order #MJ… Placed on …, Estimated delivery, 4-step bar, items, Total paid	**Pass**
09-11	**Wallets are offered like in the app** Payment Method: UPI / Card / Wallets / Cash on Delivery	**Pass**
09-12	**Save the card used, make it default, delete it** “Save this payment method for faster checkout“, Payment Methods screen	**Pass**
09-13	**Order summary total says what was paid or is due** “Total paid” / “To pay on delivery”	**Pass**
09-14	**Payment arriving while the customer cancels: never cancelled-and-paid without a refund** Pay ₹X vs Cancel at the same moment	**Pass**
09-15	**Two taps on Retry payment: one goes through** Retry payment (double tap)	**Pass**
09-16	**“Save this payment method”: checkout gets the customer, the method is saved after paying** Payment: Save this payment method → Payment methods (Google Pay · UPI linked to …)	**Pass**

Step 10 · Order tracking and history

Screens: **Order confirmed, Order status, Tracking, Order delivered, Orders list, Order details**

What the app screens need

- Orders tab: Current and Past tabs with counts, current card ("CHARRING · ARRIVING 9:25 PM", items, total, Track Order), past cards ("4 days ago", "Butter chicken bowl + butter naan", Delivered · ₹548, Reorder), empty state "No orders yet" with Popular Picks.
- Order status ("Live Kitchen"): "Right now: Charring the chicken.", "Arriving by 9:25 PM • 14 min", the kitchen's own steps with times (Order received → Marinating chicken → Charring in the tandoor → Plating & packing → Rider heading out).
- Tracking: ETA, rider card (name, rating, call, chat), preparation stages, order summary and address. The app has no map library; it already has a WebSocket client using track:subscribe, track:location and track:status — the same events the API sends.
- Order delivered: "Delivered at 9:23 PM · 2 min early", Food and Delivery stars, tags Delicious / Fresh / Warm / On time / Would reorder, Reorder, Submit.
- Order details is a placeholder in the app; View receipt and Track order buttons have no actions yet. No cancel UI exists yet (the API offers reasons).

Call flow

- Orders tab: `GET /orders?status=active|past&page=1&limit=10`; tab badges from `counts`; load more while `hasMore`. Empty → Popular dishes (`GET /menu/popular`).
- Track order / current card → Order status: `GET /orders/{id}/live`, and keep it fresh over the WebSocket (`track:subscribe` with the order id; refresh on `order:status`, `order:kitchen_step`, `track:location`).
- Tracking map: `GET /orders/{id}/tracking` (kitchen, destination, rider with photo and "4.9 (1,240 deliveries)", last position, `etaMinutes`). Call uses `callNumber` while `canCall` (out for delivery). In-app chat with the rider is not part of v1.
- Cancel (only while `canCancel`): `PATCH /orders/{id}/cancel` with one of `cancelReasons`.
- Order delivered (`canRate`): `POST /orders/{id}/rating` with `foodRating`, `deliveryRating`, `tags` from `ratingTags`.
- Reorder: `POST /orders/{id}/reorder` (on 409 CART_KITCHEN_MISMATCH ask, then `replaceCart: true`) → open Cart.
- View receipt: `GET /orders/{id}/receipt?format=pdf`; all invoices: `GET /invoices`.

APIs in this step

#	API	Auth	Used for
S10.1	`GET /orders?status=past&page=1&limit=10`	Token	My orders (Current / Past)
S10.2	`GET /orders/{{orderId}}`	Token	Order details
S10.3	`GET /orders/{{orderId}}/live`	Token	Live status
S10.4	`GET /orders/{{orderId}}/tracking`	Token	Tracking map
S10.5	`PATCH /orders/{{orderId}}/cancel`	Token	Cancel order
S10.6	`POST /orders/{{orderId}}/rating`	Token	Rate the order
S10.7	`POST /orders/{{orderId}}/reorder`	Token	Reorder
S10.8	`GET /orders/{{orderId}}/receipt`	Token	View receipt
S10.9	`GET /invoices`	Token	My invoices
S10.10	`GET /invoices/{{id}}`	Token	One invoice
S10.11	`GET /invoices/{{id}}/pdf`	Token	Invoice PDF

S10.1 · My orders (Current / Past)

`GET /api/v1/orders?status=past&page=1&limit=10`

Authenticated. Orders tab. `status`: active (Current tab: waiting for payment … out for delivery) or past (delivered, cancelled, payment failed); omit for all. `page` + `limit` (≤50), newest first.

Returns `items`, `total`, `hasMore` and `counts` { `active`, `past` } for the tab badges. Each order card: `orderNumber`, `dateLabel` ("4 days ago"), `title` ("Butter Chicken Bowl + Butter Naan"), `statusLabel`, `itemCount`, `bill.grandTotalPaise`, `arrivingByLabel` ("9:25 PM", current orders), `canRate`.

Empty list → show the empty state with Popular dishes (GET /menu/popular).

Each order also has `timesOrdered` and `timesOrderedLabel` ("Ordered 3x" when the same dishes were delivered before, else null).

Response · Past (200)


```
{
  "success": true,
  "message": "Orders fetched.",
  "data": {
    "items": [
      {
        "orderId": "6acc9c…",
        "orderNumber": "MJ-10128",
        "status": "cancelled",
        "statusLabel": "Cancelled",
        "kitchen": {
          "kitchenId": "6acc8b…",
          "name": "Koramangala Kitchen"
        },
        "items": [
          {
            "lineId": "92ba276a5896",
            "kind": "dish",
            "dishId": "6acc8b…",
            "comboId": null,
            "name": "Masala Papad",
            "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
            "isVeg": true,
            "qty": 3,
            "portion": null,
            "mealUpgrade": null,
            "options": [],
            "specialInstructions": null,
            "unitPricePaise": 7900,
            "totalPaise": 23700,
            "optionsText": "Regular"
          }
        ],
        "itemCount": 3,
        "placedAtLabel": "12 Oct 2026, 2:08 PM",
        "etaLabel": "28 minutes",
        "progress": [
          {
            "key": "confirmed",
            "label": "Order confirmed",
            "at": "2026-10-12T08:38:27.847Z",
            "time": "2:08 PM",
            "state": "cancelled"
          },
          {
            "key": "preparing",
  …
```


Response · Current (200)


```
{
  "success": true,
  "message": "Orders fetched.",
  "data": {
    "items": [
      {
        "orderId": "6acf1a…",
        "orderNumber": "MJ-10248",
        "status": "placed",
        "statusLabel": "Order placed",
        "kitchen": {
```

```
          "kitchenId": "6acc8b…",
          "name": "Indiranagar Kitchen"
        },
        "items": [
          {
            "lineId": "d1180738d21e",
            "kind": "dish",
            "dishId": "6acc8b…",
            "comboId": null,
            "name": "Masala Papad",
            "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
            "isVeg": true,
            "qty": 3,
            "portion": null,
            "mealUpgrade": null,
            "options": [],
            "specialInstructions": null,
            "unitPricePaise": 7900,
            "totalPaise": 23700,
            "optionsText": "Regular"
          }
        ],
        "itemCount": 3,
        "placedAtLabel": "14 Oct 2026, 11:29 AM",
        "etaLabel": "29 minutes",
        "progress": [
          {
            "key": "confirmed",
            "label": "Order confirmed",
            "at": "2026-10-14T05:59:16.097Z",
            "time": "11:29 AM",
            "state": "done"
          },
          {
            "key": "preparing",
  …
```


S10.2 · Order details

`GET /api/v1/orders/{{orderId}}`

Authenticated. Order confirmed / Order details / Order delivered screens: `orderNumber`, `placedAtLabel`, `etaLabel`, `progress` (4 steps), `items` (`optionsText`), `bill` (+ `billLines` style amounts), `address`, `paymentMethodLabel`, `amountPaidPaise` / `amountDueOnDeliveryPaise`, `rider`, `kitchenSteps`, `currentActivity`, `statusHistory`.

Order delivered: `deliveredAtLabel` + `punctualityLabel` (“Delivered at 9:23 PM · 2 min early”), `canRate`, `ratingTags` (Delicious, Fresh, Warm, On time, Would reorder), `rating` once rated. 404 for someone else's order.

Response · Delivered order (200)


```
{
  "success": true,
  "message": "Order fetched.",
  "data": {
    "orderId": "6acc8b…",
    "orderNumber": "MJ-10012",
    "status": "delivered",
    "statusLabel": "Delivered",
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Koramangala Kitchen"
    },
    "items": [
      {
        "lineId": "68d5c768a287",
        "kind": "dish",
        "dishId": "6acc8b…",
        "comboId": null,
        "name": "Butter Chicken Bowl",
        "imageUrl": "https://cdn…/mealji/seed/butter-chicken-bowl.jpg",
        "isVeg": false,
        "qty": 1,
        "portion": {
          "portionId": "regular",
          "label": "Regular"
        },
        "mealUpgrade": null,
```

```
        "options": [
          {
            "optionId": "869d54b75d",
            "name": "Extra Butter Jeera Rice",
            "groupName": "Choose Rice Portion",
            "pricePaise": 4000
          }
        ],
        "specialInstructions": null,
        "unitPricePaise": 38900,
        "totalPaise": 38900,
        "optionsText": "Regular • Extra Butter Jeera Rice"
      },
      {
        "lineId": "bf83c35dd3b7",
        "kind": "dish",
        "dishId": "6acc8b…",
        "comboId": null,
        "name": "Jeera Rice",
  …
```


S10.3 · Live status

`GET /api/v1/orders/{{orderId}}/live`

Authenticated. Order status ("Live Kitchen") screen; poll every 15–30 s or listen on the WebSocket (below).

`status`, `statusLabel`, `timeline` (placed → accepted → preparing → ready → out for delivery → delivered with `done` and `at`), `kitchenSteps` (the kitchen's own steps, e.g. Marinating → Charring in the tandoor → Plating & packing, with `state` and `at`), `currentActivity` ("Right now: Charring the chicken."), `etaMinutes`, `arrivingByLabel` ("Arriving by 9:25 PM • 14 min"), `rider`, `canCancel` and `cancelReasons`.

WebSocket `wss://<host>/ws?token=<accessToken>`: send `{"type":"track:subscribe","data":{"orderId":"…"}}`; receive `order:status`, `order:kitchen_step`, `track:status` (status text), `order:rider_assigned`, `track:location` ({ lat, lng, heading, speed, updatedAt }). Send `track:unsubscribe` when leaving.

For the Live Kitchen screen: `liveSteps` [{ label, state done|active|pending, timeLabel "9:04 PM" / "Now" }] already in the app's order (Order received → the kitchen's steps → Rider heading out, or Ready for pickup); `activityText` ("Right now: …"); `arrivingAtLabel` ("9:25 PM") with `etaMinutes` ("• 14 min"); `timeline[].time`; `kitchenSteps[].time`. `rider` as in S10.4.

Response · Just placed (200)


```
{
  "success": true,
  "message": "Live status.",
  "data": {
    "orderId": "6acf1a…",
    "orderNumber": "MJ-10248",
    "status": "placed",
    "statusLabel": "Order placed",
    "timeline": [
      {
        "status": "placed",
        "label": "Order placed",
        "done": true,
        "at": "2026-10-14T05:59:16.097Z",
        "time": "11:29 AM"
      },
      {
        "status": "accepted",
        "label": "Accepted by the kitchen",
        "done": false,
        "at": null,
        "time": null
      },
      {
        "status": "preparing",
        "label": "Being prepared",
        "done": false,
        "at": null,
        "time": null
      },
      {
        "status": "ready",
        "label": "Ready",
        "done": false,
```

```
        "at": null,
        "time": null
      },
      {
        "status": "dispatched",
        "label": "Out for delivery",
        "done": false,
        "at": null,
        "time": null
      },
      {
        "status": "delivered",
  …
```


Response · Being prepared (200)


```
{
  "success": true,
  "message": "Live status.",
  "data": {
    "orderId": "6acf1a…",
    "orderNumber": "MJ-10248",
    "status": "preparing",
    "statusLabel": "Being prepared",
    "timeline": [
      {
        "status": "placed",
        "label": "Order placed",
        "done": true,
        "at": "2026-10-14T05:59:16.097Z",
        "time": "11:29 AM"
      },
      {
        "status": "accepted",
        "label": "Accepted by the kitchen",
        "done": true,
        "at": "2026-10-14T05:59:16.155Z",
        "time": "11:29 AM"
      },
      {
        "status": "preparing",
        "label": "Being prepared",
        "done": true,
        "at": "2026-10-14T05:59:16.179Z",
        "time": "11:29 AM"
      },
      {
        "status": "ready",
        "label": "Ready",
        "done": false,
        "at": null,
        "time": null
      },
      {
        "status": "dispatched",
        "label": "Out for delivery",
        "done": false,
        "at": null,
        "time": null
      },
      {
        "status": "delivered",
  …
```

S10.4 · Tracking map

`GET /api/v1/orders/{{orderId}}/tracking`

Authenticated. Tracking screen: `kitchen` (name, latitude, longitude), `destination` (latitude, longitude, fullAddress), `rider` (name, masked phone, vehicle), `location` (rider's last position, also pushed as `track:location`), `trackingUrl` (delivery partner's page when available), `estimatedDeliveryAt`.

`rider`: `name`, `photoUrl`, `rating`, `deliveriesCount`, `ratingLabel` ("4.9 (1,240 deliveries)"), `vehicleNumber`, `phoneMasked`, `canCall` + `callNumber` (only while the order is out for delivery). Also `arrivingAtLabel` ("8:38 PM") and `etaMinutes` ("Arriving in 12 mins").

Response · Tracking (200)


```
{
  "success": true,
  "message": "Tracking.",
  "data": {
    "orderId": "6acf1a…",
    "status": "dispatched",
    "kitchen": {
      "name": "Indiranagar Kitchen",
      "latitude": 12.9719,
      "longitude": 77.6412
    },
    "destination": {
      "latitude": 12.9689,
      "longitude": 77.6332,
      "fullAddress": "No. 31, 6th Cross, Defence Colony, Indiranagar, Bengaluru, Karnataka, 5…"
    },
    "rider": {
      "name": "Ramesh",
      "photoUrl": "https://cdn…/mealji/seed/chef.png",
      "rating": 4.9,
      "deliveriesCount": 1240,
      "ratingLabel": "4.9 (1,240 deliveries)",
      "vehicleNumber": "KA01AB1234",
      "phoneMasked": "98******10",
      "canCall": true,
      "callNumber": "+919876543210"
    },
    "location": null,
    "trackingUrl": null,
    "deliveryStatus": "assigned",
    "estimatedDeliveryAt": "2026-10-14T06:28:16.155Z",
    "arrivingAtLabel": "11:58 AM",
    "etaMinutes": 29
  }
}
```


S10.5 · Cancel order

`PATCH /api/v1/orders/{{orderId}}/cancel`

Authenticated. Only while `canCancel` is true (Live status); send one of `cancelReasons` as `reason`. A paid order is refunded automatically. 409 once the kitchen has started (show "Contact support").

Request body


```
{
  "reason": "Ordered by mistake"
}
```


Errors to handle

Case	Status	Message shown to the user
Too late (409)	409	This order can no longer be cancelled here. Contact support.

S10.6 · Rate the order

`POST /api/v1/orders/{{orderId}}/rating`

Authenticated. Order delivered → "How was it?": `foodRating` 1–5, `deliveryRating` 1–5, `tags` (from `ratingTags`), optional `comment` (≤500) and `dishRatings` [{ dishId, rating }]. Once per order, within the rating window; 409 before delivery or when already rated.

Request body

```
{
  "foodRating": 5,
  "deliveryRating": 5,
  "tags": [
    "Delicious",
    "On time"
  ],
  "comment": null
}
```

Response · Rated (200)

```
{
  "success": true,
  "message": "Thanks for rating.",
  "data": {
    "orderId": "6accbb…",
    "orderNumber": "MJ-10196",
    "status": "delivered",
    "statusLabel": "Delivered",
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Indiranagar Kitchen"
    },
    "items": [
      {
        "lineId": "65b18b1ea4c7",
        "kind": "dish",
        "dishId": "6acc8b…",
        "comboId": null,
        "name": "Masala Papad",
        "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
        "isVeg": true,
        "qty": 2,
        "portion": null,
        "mealUpgrade": null,
        "options": [],
        "specialInstructions": null,
        "unitPricePaise": 7900,
        "totalPaise": 15800,
        "optionsText": "Regular"
      }
    ],
    "itemCount": 2,
    "placedAtLabel": "12 Oct 2026, 4:22 PM",
    "etaLabel": "29 minutes",
    "progress": [
      {
        "key": "confirmed",
        "label": "Order confirmed",
        "at": "2026-10-12T10:52:47.926Z",
        "time": "4:22 PM",
        "state": "done"
      },
      {
        "key": "preparing",
        "label": "Preparing your food",
        "at": "2026-10-12T10:52:47.975Z",
  …
```

Errors to handle

Case	Status	Message shown to the user
Already rated (409)	409	You already rated this order

S10.7 · Reorder

`POST /api/v1/orders/{{orderId}}/reorder`

Authenticated. Puts the order's items back in the cart (what can still be ordered; `skipped` + `message` list the rest), then open Cart. When the cart holds another kitchen's food: 409 `CART_KITCHEN_MISMATCH` — ask, then send `replaceCart: true`.

Request body

```
{
  "replaceCart": false
}
```

Response · Back in the cart (200)


```
{
  "success": true,
  "message": "Added to your cart.",
  "data": {
    "cart": {
      "cartId": "6acc8c…",
      "kitchen": {
        "kitchenId": "6acc8b…",
        "name": "Indiranagar Kitchen",
        "isOpenNow": true,
        "message": null
      },
      "items": [
        {
          "lineId": "cd2be8ae4d75",
          "kind": "dish",
          "dishId": "6acc8b…",
          "name": "Masala Papad",
          "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
          "isVeg": true,
          "qty": 2,
          "portion": null,
          "mealUpgrade": null,
          "options": [],
          "specialInstructions": null,
          "unitPricePaise": 7900,
          "basePricePaise": 7900,
          "originalUnitPricePaise": 7900,
          "packagingPaise": 0,
          "totalPaise": 15800,
          "isAvailable": true,
          "issue": null,
          "priceChanged": false,
          "dishIds": [
            {
              "dishId": "6acc8b…",
              "qty": 2
            }
          ],
          "optionsText": "Regular"
        }
      ],
      "itemCount": 2,
      "coupon": null,
      "bill": {
        "itemCount": 2,
  …
```

S10.8 · View receipt

`GET /api/v1/orders/{{orderId}}/receipt`

Authenticated. The GST invoice of a paid order; `?format=pdf` returns the PDF to show or share. 404 until the order is paid.

Response · Receipt (200)


```
{
  "success": true,
  "message": "Receipt fetched.",
  "data": {
    "invoiceId": "6acc8b…",
    "invoiceNumber": "MJ/2026-27/000011",
    "financialYear": "2026-27",
    "refType": "order",
    "refId": "6acc8b…",
    "periodLabel": null,
    "date": "2026-10-11",
    "issuedAt": "2026-10-11T07:09:00.000Z",
    "entity": {
      "entityId": "6acc8b…",
      "legalName": "MealJi Foods Private Limited",
      "tradeName": "Meal Ji",
      "gstin": "29AAQCM4821K1Z7",
      "fssai": "11225999000123",
      "pan": "AAQCM4821K",
      "addressLine": "3rd Floor, 80 Feet Road, 4th Block, Koramangala",
      "city": "Bengaluru",
      "state": "Karnataka",
      "stateCode": "29",
      "pincode": "560034",
      "email": "billing@mealji.example",
      "phone": "08040001234",
      "invoicePrefix": "MJ",
      "logoUrl": "https://cdn…/mealji/seed/logo-full.png",
      "signatory": "Authorised signatory",
      "bank": {
        "accountName": null,
        "accountNumber": null,
        "ifsc": null
      },
      "isDefault": true,
      "isActive": true,
      "updatedAt": "2026-10-12T07:27:29.760Z"
    },
    "customer": {
      "name": "Rahul Sharma",
      "phone": "9000000031",
      "address": "Flat 302, Lakeview Residency, 5th Cross, 6th Block, Koramangala, Bengal…",
      "state": "Karnataka",
      "email": null
    },
    "lines": [
  …
```


S10.9 · My invoices

`GET /api/v1/invoices`

Authenticated. All of the customer's invoices (orders and Meal Ji Plus), newest first, paged.

Response · Invoices (200)


```
{
  "success": true,
  "message": "Invoices fetched.",
  "data": {
    "items": [
      {
        "invoiceId": "6acc8b…",
        "invoiceNumber": "MJ/2026-27/000057",
        "financialYear": "2026-27",
        "refType": "subscription",
        "refId": "6acc8b…",
        "periodLabel": "2026-10-13 to 2026-11-11",
        "date": "2026-10-12",
        "issuedAt": "2026-10-12T07:27:52.716Z",
```

```
      "entity": {
        "entityId": "6acc8b…",
        "legalName": "MealJi Foods Private Limited",
        "tradeName": "Meal Ji",
        "gstin": "29AAQCM4821K1Z7",
        "fssai": "11225999000123",
        "pan": "AAQCM4821K",
        "addressLine": "3rd Floor, 80 Feet Road, 4th Block, Koramangala",
        "city": "Bengaluru",
        "state": "Karnataka",
        "stateCode": "29",
        "pincode": "560034",
        "email": "billing@mealji.example",
        "phone": "08040001234",
        "invoicePrefix": "MJ",
        "logoUrl": "https://cdn…/mealji/seed/logo-full.png",
        "signatory": "Authorised signatory",
        "bank": {
          "accountName": null,
          "accountNumber": null,
          "ifsc": null
        },
        "isDefault": true,
        "isActive": true,
        "updatedAt": "2026-10-12T07:27:29.760Z"
      },
      "customer": {
        "name": "Rahul Sharma",
        "phone": "9000000031",
        "email": null,
        "address": "Flat 302, Lakeview Residency, 5th Cross, 6th Block, Koramangala, Bengal…",
        "state": "Karnataka"
  …
```


S10.10 · One invoice

`GET /api/v1/invoices/{{id}}`

Authenticated. 404 for someone else's invoice.

Response · Invoice (200)


```
{
  "success": true,
  "message": "Invoice fetched.",
  "data": {
    "invoiceId": "6acc8b…",
    "invoiceNumber": "MJ/2026-27/000011",
    "financialYear": "2026-27",
    "refType": "order",
    "refId": "6acc8b…",
    "periodLabel": null,
    "date": "2026-10-11",
    "issuedAt": "2026-10-11T07:09:00.000Z",
    "entity": {
      "entityId": "6acc8b…",
      "legalName": "MealJi Foods Private Limited",
      "tradeName": "Meal Ji",
      "gstin": "29AAQCM4821K1Z7",
      "fssai": "11225999000123",
      "pan": "AAQCM4821K",
      "addressLine": "3rd Floor, 80 Feet Road, 4th Block, Koramangala",
      "city": "Bengaluru",
      "state": "Karnataka",
      "stateCode": "29",
      "pincode": "560034",
      "email": "billing@mealji.example",
      "phone": "08040001234",
      "invoicePrefix": "MJ",
      "logoUrl": "https://cdn…/mealji/seed/logo-full.png",
      "signatory": "Authorised signatory",
      "bank": {
        "accountName": null,
        "accountNumber": null,
        "ifsc": null
      },
      "isDefault": true,
      "isActive": true,
```

```
    "updatedAt": "2026-10-12T07:27:29.760Z"
  },
  "customer": {
    "name": "Rahul Sharma",
    "phone": "9000000031",
    "address": "Flat 302, Lakeview Residency, 5th Cross, 6th Block, Koramangala, Bengal…",
    "state": "Karnataka",
    "email": null
  },
  "lines": [
…
```


S10.11 · Invoice PDF

`GET /api/v1/invoices/{{id}}/pdf`

Authenticated. The invoice as a PDF.

Backend tests · 14/14 passed

Test	What the app needs	Result
10-01	**Orders list: newest first, Active and Past tabs, paged** Orders tab	**Pass**
10-02	**Each order card has what the list shows** Past card: “4 days ago”, “Butter chicken bowl + butter naan”, Delivered · ₹548, Reorder	**Pass**
10-03	**Order details: bill, address, payment, timeline** Order details screen	**Pass**
10-04	**Someone else's order is not visible** Security	**Pass**
10-05	**Live status follows the kitchen: placed → preparing → out for delivery → delivered** Order status / Tracking screens	**Pass**
10-06	**Tracking: kitchen, destination, rider and live position** Tracking map	**Pass**
10-07	**Delivered: rate the order once (double tap safe)** Order delivered → rate food and delivery	**Pass**
10-08	**Only delivered orders can be rated** Rate before delivery	**Pass**
10-09	**Cancel while allowed; refused once the kitchen has started** Cancel order	**Pass**
10-10	**Reorder fills the cart; another kitchen's cart is not silently replaced** Reorder	**Pass**
10-11	**Receipt and invoices for a paid order** View receipt / Invoices	**Pass**
10-12	**Live Kitchen list, “Right now”, arrival time and tracking ETA** Order status: Order received → steps → Rider heading out, 9:04 PM / Now	**Pass**
10-13	**Rider card: photo, “4.9 (1,240 deliveries)”, Call only while on the way** Tracking: rider card with Call	**Pass**
10-14	**Past orders: “Ordered 3x” and the app's rating chips** Orders list badge, rated tags	**Pass**

Step 11 · Notifications

Screens: **Notifications (and the bell on Home)**

What the app screens need

- Notifications screen: tabs All / Orders / Offers / Rewards / Account, sections Today / Yesterday / This Week / Earlier, rows with icon (box, moto, star, gift, crown, heart, megaphone), title, time ("2m ago", "Yesterday, 5:30 PM", "Mon, 12 Sep", "5 Sep"), body and an unread dot, and "Mark all as read".
- Push: the app has a Firebase placeholder (services/firebase/firebase.ts) — @react-native-firebase/messaging is not installed yet; it will give an FCM token to register.

Call flow

- Bell badge: `unreadNotifications` from `GET /home` (or `GET /notifications/unread-count`).
- Notifications screen: `GET /notifications?category=<tab>&page=1&limit=20`; draw `tabs` with their unread counts, group rows by `group`, show `timeLabel` and `icon`; load more while `hasMore`.
- Tap a row: `PATCH /notifications/{id}/read`, then open `deepLink.url` (mealji://orders/{orderId} → Order details, mealji://offers, mealji://rewards/history…). "Mark all as read": `PATCH /notifications/read-all`.
- Push: after sign-in and on token refresh `POST /devices` (deviceId = the x-device-id value, fcmToken, platform, appVersion); on log out `DELETE /devices/{deviceId}` before `/auth/logout`. When a push is opened: `POST /notifications/track` with its `messageId`.
- Order updates (confirmed, cooking, ready, on the way, delivered), points and offers arrive in the inbox automatically; demo numbers never get SMS.

APIs in this step

#	API	Auth	Used for
S11.1	`GET /notifications?category=all&page=1&limit=20`	Token	Notifications
S11.2	`GET /notifications/unread-count`	Token	Unread count (bell badge)
S11.3	`PATCH /notifications/{{notificationId}}/read`	Token	Open a notification
S11.4	`PATCH /notifications/read-all`	Token	Mark all as read
S11.5	`POST /devices`	Token	Register this phone for push
S11.6	`DELETE /devices/{{deviceId}}`	Token	Stop push on this phone
S11.7	`POST /notifications/track`	Token	Push opened / clicked

S11.1 · Notifications

`GET /api/v1/notifications?category=all&page=1&limit=20`

Authenticated. The Notifications screen. `category`: all | orders | offers | rewards | account (the tabs); optional `unread=true`; `page` + `limit`.

Returns `items`, `total`, `hasMore`, `unreadCount` and `tabs` [{ `key`, `label`, `unreadCount` }] (All, Orders, Offers, Rewards, Account).

Each item: `title`, `body`, `icon` (the app's icon names: box, moto, star, gift, crown, heart, megaphone), `iconColor`, `group` (Today / Yesterday / This Week / Earlier — the section heading), `timeLabel` ("2m ago", "Yesterday, 5:30 PM", "Mon, 12 Sep", "5 Sep"), `isRead` (unread dot), `deepLink.url` (e.g. mealji://orders/{orderId} → Order details), `imageUrl`.

Order updates read as in the app: "Order Confirmed" ("Your order #MJ-… has been confirmed. We're cooking it fresh for you!"), "Out for Delivery" ("Your order #MJ-… is on the way! Get ready for a delicious meal."), "You Earned 50 Points!", "We'd Love Your Feedback" (heart).

Response · All (200)


```
{
  "success": true,
  "message": "Notifications fetched.",
  "data": {
    "items": [
      {
        "notificationId": "6accbc…",
```

```
        "category": "orders",
        "title": "Order cancelled",
        "body": "Order MJ-10212 was cancelled.",
        "icon": "box",
        "iconColor": "#EA580C",
        "group": "This Week",
        "timeLabel": "Mon, 12 Oct",
        "imageUrl": null,
        "deepLink": {
          "url": "mealji://orders/6accbc8d52a19767c12351f1"
        },
        "messageId": "a8b3760f-af54-4b23-988a-5b0a9a351d5b",
        "isRead": false,
        "createdAt": "2026-10-12T10:55:23.024Z"
      },
      {
        "notificationId": "6accbc…",
        "category": "orders",
        "title": "Out for Delivery",
        "body": "Your order #MJ-10212 is on the way!\nGet ready for a delicious meal.",
        "icon": "moto",
        "iconColor": "#EA580C",
        "group": "This Week",
        "timeLabel": "Mon, 12 Oct",
        "imageUrl": null,
        "deepLink": {
          "url": "mealji://orders/6accbc8d52a19767c12351f1"
        },
        "messageId": "e41b325b-bc71-4e12-8d91-9ad344635c77",
        "isRead": true,
        "createdAt": "2026-10-12T10:55:09.921Z"
      },
      {
        "notificationId": "6accbc…",
        "category": "orders",
        "title": "Ready",
        "body": "Order MJ-10212 is ready.",
        "icon": "box",
        "iconColor": "#EA580C",
  …
```


Response · Offers tab (200)


```
{
  "success": true,
  "message": "Notifications fetched.",
  "data": {
    "items": [
      {
        "notificationId": "6acc8c…",
        "category": "offers",
        "title": "Special Offer Just for You",
        "body": "Get ₹150 off on Butter Chicken Bowls. Use code FOODIE150.",
        "icon": "gift",
        "iconColor": "#EA580C",
        "group": "This Week",
        "timeLabel": "Sun, 11 Oct",
        "imageUrl": null,
        "deepLink": {
          "url": "mealji://offers"
        },
        "messageId": null,
        "isRead": false,
        "createdAt": "2026-10-11T05:28:01.845Z"
      }
    ],
    "page": 1,
    "limit": 20,
    "total": 1,
    "hasMore": false,
    "unreadCount": 51,
    "tabs": [
      {
        "key": "all",
        "label": "All",
        "unreadCount": 51
      },
      {
        "key": "orders",
```

```
        "label": "Orders",
        "unreadCount": 40
      },
      {
        "key": "offers",
        "label": "Offers",
        "unreadCount": 1
      },
      {
        "key": "rewards",
  …
```

S11.2 · Unread count (bell badge)

`GET /api/v1/notifications/unread-count`

Authenticated. `unreadCount` for the bell on Home (Home also returns it as `unreadNotifications`).

Response · Count (200)

```
{
  "success": true,
  "message": "Unread count.",
  "data": {
    "unreadCount": 51
  }
}
```

S11.3 · Open a notification

`PATCH /api/v1/notifications/{{notificationId}}/read`

Authenticated. Call when the user taps a row (then open `deepLink.url`). Returns the item and the new `unreadCount`. 404 for someone else's notification.

Response · Read (200)

```
{
  "success": true,
  "message": "Marked read.",
  "data": {
    "notificationId": "6accbb…",
    "category": "orders",
    "title": "Order cancelled",
    "body": "Order MJ-10190 was cancelled.",
    "icon": "box",
    "iconColor": "#EA580C",
    "group": "Today",
    "timeLabel": "3m ago",
    "imageUrl": null,
    "deepLink": {
      "url": "mealji://orders/6accbb1c52a19767c12324c5"
    },
    "messageId": "3b1f2462-0b74-4ad8-b2c6-c69cba569541",
    "isRead": true,
    "createdAt": "2026-10-12T10:49:17.240Z",
    "unreadCount": 49
  }
}
```

S11.4 · Mark all as read

`PATCH /api/v1/notifications/read-all`

Authenticated. “Mark all as read”. Returns `unreadCount: 0`.

Response · All read (200)

```
{
  "success": true,
  "message": "All read.",
  "data": {
    "unreadCount": 0
  }
}
```

S11.5 · Register this phone for push

`POST /api/v1/devices`

Authenticated. After sign-in and whenever Firebase gives a new token: `deviceId` (the same id as the x-device-id header), `fcmToken` (null when the user refused notifications), `platform` (android | ios | web), `appVersion`, `osVersion`.

If the same token was registered by another account on this phone, that registration is switched off (no pushes to the wrong person). The app does not have Firebase yet (services/firebase/firebase.ts is a placeholder: @react-native-firebase/messaging).

Request body

```
{
  "deviceId": "{{deviceId}}",
  "fcmToken": "<FCM registration token>",
  "platform": "android",
  "appVersion": "1.0.0",
  "osVersion": "14"
}
```

Response · Registered (200)

```
{
  "success": true,
  "message": "Device registered.",
  "data": {
    "deviceId": "postman-device-012",
    "registered": true
  }
}
```

Errors to handle

Case	Status	Message shown to the user
Bad platform (422)	422	Validation failed (field: deviceId, platform)

S11.6 · Stop push on this phone

`DELETE /api/v1/devices/{{deviceId}}`

Authenticated. Call on Log out (before the logout call).

Response · Removed (200)

```
{
  "success": true,
  "message": "Device removed.",
  "data": {
    "deviceId": "postman-device-012",
    "removed": true
  }
}
```

S11.7 · Push opened / clicked

`POST /api/v1/notifications/track`

Authenticated. When the user opens a push (its data has `messageId`): `event` opened | clicked, `channel` push. Used for campaign reports. 404 for a message that was not sent to this user.

Request body

```
{
  "messageId": "{{messageId}}",
  "event": "opened",
  "channel": "push"
}
```

Backend tests · 9/9 passed

Test	What the app needs	Result
11-01	**Inbox rows as the screen draws them** Groups Today / Yesterday / This Week / Earlier, “2m ago”, icon, unread dot	Pass
11-02	**Tabs with unread counts** All / Orders / Offers / Rewards / Account	Pass
11-03	**Paging** Scroll for older notifications	Pass
11-04	**Open one: it is read and the bell goes down; others' are hidden** Tap a notification	Pass
11-05	**Mark all as read** “Mark all as read”	Pass
11-06	**Order updates arrive in the inbox with the order link and icon** Order Confirmed / Out for Delivery notifications	Pass
11-07	**Demo numbers never get a real SMS** Safety: demo accounts	Pass
11-08	**Push: register this phone, move the token, remove it** App start / sign-in / log out	Pass
11-09	**Push opens are tracked only for my messages** Tap on a push	Pass

Step 12 · Rewards and referrals

Screens: **Rewards (Offers tab), Meal Ji points (View History), Refer a friend**

What the app screens need

- Rewards: points card “Your Meal Ji Points 2,450” with View History; Meal Ji Plus banner (Upgrade Now — subscriptions step); Redeem Rewards cards (image, two-line name, points, Redeem) with View All; Earn More Points rows: Place an order +50, Refer a friend +200, Write a review +100, Celebrate your birthday +250 (icons cart, user, star, gift).
- Meal Ji points: balance, “Updated 2m ago”, “You have enough points for 3 free rewards!”, Recent Activity with a This Month dropdown; rows show title (“Order #MJ-8921”, “Free Butter Chicken Bowl”, “Referral Bonus”, “Review Reward”), “Today, 10:45 AM” / “Oct 24, 2023”, ±points in green/red, Earned / Redeemed and an icon.
- Help: the FAQ list asks “How do Meal Ji Points work?” and has an “Offers & Rewards” help category — covered with the support step.
- The app's rewards and history are still hard-coded sample data; Redeem, View All and Upgrade Now have no actions yet. There is no Refer-a-friend or “enter a code” screen yet — the API is ready for both.

Call flow

- Rewards tab: `GET /rewards/overview` (points card, `message`, `rewards` with `canRedeem`/`pointsShort`, `earn`). Enable Redeem only when `canRedeem`. Each Earn More Points row has a `deepLink` for its chevron: mealji://menu, mealji://referrals, mealji://orders?status=past, mealji://profile/edit (add your birthday).
- View History: `GET /rewards/history?period=this_month&page=1&limit=20`; the dropdown uses `periods` (This Month / Last Month / All time); load more while `hasMore`.
- Redeem: `POST /rewards/{rewardId}/redeem` → show `couponCode` and `howToUse`; refresh the tab (`pointsLeft`). 409 = not enough points or a second tap.
- Using it: the code appears first in the checkout Offers list (`GET /promos/available`, `isReward: true`); apply with `POST /cart/promo { code }` — a free-dish reward adds the dish to the cart and takes its price off. “My rewards”: `GET /rewards/redemptions`.
- Refer a friend: `GET /referrals/me` → share `shareText` / `shareLink`, show `stats` and `referrals`. A new customer enters a code with `POST /referrals/apply { code }` (before their first delivered order).
- Points arrive by themselves (no app call): +50 when an order is delivered, +100 when it is rated, +200 to both sides on the referred friend's first delivered order, +250 on the birthday (needs `dob` from step 03). Cancelled orders take their points back; a cancelled order's reward code can be used again. Each award also lands in the Notifications Rewards tab as in the app: “You Earned 50 Points!” · “Yay! You earned 50 points for your recent order. Keep ordering, keep earning!” (star icon).

APIs in this step

#	API	Auth	Used for
S12.1	`GET /rewards/overview`	Token	Rewards tab
S12.2	`GET /rewards/history?period=this_month&page=1&limit=20`	Token	Points history
S12.3	`POST /rewards/{{rewardId}}/redeem`	Token	Redeem a reward
S12.4	`GET /rewards/redemptions`	Token	My reward codes
S12.5	`GET /promos/available`	Token	Offers include my reward codes
S12.6	`POST /cart/promo`	Token	Use a reward code at checkout
S12.7	`GET /rewards/earn-methods`	Token	Earn methods
S12.8	`GET /rewards/catalog`	Token	Rewards catalogue
S12.9	`GET /rewards/summary`	Token	Points summary
S12.10	`GET /referrals/me`	Token	My referral code
S12.11	`POST /referrals/apply`	Token	Apply a friend's code

S12.1 · Rewards tab

`GET /api/v1/rewards/overview`

Authenticated. Everything on the Rewards tab in one call.

- Points card: `points` ("2,450"), `valuePaise` (what they are worth), `tier`, `nextTier` { name, pointsNeeded }, `expiringSoon`.
- `message` + `affordableCount`: "You have enough points for 3 free rewards!" (or how many more points the cheapest reward needs).
- Redeem Rewards (`rewards`): `rewardId`, `name`, `imageUrl`, `points`, `pointsLabel` ("800 pts"), `canRedeem` (enable the Redeem button), `pointsShort` (points still needed), `kind` (dish | flat | percent | free_delivery), `dishName`, `dishQty`, `description`.
- Earn More Points (`earn`): `title`, `subtitle`, `pointsLabel` ("+50"), `icon` (the app's names: cart, user, star, gift), `points`.

The "Meal Ji Plus · Upgrade Now" banner belongs to the subscriptions step.

Response · Rewards tab (200)


```
{
  "success": true,
  "message": "Rewards.",
  "data": {
    "enabled": true,
    "points": 1300,
    "valuePaise": 32500,
    "pointValuePaise": 25,
    "lifetimePoints": 1300,
    "tier": "Silver Tandoor",
    "tiers": [
      {
        "name": "Spice Bronze",
        "threshold": 0,
        "current": false
      },
      {
        "name": "Silver Tandoor",
        "threshold": 1000,
        "current": true
      },
      {
        "name": "Gold Kadai",
        "threshold": 3000,
        "current": false
      },
      {
        "name": "Black Makhani",
        "threshold": 7500,
        "current": false
      }
    ],
    "nextTier": {
      "name": "Gold Kadai",
      "pointsNeeded": 1700
    },
    "expiringSoon": null,
    "affordableCount": 6,
    "message": "You have enough points for 6 free rewards!",
    "rewards": [
      {
        "rewardId": "6acc8c…",
        "name": "Free Butter Chicken Bowl",
        "description": "Our signature bowl, on us",
        "imageUrl": "https://cdn…/mealji/seed/rewards-butter-chicken.jpg",
        "points": 800,
  …
```

S12.2 · Points history

`GET /api/v1/rewards/history?period=this_month&page=1&limit=20`

Authenticated. The Meal Ji points screen (View History).

- Header card: `points`, `updatedLabel` (“Updated 2m ago”), `message` (“You have enough points for 3 free rewards!”).
- Filter (“This Month” dropdown): `period` = this_month | last_month | all (default all); `periods` lists the choices. A wrong value is 422.
- Recent Activity rows (`items`): `title` (“Order #MJ-8921”, “Free Butter Chicken Bowl”, “Referral Bonus”, “Review Reward”, “Birthday Treat”), `dateLabel` (“Today, 10:45 AM”, “Yesterday, 8:20 PM”, “Oct 24, 2023”), `pointsLabel` (“+50” / “-800”), `isPositive` (green/red), `typeLabel` (Earned / Redeemed / Expired / Reversed / Adjusted), `icon` (cart, gift, user, star, clock, refresh, wallet).
- `page`, `limit`, `total`, `hasMore` for scrolling.

Response · This Month (200)


```
{
  "success": true,
  "message": "Points history.",
  "data": {
    "points": 1300,
    "updatedLabel": "Updated 3h ago",
    "affordableCount": 6,
    "message": "You have enough points for 6 free rewards!",
    "period": "this_month",
    "periods": [
      {
        "key": "this_month",
        "label": "This Month"
      },
      {
        "key": "last_month",
        "label": "Last Month"
      },
      {
        "key": "all",
        "label": "All time"
      }
    ],
    "items": [
      {
        "transactionId": "6acc8b…",
        "title": "Review Reward",
        "points": 100,
        "pointsLabel": "+100",
        "isPositive": true,
        "type": "earned",
        "kind": "earned",
        "typeLabel": "Earned",
        "icon": "star",
        "source": "review",
        "referenceId": "6acc8b…",
        "dateLabel": "Today, 12:57 PM",
        "expiresAt": "2027-10-07T07:27:41.612Z",
        "createdAt": "2026-10-12T07:27:41.613Z"
      },
      {
        "transactionId": "6acc8b…",
        "title": "Order #MJ-10012",
        "points": 50,
        "pointsLabel": "+50",
        "isPositive": true,
  …
```


Response · All time (200)


```
{
  "success": true,
  "message": "Points history.",
  "data": {
    "points": 1300,
    "updatedLabel": "Updated 3h ago",
    "affordableCount": 6,
    "message": "You have enough points for 6 free rewards!",
    "period": "all",
    "periods": [
```

```
      {
        "key": "this_month",
        "label": "This Month"
      },
      {
        "key": "last_month",
        "label": "Last Month"
      },
      {
        "key": "all",
        "label": "All time"
      }
    ],
    "items": [
      {
        "transactionId": "6acc8b…",
        "title": "Review Reward",
        "points": 100,
        "pointsLabel": "+100",
        "isPositive": true,
        "type": "earned",
        "kind": "earned",
        "typeLabel": "Earned",
        "icon": "star",
        "source": "review",
        "referenceId": "6acc8b…",
        "dateLabel": "Today, 12:57 PM",
        "expiresAt": "2027-10-07T07:27:41.612Z",
        "createdAt": "2026-10-12T07:27:41.613Z"
      },
      {
        "transactionId": "6acc8b…",
        "title": "Order #MJ-10012",
        "points": 50,
        "pointsLabel": "+50",
        "isPositive": true,
  …
```


Errors to handle

Case	Status	Message shown to the user
Wrong period (422)	422	Choose This Month, Last Month or All time (field: period)

S12.3 · Redeem a reward

`POST /api/v1/rewards/{{rewardId}}/redeem`

Authenticated. The Redeem button. No body. Spends the points and returns a personal code: `couponCode`, `expiresAt`, `pointsLeft`, `howToUse` (show it in the success sheet), `reward`.

Then the customer applies the code at checkout with POST /cart/promo { code } (S12.6). A free-dish reward puts the dish in the cart by itself and takes its price off.

Errors: 409 “Not enough points” · 409 REDEEM_IN_PROGRESS (a second tap while the first is running; charged once) · 403 another tier's reward · 404 unknown / switched off · 422 bad ID.

Response · Redeemed (201) (201)


```
{
  "success": true,
  "message": "Reward redeemed. Use the code at checkout.",
  "data": {
    "redemptionId": "6accbc…",
    "reward": {
      "rewardId": "6acc8b…",
      "name": "Free Gulab Jamun",
      "description": "Added to your next order",
      "imageUrl": "https://cdn…/mealji/seed/rewards-gulab-jamun.jpg",
      "points": 300,
      "kind": "dish",
      "value": 0,
      "maxDiscountPaise": 0,
      "minOrderPaise": 0,
      "dishId": "6acc8b…",
      "dishName": null,
      "dishQty": 1,
      "pointsLabel": "300 pts",
```

```
      "validDays": 15,
      "stock": 0,
      "stockLeft": null,
      "tiers": [],
      "sortOrder": 2,
      "isActive": true
    },
    "couponCode": "RW31995A34",
    "expiresAt": "2026-10-27T10:52:51.054Z",
    "pointsLeft": 0,
    "howToUse": "Apply RW31995A34 at checkout — we add Gulab Jamun (2 pcs) to your cart …"
  }
}
```


Errors to handle

Case	Status	Message shown to the user
Not enough points (409)	409	Not enough points

S12.4 · My reward codes

`GET /api/v1/rewards/redemptions`

Authenticated. Codes this customer redeemed (latest 50): `rewardName`, `points`, `couponCode`, `expiresAt`, `status` (issued | used | expired). If the order that used a code is cancelled, the code is issued again.

Response · My codes (200)


```
{
  "success": true,
  "message": "Your rewards.",
  "data": [
    {
      "redemptionId": "6accbc…",
      "rewardName": "Free Gulab Jamun",
      "points": 300,
      "couponCode": "RW31995A34",
      "expiresAt": "2026-10-27T10:52:51.054Z",
      "status": "issued",
      "createdAt": "2026-10-12T10:52:51.093Z"
    }
  ]
}
```


S12.5 · Offers include my reward codes

`GET /api/v1/promos/available`

Authenticated. The checkout Offers list (step 08) now also lists the customer's own reward codes first: `isReward: true`, `title` (“Free Gulab Jamun”), `label`, `applicable`, `message` (e.g. “Add Gulab Jamun (2 pcs) to your cart to use this reward”), `dishId`. Other customers never see them.

Response · Offers with a reward code (200)


```
{
  "success": true,
  "message": "Offers fetched.",
  "data": [
    {
      "couponId": "6accbc…",
      "code": "RW31995A34",
      "title": "Free Gulab Jamun",
      "label": "Free Gulab Jamun",
      "description": "Added to your next order",
      "terms": [],
      "type": "flat",
      "value": 9900,
      "maxDiscountPaise": 0,
      "minOrderPaise": 0,
      "validTo": "2026-10-27T10:52:51.054Z",
      "firstOrderOnly": false,
      "paymentMethods": [],
      "isReward": true,
      "dishId": "6acc8b…",
      "applicable": false,
      "message": "Add Gulab Jamun (2 pcs) to your cart to use this reward",
      "discountPaise": 0,
```

```
      "shortByPaise": 0
    },
    {
      "couponId": "6acc8b…",
      "code": "UPI25",
      "title": "₹25 off with UPI",
      "label": "₹25 off",
      "description": "Pay with any UPI app",
      "terms": [],
      "type": "flat",
      "value": 2500,
      "maxDiscountPaise": 0,
      "minOrderPaise": 14900,
      "validTo": null,
      "firstOrderOnly": false,
      "paymentMethods": [
        "upi"
      ],
      "isReward": false,
      "dishId": null,
      "applicable": true,
      "message": "You save ₹25",
      "discountPaise": 2500,
  …
```


S12.6 · Use a reward code at checkout

`POST /api/v1/cart/promo`

Authenticated. Same call as any offer (step 08). For a free-dish reward the dish is added to the cart if it is not there yet and its price comes off (`bill.discountPaise`). Somebody else's reward code is 422 “This code is not valid”.

Request body

```
{
  "code": "{{rewardCode}}"
}
```

Response · Free dish applied (200)


```
{
  "success": true,
  "message": "Offer applied.",
  "data": {
    "cartId": "6acc8b…",
    "kitchen": {
      "kitchenId": "6acc8b…",
      "name": "Indiranagar Kitchen",
      "isOpenNow": true,
      "message": null
    },
    "items": [
      {
        "lineId": "45f896607ff0",
        "kind": "dish",
        "dishId": "6acc8b…",
        "name": "Masala Papad",
        "imageUrl": "https://cdn…/mealji/seed/masala-papad.jpg",
        "isVeg": true,
        "qty": 3,
        "portion": null,
        "mealUpgrade": null,
        "options": [],
        "specialInstructions": null,
        "unitPricePaise": 7900,
        "basePricePaise": 7900,
        "originalUnitPricePaise": 7900,
        "packagingPaise": 0,
        "totalPaise": 23700,
        "isAvailable": true,
        "issue": null,
        "priceChanged": false,
        "dishIds": [
          {
            "dishId": "6acc8b…",
            "qty": 3
          }
        ],
```

```
        "optionsText": "Regular"
      },
      {
        "lineId": "313142a64006",
        "kind": "dish",
        "dishId": "6acc8b…",
        "name": "Gulab Jamun (2 pcs)",
        "imageUrl": "https://cdn…/mealji/seed/rewards-gulab-jamun.jpg",
  …
```


Errors to handle

Case	Status	Message shown to the user
Someone else's code (422)	422	This code is not valid (field: code)

S12.7 · Earn methods

`GET /api/v1/rewards/earn-methods`

Authenticated. Only the "Earn More Points" rows (S12.1 already includes them).

Response · Earn methods (200)


```
{
  "success": true,
  "message": "Ways to earn.",
  "data": [
    {
      "key": "order",
      "title": "Place an order",
      "subtitle": "Get 50 points on every order",
      "points": 50,
      "icon": "cart",
      "description": "Every delivered order",
      "deepLink": "mealji://menu",
      "pointsLabel": "+50"
    },
    {
      "key": "referral",
      "title": "Refer a friend",
      "subtitle": "Both get 200 points",
      "points": 200,
      "icon": "user",
      "description": "When their first order is delivered",
      "deepLink": "mealji://referrals",
      "pointsLabel": "+200"
    },
    {
      "key": "review",
      "title": "Write a review",
      "subtitle": "Tell us what you love",
      "points": 100,
      "icon": "star",
      "description": "Once per delivered order",
      "deepLink": "mealji://orders?status=past",
      "pointsLabel": "+100"
    },
    {
      "key": "birthday",
      "title": "Celebrate your birthday",
      "subtitle": "A special treat from us",
      "points": 250,
      "icon": "gift",
      "description": "Add your birthday to your profile",
      "deepLink": "mealji://profile/edit",
      "pointsLabel": "+250"
    }
  ]
}
```

S12.8 · Rewards catalogue

`GET /api/v1/rewards/catalog`

Authenticated. Only the Redeem Rewards cards (S12.1 already includes them), e.g. for a “View All” screen.

Response · Catalogue (200)


```
{
  "success": true,
  "message": "Reward catalogue.",
  "data": [
    {
      "rewardId": "6acc8c…",
      "name": "Free Butter Chicken Bowl",
      "description": "Our signature bowl, on us",
      "imageUrl": "https://cdn…/mealji/seed/rewards-butter-chicken.jpg",
      "points": 800,
      "kind": "dish",
      "value": 0,
      "maxDiscountPaise": 0,
      "minOrderPaise": 0,
      "dishId": "6acc8b…",
      "dishName": "Butter Chicken Bowl",
      "dishQty": 1,
      "pointsLabel": "800 pts",
      "validDays": 30,
      "stock": 0,
      "stockLeft": null,
      "tiers": [],
      "sortOrder": 0,
      "isActive": true,
      "canRedeem": true,
      "pointsShort": 0,
      "inStock": true
    },
    {
      "rewardId": "6acc8b…",
      "name": "Free Cold Coffee",
      "description": "Added to your next order",
      "imageUrl": "https://cdn…/mealji/seed/rewards-cold-coffee.jpg",
      "points": 400,
      "kind": "dish",
      "value": 0,
      "maxDiscountPaise": 0,
      "minOrderPaise": 0,
      "dishId": "6acc8b…",
      "dishName": "Cold Coffee",
      "dishQty": 1,
      "pointsLabel": "400 pts",
      "validDays": 15,
      "stock": 0,
      "stockLeft": null,
      "tiers": [],
  …
```


S12.9 · Points summary

`GET /api/v1/rewards/summary`

Authenticated. Only the points card (S12.1 already includes it): `points`, `valuePaise`, `lifetimePoints`, `tier`, `tiers`, `nextTier`, `expiringSoon`.

Response · Summary (200)


```
{
  "success": true,
  "message": "Rewards summary.",
  "data": {
    "enabled": true,
    "points": 1300,
    "valuePaise": 32500,
    "pointValuePaise": 25,
    "lifetimePoints": 1300,
    "tier": "Silver Tandoor",
    "tiers": [
      {
        "name": "Spice Bronze",
        "threshold": 0,
```

```
        "current": false
      },
      {
        "name": "Silver Tandoor",
        "threshold": 1000,
        "current": true
      },
      {
        "name": "Gold Kadai",
        "threshold": 3000,
        "current": false
      },
      {
        "name": "Black Makhani",
        "threshold": 7500,
        "current": false
      }
    ],
    "nextTier": {
      "name": "Gold Kadai",
      "pointsNeeded": 1700
    },
    "expiringSoon": null
  }
}
```


S12.10 · My referral code

`GET /api/v1/referrals/me`

Authenticated. Refer a friend: `code`, `shareText` (ready for the share sheet), `shareLink` (null until the website exists — share `shareText`), `pointsPerReferral`, `stats` { invited, converted, pointsEarned }, `referrals` [{ name (first name), status pending | converted | rejected, joinedAt, convertedAt }]. Both sides get the points when the friend's first order is delivered.

Response · My code (200)


```
{
  "success": true,
  "message": "Referrals.",
  "data": {
    "code": "RAHUB035",
    "shareLink": null,
    "shareText": "Order home-style meals on Meal Ji with my code RAHUB035 and we both get…",
    "pointsPerReferral": 200,
    "stats": {
      "invited": 0,
      "converted": 0,
      "pointsEarned": 0
    },
    "referrals": []
  }
}
```


S12.11 · Apply a friend's code

`POST /api/v1/referrals/apply`

Authenticated. A new customer enters a friend's code (any case), before their first delivered order. Errors: 422 not a valid code · 409 own code · 409 already used a code · 409 “Referral codes are for new customers” · 422 shorter than 4 characters.

Request body


```
{
  "code": "RAHU79E7"
}
```


Errors to handle

Case	Status	Message shown to the user
Own code (409)	409	You cannot use your own code
Wrong code (422)	422	This referral code is not valid (field: code)
Existing customer (409)	409	Referral codes are for new customers

Backend tests · 17/17 passed

Test	What the app needs	Result
12-01	**Rewards tab in one call: points card, rewards to redeem, ways to earn** Rewards screen	Pass
12-02	**Earn More Points matches the screen and the points actually given** Place an order +50 · Refer a friend +200 · Write a review +100 · Birthday +250	Pass
12-03	**Redeem Rewards cards: name, live image, points, Redeem enabled only when affordable** Redeem Rewards row	Pass
12-04	**Points history: balance, "enough points for N free rewards", activity rows** Meal Ji points screen	Pass
12-05	**This Month filter and paging** "This Month" dropdown, scroll for more	Pass
12-06	**Redeem: points go down once, a personal code is issued, history shows it** Tap Redeem	Pass
12-07	**Double tap on Redeem charges once** Two taps on Redeem	Pass
12-08	**Refusals: not enough points, unknown, bad ID, another tier's reward** Redeem errors	Pass
12-09	**The reward shows in Offers for its owner only, and nobody else can use the code** Offers / Apply coupon	Pass
12-10	**Free dish reward: applying it adds the dish and takes its price off** Apply a free-dish reward at checkout	Pass
12-11	**Delivered order earns +50, titled with the order number; rating earns +100** Place an order / Write a review	Pass
12-12	**My referral code, share text and stats** Refer a friend	Pass
12-13	**A friend applies the code once; wrong, own and repeat codes are refused** Enter a referral code	Pass
12-14	**Both get +200 after the friend's first delivered order** Referral payout	Pass
12-15	**Birthday treat +250 once a year** Celebrate your birthday	Pass
12-16	**Signed out, other customers and admin-only routes are refused** Security	Pass
12-17	**Payment methods wallet card: points, "Updated …", rewards message** Meal ji wallet card	Pass

Load and scale testing (steps 01–12)

Two questions: do the APIs stay fast with a lot of data, and with many people using the app at once? Tests run on a separate database (`mealji_load`) so the demo data is never touched; SMS, Google Maps and S3 are switched off for these runs.

Large data

Data	Amount
Customers (each with a saved location)	50,021
Addresses / favourites	100,000 / 250,000
Orders (history) / notifications	200,000 / 500,000
Search history	500,000
Dishes per kitchen menu	200
Points history rows (1 customer in 100 has 1,000)	1,490,000
Reward codes / referrals	16,667 / 10,000

Every query uses an index

MongoDB's query plans on the large data: each query reads about as many records as it returns. The only wide read is "trending searches" (all searches of the last 7 days); it is cached for 10 minutes and now computed once at a time.

Query behind the API	Time	Read → returned	Result
02 sign-in: user by phone	1 ms	1 → 1	OK
03 profile: orders count	2 ms	4 → 4	OK
03 profile: favourites	1 ms	5 → 5	OK
04 addresses list	0 ms	2 → 2	OK
05 home: unread notifications	3 ms	6 → 6	OK
05 home: your usual	2 ms	4 → 4	OK
06 menu: live dishes of a kitchen	28 ms	200 → 200	OK
06/07 hearts on a dish list	1 ms	1 → 1	OK
07 favourites list	1 ms	5 → 5	OK
07 search: recent	1 ms	10 → 10	OK
07 search: typing (last minute)	1 ms	0 → 0	OK
08 cart: the customer's cart	0 ms	0 → 0	OK
08 offers: customer's past coupon use	2 ms	1 → 1	OK
08 offers: delivered orders (first-order offers)	4 ms	4 → 4	OK
11 inbox page (a tab)	1 ms	10 → 0	OK
11 unread per tab	8 ms	6 → 1	OK
12 points history page	1 ms	20 → 20	OK
12 points history: This Month	3 ms	20 → 20	OK
12 expiring soon (points card)	19 ms	80 → 1	OK
12 redeem: oldest lots first	24 ms	410 → 410	OK
12 daily job: expired lots	79 ms	5,000 → 5,000	OK
12 my reward codes	2 ms	0 → 0	OK
12 offers: public offers + my reward codes	6 ms	15 → 7	OK
12 referral code lookup	2 ms	0 → 0	OK

Query behind the API	Time	Read → returned	Result
12 my referrals	1 ms	0 → 0	OK
12 referral payout check	0 ms	0 → 0	OK
07 search: trending (7 days)	1269 ms	120,274 → 23	OK

Many users at once

A load runner signs in many customers and sends them through the step 01–07 screens with realistic weights (Home and Menu most often). Each virtual user makes calls back to back, so 25 concurrent ≈ 600–800 people actively using the app.

Database round trips per request (lower is faster)

API	Before	After
GET /home	14.3	9.1
GET /menu	6.9	2
GET /menu/items/{id}	8	2
GET /search	8	4
GET /users/me/favorites	13	5
GET /users/me	10	7
GET /users/me/preferences	11	4
POST /cart/items (add to cart)	15.3	8.2
GET /promos/available	17.8	8.1
GET /cart / POST /checkout/summary	9.1 / 9.2	4 / 4
Every signed-in call (sign-in check)	2–8	1

With step 9 (placing orders and paying) in the mix, 25 concurrent: 160 requests/s, p95 319 ms; placing an order p95 606 ms and pay + verify 650 ms (about 19 database calls each at the laptop's 2 ms per call) — just over the 500 ms write target here, expected to drop on production servers.

Same laptop, same 10 concurrent users: 44 → 129 requests/s, p95 569 → 161 ms (about 3× more capacity per server process). With step 8 added: 143 requests/s, p95 138 ms, every endpoint within target. With 8 processes: 25 concurrent ≈ 960 active users at 192 requests/s, p95 236 ms (all pass); 50 concurrent at 297 requests/s, p95 402 ms (a few write calls just over target).

Step 12 (rewards) under load — 12 Oct

- Every rewards query uses an index on 1.49 million points-history rows, also for the customer with the longest history (1,000 rows): history page 1–3 ms, “expiring soon” 19 ms, spending points (oldest first) 24 ms, the daily expiry job 79 ms per 5,000 lots.
- Found and fixed: the checkout Offers list (public offers + the customer's reward codes) read 16,674 coupons per call (114 ms); with a plain owner index it reads 15 (6 ms).
- One call at a time on the large data, the rewards screens cost the same as or less than Menu and Home: Rewards tab 25–120 ms, points history 25–89 ms, referrals 44–49 ms, wallet card 20–45 ms (Menu 64–92 ms, Home 141–156 ms) — the heaviest customer is no slower than a normal one.
- Redeem burst: 200 customers double-tapped Redeem at the same moment (400 taps) → exactly 200 redeemed and 200 refused as a second tap; nobody was charged twice, no balance went below zero, and every charge has one reward code and one history row.
- Many users at once (14 Oct, steps 01–12 mixed, one API process): 10 concurrent users → 124 requests/s, p95 150 ms; every steps 10–12 endpoint within target (orders tabs p95 112 ms, live 83 ms, tracking 79 ms, notifications 80 ms, Rewards tab 103 ms, points history 101 ms, referrals 125 ms). At 25 users one process is CPU-bound (142 requests/s): reads p95 230–330 ms, writes slower; no errors.
- Redeem and Place order, one call at a time on the large data: Redeem 61 ms median (147 ms for the customer with the longest history); spending now reads only the oldest 50 point lots. Under 25–50 users on the shared laptop (CPU also used by other projects) they reach 0.5–1.8 s: contention, not the code. 0 server errors in about 18,000 requests across today's runs.

- Repeat `npm run load:run -- --stages=25,50,100` with `npm run load:api:cluster` on an idle machine or staging to set real capacity (the cluster could not show its gain while other work used the CPU).

What was changed for scale

- The sign-in check ran once per router (up to 4 times per request); now once, with roles and permissions cached for 30 s.
- The serving kitchen, active kitchen list, app config, live Meal Ji Plus plans and resolved settings are kept in memory for a few seconds (cleared immediately on changes), instead of being read from MongoDB/Redis on every request.
- Parsed menus are kept in memory for 10 s; menu edits clear them.
- Place names for a saved location are cached per point (~10 m) for 30 days: far fewer paid Google calls.
- Profile counts are only computed for the profile; preferences no longer load them.
- Cart: dishes and combos are priced from the cached live menu (stock is still re-checked when the order is placed); the offers list no longer re-prices the whole bill; GST entity per kitchen cached 30 s.
- Two quick taps on + used to lose items (8 taps → 1 in the cart). The cart now detects a parallel save and retries on fresh data (optimistic concurrency).
- New index for a customer's coupon history (the cart reads it on every view): reads 1 record instead of scanning all redemptions.
- Points ledger: indexes for the lots that still have points (spending, “expiring soon”, the daily expiry job); coupons: a plain owner index for the Offers list with reward codes.
- One API process uses one CPU core (about 140 requests/s on the test laptop). `npm run start:cluster` runs one process per core; background jobs run once in `npm run worker`. Live updates go through Redis, so this needs no sticky sessions.

Limits of these numbers

- Measured on a shared developer laptop: MongoDB and Redis there are slow (2–3.5 ms per call; Redis runs through Docker) and other projects were using the same CPU, so the absolute numbers are low and varied between runs. With 8 processes the laptop reached 170–200 requests/s; latency targets (p95 under 300 ms for reads) were met at 10–25 concurrent users.
- Before launch, repeat `npm run load:run` on a staging server sized like production (MongoDB replica set, Redis close to the API) to set the real capacity and the number of API processes.

What changed in the backend for these steps

- App config adds `otpLength`, `otpResendSeconds` (now 30 s, matching the OTP screen), `countryCodes`, `deliveryEtaLabel`, `pickupReadyMinutes`, `servedCitiesCount`.
- Onboarding slides carry `layout`, `highlight`, `ctaLabel` and `items` so the app's custom steps are admin-managed (Content → Onboarding in the admin console).
- Login accepts "+91 90000 00031", spaces or a leading 0; an email gets a clear "mobile number only" message.
- The signed-in user and `/users/me` include `points`, `tier` and `avatarUrl`.
- Protected paths can no longer be reached by changing letter case (e.g. `/USERS/ME`).
- Profile: name needs 2+ characters; gender accepts the app's wording ("Male", "Prefer not to say").
- Place search and place details now use Google's Places API (New), which the server key supports; the old Places API was blocked for the key.
- Home adds `headline` and `searchPlaceholder` (admin-editable in Settings → App).
- Filters: `GET /dishes/filters` returns the app's quick chips and sort labels; `GET /dishes` accepts `quick=veg,fast,top_rated`, `sort=prep_time`, `relevance`, … and several cuisines (`cuisine=North Indian,Fusion`).
- About chef adds `tagline`, `quote`, `paragraphs`, `standardTitle`, `pillars`, `ctaLabel`, `ctaDeepLink` (admin: Content → About the chef).
- Images in CMS content must come from Meal Ji uploads (or the demo images) in every environment; images from other sites are refused.
- Menu / dish / combos: dishes of hidden categories disappear everywhere; dishes say why they cannot be ordered (`unavailableReason`, `unavailableMessage`, including breakfast/lunch/snacks/dinner hours); option groups say `required/multiple`; portions carry `serves`; "Make it a meal" has image, original price and saving; dish detail has `chef`, `isFavorite` and the kitchen's open state; combos carry item photos and prices, `savingsPaise`, `audience` and chips; `GET /combos/{id}` and the app's `GET /kitchens/{id}/menu` were added; every filter value is validated (422).
- Search / favourites: typo-tolerant search ("briyani"), suggestions and popular dishes for zero results, recent searches no longer store every typed letter, remove one recent search, search near a chosen point; favourites only for live dishes, 201/200 on repeat, `orderableHere` and `kitchenName`; profile `stats` (orders, favourites, addresses).
- Sign-in tokens are now pinned to HS256 and verified with a cached key.
- Veg / All is now one rule everywhere: the saved preference applies on Home, Menu, Dishes, Combos, Search, Popular and all suggestions; the toggle saves it via PATCH /users/me/preferences; `?veg=` overrides one call; responses return `vegOnly`.
- Search and filters in one call: `GET /search` accepts the same filters, sort and paging as `GET /dishes` (shared rules and validation), returns `filtersApplied`, `hasMore`, and says when the filters removed every result (`filteredOut`, `emptyMessage`).
- Notifications: items now carry the app's icon names, `group` (Today / Yesterday / This Week / Earlier) and `timeLabel`; the list returns `tabs` with unread counts and `hasMore`; `category=all` is accepted; Out for delivery uses the scooter icon; the seed adds Offers and Account examples for the demo customers.
- Order tracking and history: two taps on Submit could rate an order twice, and parallel ratings could overwrite dish/kitchen averages — now one rating per order and atomic averages. Reorder silently replaced a cart from another kitchen; it now asks (409 CART_KITCHEN_MISMATCH) unless `replaceCart: true`. Added for the screens: list `counts` (tab badges) and `hasMore`, `title`, `dateLabel`, `arrivingByLabel`, `deliveredAtLabel`, `punctualityLabel`, `canRate`, `ratingTags`, `cancelReasons`. Demo order history now has realistic delivery times.
- Place order and pay: a double tap on Pay made several orders (3 taps → 3 orders, stock taken 3 times); now one order per customer at a time (409 ORDER_IN_PROGRESS) plus Idempotency-Key replays. An old checkout paid after a retry used to charge twice with no refund; the extra payment is now refunded automatically and the order stays paid. The scheduled time comes from the validated cart (made-up times are refused). If the gateway is down the order is kept with `paymentError` and Retry payment. Wallets are offered like in the app; Order confirmed gets `placedAtLabel`, `etaLabel`, `progress`, `optionsText`, `paymentMethodLabel`, amounts paid / due; saved methods no longer expose the gateway token.
- Cart / checkout: parallel taps no longer lose items; editing a line validates portion/options and merges identical lines; unavailable combos cannot be added; the default address is used automatically; ready-to-print `billLines`, `optionsText`, `etaLabel`, `savingsMessage`, `freeDeliveryMessage`, payment method labels; Schedule for later (`scheduledFor`, only offered slots); free delivery now applies from the threshold up (₹299 or more) and Meal Ji Plus

members never see “₹X more”; profile `stats.paymentMethods` counts saved cards/UPI.

- Audit fixes (12 Oct) — money and orders: two status changes at the same moment (kitchen accepts while the order auto-cancels, cancel vs deliver) can no longer both apply; an order's events are saved together with its status; a payment arriving while the order is being cancelled is refunded; a refund is counted once even when our call and the gateway's notification arrive together; a refund that timed out is checked with the gateway before any retry (never refunded twice); a delivery cannot be booked twice; failed refunds no longer leave a cancelled order unrefunded; stock is released for the day it was taken; a late payment takes the stock again; double taps on Retry payment take stock once; with Redis down, order placement and Redeem refuse (503) instead of running without their lock.
- Audit fixes — data: `"false"` sent as text is now false (`usePoints`, `replaceCart`, `mealUpgrade`…); points spending and expiry no longer overwrite each other and expiry has no daily cap; a reward code that cannot be issued gives the points back; dish popularity counts once per delivered order; one GST invoice per order (none for cancelled-and-refunded orders); one default address per customer; a referral pays out even if a second delivery lands first.
- Audit fixes — security: the email-bounce webhook needs its token in production; changing the phone number no longer reveals whether another number is registered (and is limited to 5 a day); support attachments are served as 1-hour signed links only to people who can see the ticket; upload sizes are checked after upload; a ticket can only reference the customer's own subscription; message open/click stats count only the owner's messages; the live-updates socket checks sign-in like the API.
- Audit fixes — messages: every 422 has a plain-English `message` (the first problem); fields without their own message say what to enter instead of “Invalid value”; developer wording (“dob must be YYYY-MM-DD”, “slot: …”) is gone; change-phone accepts “+91 98765 43210”; a cart whose kitchen was switched off keeps its items with KITCHEN_UNAVAILABLE; a suspended account gets 403 “Your account is suspended…”; page is limited to 1000; invalid referral code is 422; Order details show the invoice number; the landmark is shown as typed; Home “Fast Delivery” shows the real time and features carry `imageUrl`; the Home combos card opens mealji://combos; `shareUrl`/`shareLink` are null until the website exists (`shareText` always); customer messages say “Meal Ji”; points history titles use the app's casing (“Referral Bonus”).
- Payment methods and pickup (14 Oct): each customer gets a Razorpay customer once; checkout receives `customerId`, `canSaveMethod` and `checkoutOptions`; `POST /orders` takes `savePaymentMethod` and the method is saved automatically after capture; saved UPI shows the app from its handle (Google Pay, PhonePe, Paytm…) and cards show “Expires 12/26”. The cart's `kitchen` adds `address` and `pickupReadyLabel` for the pickup card.
- Audit fixes — speed and reliability: indexes for the orders tabs and the notification tabs; delivery slots no longer price the whole cart; stock is reserved in one read and “items left” refreshes at once; trending and the reward list are cached; the nightly customer-profile job aggregates instead of reading every order; birthdays are looked up by an index; in-memory caches are cleared in every API process; failed background events show on /health/ready and /metrics and can be replayed by an admin (`GET /admin/outbox/failed`, `POST /admin/outbox/replay`); business refusals in event handlers are no longer retried 8 times; crash handlers, a 10-second shutdown limit, a worker heartbeat and restart back-off; production needs CORS_ORIGIN.
- Rewards and referrals: a free-dish reward used to issue a ₹0 coupon — it now takes the dish's price off (add-ons excluded) at whichever kitchen serves the customer, and applying it adds the dish to the cart. Reward codes belong to the customer who redeemed them (others get 422), show first in Offers, and come back if their order is cancelled. A double tap on Redeem is refused while the first runs. Two parallel referral-code taps no longer risk a server error. Added for the screens: `GET /rewards/overview` (whole tab in one call), earn rows in the app's wording with icons and “+50” labels, `pointsLabel`/`pointsShort`/`dishName`, history `dateLabel`, `pointsLabel`, `typeLabel`, `icon`, `updatedLabel`, the This Month / Last Month filter, and history titles “Order #MJ-…”, “Review Reward”, “Referral Bonus”, “Birthday Treat”. Points notifications use the app's wording (“You Earned 50 Points!” with a line per reason) and earn rows carry `deepLink`. The demo rewards now match the app (Butter Chicken Bowl 800, Cold Coffee 400, Gulab Jamun 300, Garlic Naan ×2 500). New ledger indexes keep spending and the daily expiry job fast for customers with long histories.
- Re-scan of steps 01–12 against the app (12 Oct): addresses accept phones typed as “+91 98765 43210” and no longer write “Near Near …”; categories carry `slug` (deep links like mealji:menu?category=biryani) and the Menu-tab `icon`; new banner places `combos_intro` and `combos_hero`, and `menu_hero` is now the app's “MEAL COMBOS · Explore Combos” card (all editable in the admin); dish detail adds `highlightBadges` with icons, `deepLink`/`shareUrl` and whole-meal prices for “Make it a meal” (`totalPricePaise`, `totalOriginalPricePaise`, `totalSavingsPaise`); Search's trending chips carry `type` and the “Under ₹200” chip opens the dish list with its `filters`; delivery options

read “Standard Delivery · 31 min (Today)” / “Schedule for later · Choose date & time”; orders carry `totalLabel` (“Total paid” / “To pay on delivery”), and cash counts as paid once collected; `GET /wallet` (the Payment-methods points card) is now part of step 09 with `updatedLabel` and `rewardsMessage`; the rider card has photo, `ratingLabel` (“4.9 (1,240 deliveries)”) and Call only while out for delivery; Order status adds `liveSteps` (Order received → kitchen steps → Rider heading out, “9:04 PM” / “Now”), `activityText` and `arrivingAtLabel`, and Tracking adds `etaMinutes`; past orders show `timesOrderedLabel` (“Ordered 3x”); demo ratings use the app's chips; notifications read “Order Confirmed”, “Out for Delivery” and “We'd Love Your Feedback”.

- Addresses: `label` accepts Home / Office / Other or any custom text; responses add `displayLabel`. `latitude`/`longitude` are optional — the server places the address from its text or pincode, and refuses an address it cannot find with a 422 on `pincode`.

Notes for the app team (re-scan of steps 01–12, 12 Oct)

The screens for steps 01–12 still run on sample data. When wiring them to the API, these points from the re-scan need changes in the app (the API already supports them):

- API stubs point to old paths and fields: `/auth/otp/send` → `POST /auth/login` (`countryCode + phoneNumber`), `/auth/otp/verify` → `POST /auth/verify-otp` (`userId + otp`, header `x-device-id`), `/auth/profile` and `/profile` → `/users/me`, `/profile/addresses` → `/users/me/addresses`, `/cart/sync` → the `/cart` calls of step 08, `/home/banners` and `/home/categories` → `GET /home`. Payment methods are `upi | card | netbanking | wallet | cod` (not ONLINE/COD/WALLET), with an `Idempotency-Key` header on `POST /orders`.
- Field names: dishes use `dishId`, `name`, `pricePaise` (paise, ₹1 = 100) instead of `id`, `title`, `price`; search returns `dishes`, `combos`, `categories`, `dishCount`, `searchId`; the profile counts are `stats.paymentMethods` (not `payments`); date of birth is `YYYY-MM-DD`; gender values are `male`, `female`, `other`, `prefer_not` (the API also accepts "Prefer not to say").
- Icons: the API uses the icon names from the app's own sample data — `box`, `moto`, `megaphone` (notifications) and `cart` (rewards) — but `V4Icon` does not draw them yet (it has `shopping-bag`, `bike`, `bell`, `shopping-cart`). Add these four names to `V4Icon` (or map them).
- Wording: use the API's labels instead of the hard-coded ones — Addresses show `displayLabel` ("Office", not "Work"); the Addresses footer should use `servedCitiesCount` (not "50+ cities"); the Filters sheet's cuisines come from `GET /dishes/filters` (Biryani, Healthy and Desserts there are not cuisines); order status badges use `statusLabel`; the OTP screen uses `otpLength` and `resendAfterSec`; onboarding uses `deliveryEtaLabel`.
- Screens not reachable yet: Profile setup, Checkout, Payment, Address and Order success are not in `RootNavigator`; the app has no deep-link (`linking`) setup for the `mealji://` links the API returns (banners, notifications, earn rows, dish share).
- Not built yet: Refer-a-friend and "enter a code" screens (step 12 API is ready), tip and points on the bill summary, delivery slot picker (only the API's half-hour `scheduled` windows are accepted), cancel-order sheet (`cancelReasons`), push registration (Firebase is a placeholder).
- Kept as app text (no API field): Dish detail "Comfort in every Bite / Our Signature Dish" and "Size guide", Search "No results found", the Tracking stage descriptions and "Your food is being freshly prepared." on Order confirmed. Not in v1: "Watch the Kitchen" live video and in-app chat with the rider (hide or disable those buttons).
- Share: `shareUrl` (dish) and `shareLink` (referral) are null until the Meal Ji website exists — share `shareText`. Validation errors: show `message` (it is always the first problem in plain English); `errors[]` has one entry per field.
- Save this payment method: send `savePaymentMethod: true` with POST /orders and open Razorpay checkout with `payment.checkoutOptions` (customer_id, save: 1); the card/UPI is saved after the payment. Payment methods rows use `title` / `subtitle` ("Google Pay" · "UPI linked to …", "Visa •••• 4242" · "Expires 12/26"). Pickup card: `cart.kitchen.address.fullAddress` and `pickupReadyLabel`.